Rep Examen APX

1. **¿Cuáles son los tipos de excepciones en Java?**

   a. Throwable
   b. Object
   c. Checked
   d. Unchecked
   e. Todas las opciones son correctas.

La respuesta correcta es:

a.Throwable
c. Checked
d . Unchecked

✅ Justificación:

En Java, las **excepciones** se dividen principalmente en dos tipos:

1. **Checked Exceptions (comprobadas):**
   - Se verifican en tiempo de compilación.
   - El compilador obliga a manejarlas con `try-catch` o declararlas con `throws`.
   - Ejemplos: `IOException`, `SQLException`.
2. **Unchecked Exceptions (no comprobadas):**
   - Se verifican en tiempo de ejecución.
   - Son subclases de `RuntimeException`.
   - No es obligatorio capturarlas.
   - Ejemplos: `NullPointerException`, `ArrayIndexOutOfBoundsException`.

❌ Por qué no las otras:

- **b. Object:**
  Es la clase base de todos los objetos en Java, pero no está directamente relacionada con excepciones.

- **e. Todas las opciones son correctas:**
  Incorrecto, porque `Object` no es un tipo de excepción y `Throwable` es una clase raíz, no una categoría práctica de excepción.

2. ¿Cuáles de los siguientes comandos de Git se utilizan para gestionar ramas en un repositorio? Selecciona todas las que correspondan.

a. git branch
b. git merge
c. git init
d. git checkout
e. git commit

La respuesta correcta es:

a. git branch
b. git merge
d. git checkout

✅ Justificación:

1. **`git branch`** → Se usa para **crear, listar o eliminar ramas**.

2. **`git merge`** → Se usa para **fusionar ramas**.

3. **`git checkout`** (o en versiones más nuevas `git switch`) → Se utiliza para **cambiar de rama** o restaurar archivos.

❌ Por qué no las otras:

- `git init` → Inicializa un repositorio Git, **no gestiona ramas**.
- `git commit` → Guarda cambios en el historial, **pero no maneja ramas** directamente.

3. En el contexto del versionado semántico (Semantic Versioning), si un desarrollador cambia la versión de una biblioteca de 1.4.3 a 2.0.0 ¿Que implicaciones tiene este cambio según las reglas del versionado semántico?

a. Señala un cambio en el nivel de parche, resolviendo errores sin agregar nuevas funcionalidades.
b. Ninguna opción es correcta.
c. Marca la adición de nuevas funcionalidades manteniendo la compatibilidad hacia atrás.
d. Representa cambios importantes o incompatibles con versiones anteriores, que pueden afectar la compatibilidad.
e. Indica una mejora menor que no afecta la compatibilidad hacia atrás.

La respuesta correcta es:

d. Representa cambios importantes o incompatibles con versiones anteriores, que pueden afectar la compatibilidad. ✅

✅ Justificación:

En **Semantic Versioning (SemVer)**, la versión se define como:

`MAJOR.MINOR.PATCH`

1. **MAJOR (primer dígito)** → Se incrementa cuando hay **cambios incompatibles** que pueden romper la compatibilidad con versiones anteriores.
    - Ejemplo: `1.x.x → 2.0.0`.

2. **MINOR (segundo dígito)** → Se incrementa cuando se añaden **nuevas funcionalidades compatibles hacia atrás**.
    - Ejemplo: `1.4.3 → 1.5.0`.
3. **PATCH (tercer dígito)** → Se incrementa cuando se corrigen **errores o pequeños ajustes** sin cambios de compatibilidad.
    - Ejemplo: `1.4.3 → 1.4.4`.

4. ¿Cuál de los siguientes componentes es parte de una solicitud HTTP?

a. Headers, scripts, funciones.
b. URL, cookies, archivos adjuntos.
c. Encabezados de la respuesta, cuerpo de la respuesta, estado de la respuesta.
d. URL, headers, cuerpo de la solicitud.

La respuesta correcta es:

d. URL, headers, cuerpo de la solicitud. ✅

✅ Justificación:

Una **solicitud HTTP** está compuesta por:

1. **URL (o línea de solicitud):** Indica el recurso al que se quiere acceder.
2. **Headers (encabezados):** Contienen metadatos como tipo de contenido, autenticación, cookies, etc.
3. **Cuerpo (body):** Opcional; se utiliza principalmente en métodos como `POST` o `PUT` para enviar datos al servidor.

❌ Por qué no las otras:

- **a. Headers, scripts, funciones** → Los scripts y funciones no forman parte de una solicitud HTTP.

- **b. URL, cookies, archivos adjuntos** → Aunque las cookies pueden viajar en headers, “archivos adjuntos” no es un componente formal de la solicitud.

- **c. Encabezados de la respuesta, cuerpo de la respuesta, estado de la respuesta** → Eso pertenece a la **respuesta HTTP**, no a la solicitud.

5. ¿Cuál de las siguientes opciones describe correctamente un servicio web (web service)?

a. Una aplicación o componente que se comunica con otras aplicaciones a través de la red utilizando protocolos estándar.
b. Un servicio que se utiliza exclusivamente para la transferencia de archivos grandes entre servidores.
c. Una interfaz gráfica que permite a los usuarios interactuar con una aplicación web.
d. Un servicio que se ejecuta localmente en el dispositivo del usuario sin necesidad de conectarse a una red.

La respuesta correcta es:

a. Una aplicación o componente que se comunica con otras aplicaciones a través de la red utilizando protocolos estándar. ✅

✅ Justificación:

Un **servicio web (web service)** es:

- Un **componente de software** que permite la comunicación e interoperabilidad entre distintas aplicaciones a través de la red.

- Utiliza **protocolos estándar** como **HTTP, SOAP, REST, XML, JSON**, etc.

- No depende del lenguaje de programación ni de la plataforma, lo que facilita la integración entre sistemas heterogéneos.

❌ Por qué no las otras:

- **b. Exclusivo para transferir archivos grandes** → Eso describe más un servicio de transferencia de archivos (FTP, SFTP), no un servicio web.
- **c. Una interfaz gráfica** → Eso describe una **interfaz de usuario (UI)**, no un servicio web.
- **d. Se ejecuta localmente sin red** → Contradice la esencia de un web service, que requiere red para comunicar aplicaciones.

6. ¿Cuál de las siguientes afirmaciones sobre el ciclo for mejorado introducido en Java 5 es verdadera?

a. El ciclo for mejorado solo se puede utilizar con matrices.
b. El ciclo for mejorado se puede utilizar con cualquier tipo de colección y matriz.
c. El ciclo for mejorado permite la modificación de los elementos de la colección durante la iteración.
d. El ciclo for mejorado no requiere que la colección implemente la interfaz Iterable.

La respuesta correcta es:

b. El ciclo for mejorado se puede utilizar con cualquier tipo de colección y matriz. ✅

✅ Justificación:

El **ciclo for mejorado** (o *enhanced for loop*, introducido en Java 5) tiene la forma:

```
for (Tipo variable : colección_o_arreglo) {
    // uso de variable
}
```

- Puede usarse tanto con **arreglos (arrays)** como con **colecciones** que implementen la interfaz **`Iterable`** (por ejemplo: `ArrayList`, `HashSet`, etc.).

- **No permite modificar directamente la colección** durante la iteración (para eso se usa `Iterator`).

- **Sí requiere que la colección implemente `Iterable`**, por lo que la opción **d** es incorrecta.

7. ¿Cuál de las siguientes características es fundamental en una base de datos relacional?

a. Almacenamiento de datos en formato JSON.
b. Ninguna opción es correcta.
c. Utilización de nodos y relaciones para representar datos.
d. Organización de datos en tablas con filas y columnas.
e. Almacenamiento de datos en un sistema de archivos distribuido.

La respuesta correcta es:

d. Organización de datos en tablas con filas y columnas. ✅

✅ Justificación:

En una **base de datos relacional (RDBMS)**:

- Los datos se organizan en **tablas** (también llamadas relaciones).

- Cada tabla está compuesta por **filas (tuplas)** y **columnas (atributos)**.

- Se utilizan **claves primarias y foráneas** para establecer relaciones entre tablas.

❌ Por qué no las otras:

- **a. JSON** → Propio de bases de datos NoSQL orientadas a documentos.

- **c. Nodos y relaciones** → Propio de bases de datos gráficas (ej: Neo4j).

- **e. Archivos distribuidos** → Relacionado con sistemas como Hadoop/HDFS, no con bases relacionales.

- **b. Ninguna opción** → Incorrecto, porque sí hay una opción correcta (**tablas con filas y columnas**).

8. ¿Cuál es el archivo utilizado para agregar la dependencia del JDBC Utility a una aplicación?

A. config.xml
B. config.yaml
C. pom.xml
D. pom.yaml
E. settings.xml

La respuesta correcta es:

C. pom.xml ✅

✅ Justificación:

En proyectos **Maven** (el sistema de gestión de dependencias más usado en Java):

- El archivo `pom.xml` (**Project Object Model**) se utiliza para **declarar dependencias**, como el **JDBC Utility** u otras librerías externas.

- Maven descarga automáticamente dichas dependencias desde repositorios centrales y las incluye en el proyecto.

❌ Por qué no las otras:

- **config.xml / config.yaml** → Son archivos de configuración genéricos, no se usan para dependencias en Maven.

- **pom.yaml** → No existe en Maven, solo **pom.xml**.

- **settings.xml** → Archivo de configuración global de Maven (ubicado en `.m2`), pero no sirve para declarar dependencias de un proyecto.

9. public class Test {

```
int[] array = {1, 2, 3, 4, 5};

public static void main(String[] args) {

    new Test().go(1);

}

void go(int x) {

    array[x % 5] = x++;

    go(x);

}
```

}

a. A java.lang.ArrayIndexOutOfBoundsException is thrown.
b. A java.lang.IllegalStateException is thrown.
c. A java.lang.StackOverflowError is thrown.
d. The program runs with no output.
e. Compilation fails

La respuesta correcta es:

c. A `java.lang.StackOverflowError` is thrown. ✅

✅ Justificación:

Analicemos el código:

```
public class Test {
    int[] array = {1, 2, 3, 4, 5};
    public static void main(String[] args) {
        new Test().go(1);
    }

    void go(int x) {
        array[x % 5] = x++;
        go(x);
    }
}
```

1. **Se inicializa el arreglo:** `array = {1,2,3,4,5}`.
2. Se invoca `go(1)`.
3. Dentro de `go`:
    - Se calcula `array[x % 5] = x++`.
    - Para `x=1`: `array[1 % 5] = array[1] = 1`, luego `x` se incrementa a `2`.
    - Después se llama **recursivamente** a `go(x)` → `go(2)`.
4. Esto sigue repitiéndose indefinidamente porque:
    - Siempre se hace `go(x)` con un nuevo valor de `x`.
    - No existe **condición de parada** en la recursión.

➡️ El resultado es una **recursión infinita**, que agota la memoria de la pila de ejecución y provoca un **`StackOverflowError`** en tiempo de ejecución.

❌ Por qué no las otras:

- **a. ArrayIndexOutOfBoundsException** → No ocurre porque `x % 5` siempre genera un índice válido entre `0` y `4`.
- **b. IllegalStateException** → No hay un estado ilegal en este código.
- **d. El programa corre sin output** → Cierto que no hay `System.out.println`, pero nunca termina correctamente, lanza error.
- **e. Compilation fails** → El código es válido y compila bien.

10. class Parent {

```
    public void display() {
        System.out.println("Parent");
    }
}
class Child extends Parent {
    public void display() {
        System.out.println("Child");
    }
}
public class Test {
    public static void main(String[] args) {
        Parent obj = new Child();
        ((Child) obj).display();
```

```
    }
}
```

a. Child
b. Parent
c. Compilation error
d. Runtime error

La respuesta correcta es:

a. Child ✅

✅ Justificación:

Analicemos el código:

```
Parent obj = new Child();
((Child) obj).display();
```

Se crea un objeto de tipo **Child**, pero la referencia es de tipo **Parent**.

```
Parent obj = new Child();
```

1.

Se hace un **cast explícito** a `Child`:

```
((Child) obj)
```

2.
    - Esto es válido porque **el objeto real es de tipo Child**.

3. Se llama al método `display()`.
    - Java usa **polimorfismo dinámico**, por lo que se ejecuta la versión **del objeto real**, no del tipo de referencia.

- Por eso se imprime `"Child"`.

❌ Por qué no las otras:

- **b. Parent** → Se imprimiría si se llamara desde una referencia de tipo Parent sin polimorfismo, pero aquí el objeto real es `Child`.
- **c. Compilation error** → No hay error de compilación; el cast es correcto.
- **d. Runtime error** → No ocurre `ClassCastException` porque `obj` realmente apunta a un `Child`.

11. ¿Cuál de las siguientes sentencias acerca del patrón de diseño Singleton es verdadera?

a. El patrón Singleton es usado principalmente para facilitar la herencia y el polimorfismo en la programación orientada a objetos.
b. El patrón Singleton permite que una clase sea instanciada múltiples veces y provee un punto global de acceso a la última instancia creada.
c. El patrón Singleton es usado para crear una colección de objetos que pueden ser accedidos globalmente.
d. El patrón Singleton asegura que una clase tiene sólo una instancia y provee un punto global de acceso a ella.

La respuesta correcta es:

d. El patrón Singleton asegura que una clase tiene solo una instancia y provee un punto global de acceso a ella. ✅

✅ Justificación:

El **patrón Singleton** se utiliza en programación orientada a objetos para:

1. **Garantizar que una clase tenga una sola instancia** en toda la aplicación.

2. **Proveer un punto de acceso global** a esa instancia, generalmente mediante un método estático como `getInstance()`.

Ejemplo clásico:

```
public class Singleton {
    private static Singleton instance;

    private Singleton() {} // Constructor privado

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

❌ **Por qué no las otras:**

- **a. Facilitar herencia y polimorfismo** → No es el objetivo del Singleton.
- **b. Permitir múltiples instancias** → Contradice la idea central de Singleton (solo una instancia).
- **c. Crear colección de objetos** → Eso describe más un patrón **Multiton** o un **Factory**, no un Singleton.

12. What will be the result of the following code execution involving streams?

import java.util.Arrays;

import java.util.List;

public class Test {

```
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
        long count = numbers.stream()
                            .map(n -> n * 2)
                            .filter(n -> n > 5)
                            .count();
        System.out.println(count);
    }
}
```

a. 0
b. 2
c. 3
d. 4
e. Compilation fails.

The correct answer is:

c. 3 ✅

✅ Explanation:

Let's break down the code step by step:

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

long count = numbers.stream()
                    .map(n -> n * 2)       // Step 1: Multiply each number by 2
                    .filter(n -> n > 5)    // Step 2: Keep only numbers greater than 5
```

```
                    .count();                // Step 3: Count
the remaining numbers
```

1. **Original list:** `[1, 2, 3, 4, 5]`
2. **After `map(n -> n * 2)`:** `[2, 4, 6, 8, 10]`
3. **After `filter(n -> n > 5)`:** `[6, 8, 10]`
4. **Count:** `3`

❌ Why not the others:

- **0** → Too low; there are elements > 5.
- **2 or 4** → Incorrect based on the filtering step.
- **Compilation fails** → The code is valid Java 8+ stream syntax.

13. ¿Qué comando permite enviar una solicitud de transacción a un servidor APX?

a. apx show in --project path=./TESTT001-01-ES
b. apx add req --name="TESTT001-01-ES-test.json" -y --details
c. apx send req --name="TESTT001-01-ES-test.json" -y --details
d. apx check --repair

La respuesta correcta es:

c. apx send req --name="TESTT001-01-ES-test.json" -y --details ✅

✅ Justificación:

En el contexto de **APX (Automation Platform for X transactions)**:

- `apx send req` → Se utiliza para **enviar una solicitud de transacción** al servidor APX.
- Los parámetros:
    - `--name="..."` → Especifica el archivo de la solicitud.
    - `-y` → Confirma la acción automáticamente.
    - `--details` → Muestra información detallada de la transacción enviada.

❌ **Por qué no las otras:**

- **a. apx show in ...** → Solo muestra información del proyecto, **no envía transacciones**.
- **b. apx add req ...** → Se utiliza para **agregar o registrar una solicitud**, pero no la envía.
- **d. apx check --repair** → Verifica y repara posibles errores del entorno, **no envía solicitudes**.

👉 Entonces, el comando para enviar la solicitud de transacción es `apx send req` con los parámetros indicados.

14. public class Test {

public static void main(String... arguments) {

final char a = 'A', d = 'D';

char grade = 'B';

```
        switch (grade) {
            case a:
            case 'B':
                System.out.print("B");
            case 'C':
                System.out.print("C");
            case d:
            case 'F':
                System.out.print("F");
        }
    }
}
```

a. A
b. BF
c. Compilation fails
d. C
e. ACF
f. BCF
g. B

La respuesta correcta es:

f. BCF ✅

✅ Justificación:

Analicemos el código paso a paso:

```
final char a = 'A', d = 'D';
char grade = 'B';

switch (grade) {
    case a:
    case 'B':
        System.out.print("B");
    case 'C':
        System.out.print("C");
    case d:
    case 'F':
        System.out.print("F");
}
```

1. `grade` es `'B'`.
2. El `switch` busca el `case` correspondiente:
    - `case a:` → `'A'` → no coincide
    - `case 'B':` → coincide ✅ → empieza a ejecutar desde aquí
3. En Java, **switch sin `break` ejecuta los casos siguientes ("fall-through")**:
    - Imprime `"B"`
    - Luego pasa a `case 'C'` → imprime `"C"`
    - Luego pasa a `case d / 'F'` → imprime `"F"`
4. No hay `break`, por lo que el resultado final es **BCF**.

❌ Por qué no las otras:

- **A, BF, D, ACF, B** → Ignoran el efecto "fall-through" de Java.

- **Compilation fails** → No hay error; `final char` se puede usar en `case`.

👉 Entonces, la salida del programa es **BCF**.

15. What will be the output of the following code snippet?

```
class StaticNonStaticBlockTest {
   static {
      System.out.println("Static block");
   }
   {
      System.out.println("Instance block");
   }
   StaticNonStaticBlockTest() {
      System.out.println("Constructor");
   }
   public static void staticMethod() {
      System.out.println("Static method");
   }
   public static void main(String[] args) {
      StaticNonStaticBlockTest.staticMethod();
```

```
        new StaticNonStaticBlockTest();
    }
}
```

a.Static method Static block Instance block Constructor
b.Static block Instance block Constructor Static method

R -> static block, static method, instance block, constructor

16. ¿Cual es la consola para la configuración de la aplicación en APX Online?

a. Chrome Console
b. APX Operation Console
c. Apache Felix Online Console
d. Builder Console
e. JBoss Console

La respuesta correcta es:

b. APX Operation Console ✅

✅ Justificación:

En **APX Online**, la **configuración y administración de la aplicación** se realiza desde la **APX Operation Console**, que permite:

- Visualizar el estado de los proyectos y aplicaciones.
- Configurar parámetros de ejecución.
- Monitorear transacciones y logs.
- Gestionar solicitudes y respuestas de manera centralizada.

❌ Por qué no las otras:

- **Chrome Console** → Es la consola del navegador, no sirve para administrar APX.
- **Apache Felix Online Console** → Sirve para administrar OSGi bundles, no es la interfaz principal de APX Online.
- **Builder Console** → Se usa para construir proyectos, no para configuración de aplicaciones en ejecución.
- **JBoss Console** → Consola de administración de JBoss, no específica de APX.

👉 Entonces, la consola correcta para la configuración de la aplicación en APX Online es **APX Operation Console**.

17. public class StringInternExample {

```
    public static void main(String[] args) {
        String str1 = new String("example");
        String str2 = "example";
        String str3 = str1.intern();
        System.out.println(str1 == str2);
        System.out.println(str2 == str3);
        System.out.println(str1 == str3);
    }
}
```

a.

false

true

false

b.

true

true

true

c.

false

false

false

d.

true

false

false

a
b
c
d

La respuesta correcta es:

a. ✅

✅ Justificación:

Analicemos el código:

```
String str1 = new String("example"); // Crea un objeto String en heap
String str2 = "example";             // Referencia al String literal en el String pool
String str3 = str1.intern();         // Devuelve la referencia al literal "example" del String pool
```

1. **`str1 == str2`** → **false**
    - `str1` apunta al objeto en el **heap**,
    - `str2` apunta al **String literal** en el pool.
2. **`str2 == str3`** → **true**
    - `str3` apunta al **mismo String literal** que `str2` en el pool.
3. **`str1 == str3`** → **false**
    - `str1` sigue apuntando al objeto en heap, distinto del literal.

✅ Salida del programa:

```
false
true
false
```

Por lo tanto, la opción correcta es **a**.

18. Este patrón suele representarse como una única librería que contiene la mayoría de la funcionalidad y que otras librerías utilizan incorrectamente de apoyo. La clave del problema es que puede contener un porcentaje muy elevado de responsabilidades dentro de las solución a diseñar:

Contenedor magico
Blob
CRUD
Libreria

La respuesta correcta es:

Blob ✅

✅ Justificación:

El **patrón Blob** (también llamado **God Object**) se caracteriza por:

- Concentrar **demasiadas responsabilidades** en una sola clase o módulo.
- Actuar como un **"contenedor" de funcionalidad**, mientras otras clases dependen de él.
- Generar **acoplamiento excesivo** y dificultar el mantenimiento del código.

❌ Por qué no las otras:

- **Contenedor mágico** → No es un patrón reconocido en diseño de software.
- **CRUD** → Se refiere a operaciones básicas de bases de datos (Create, Read, Update, Delete).
- **Librería** → Es un concepto general de agrupación de código, no describe la concentración excesiva de responsabilidades.

👉 Entonces, la opción correcta es **Blob**.

19. public class Test {

```
public static void op1(String s) {
    s += new String("Java");
}
public static void stringTest() {
    String msg = new String("Hello");
    op1(msg);
    System.out.println(msg);
}
public static void main(String... String) {
    stringTest();
}
```

}

What is the result?

a. HelloJava
b. Hello
c. An exception is thrown at runtime
d. Java
e. Compilation fails
f. null

The correct answer is:

b. Hello ✅

✅ Explanation:

Let's analyze the code:

```
public static void op1(String s) {
    s += new String("Java");
}

public static void stringTest() {
    String msg = new String("Hello");
    op1(msg);
    System.out.println(msg);
}
```

1. `msg` is initialized as `"Hello"`.
2. `op1(msg)` is called:
    - Inside `op1`, `s` is a **copy of the reference** to the string.
    - `s += new String("Java")` creates a **new String object**, but it **does not change** the original `msg` reference.
3. Back in `stringTest()`, `msg` still points to the original `"Hello"`.

✅ Key points:

- Strings in Java are **immutable**.
- Passing a String to a method **passes the reference by value**, so reassigning `s` inside the method does not affect `msg`.

Output:

```
Hello
```

20. Dado el siguiente código:

```
import java.util.List;
import java.util.function.Predicate;
class Person {
    String name;
    public Person(String name) {
        this.name = name;
    }
    @Override
    public String toString() {
        return name;
    }
}
public class Test {
    static List<Person> filter(List<Person> input, Predicate<Person> predicate) {
        return input.stream().filter(predicate).toList();
    }
```

```
    public static void main(String[] args) {

        var a = new Person("John");

        var b = new Person("Jane");

        var c = new Person("Mark");

        var input = List.of(a, b, c);

        var output = filter(input, p -> !p.name.startsWith("J"));

        System.out.println(output);

    }

}
```

What is the given program's output?

a. [John, Jane, Mark]
b. null
c. [Jane]
d. []
e. [Mark]
f. [John, Jane, Mark]
g. [John, Jane]
h. [John]

The correct answer is:

e. [Mark] ✅

✅ Explanation:

1. **Input list:**

```
List.of(a, b, c) → [John, Jane, Mark]
```

2. **Predicate used in `filter`:**

```
p -> !p.name.startsWith("J")
```

- Keeps **only persons whose name does NOT start with "J"**.
- `"John"` → starts with "J" → **filtered out**
- `"Jane"` → starts with "J" → **filtered out**
- `"Mark"` → does **not** start with "J" → **kept**

3. **Output list:**

```
[Mark]
```

✅ **Key points:**

- `Predicate<Person>` defines the filtering condition.
- `stream().filter(predicate).toList()` collects **only the elements that satisfy the predicate**.

So, the program prints:

```
[Mark]
```

21. ¿Cuál será el resultado de ejecutar el siguiente código?

class Parent {

public void display() {

```
        System.out.println("Parent");
    }
}
class Child extends Parent {
    public void display() {
        System.out.println("Child");
    }
}
public class Test {
    public static void main(String[] args) {
        Parent obj = new Child();
        obj.display();
    }
}
```

a. Parent
b. Error de ejecución
c. Child
d. Error de compilación

La respuesta correcta es:

c. Child ✅

✅ Justificación:

Analicemos el código:

```
Parent obj = new Child();
obj.display();
```

1. Se crea un objeto de tipo **Child**, pero la referencia es de tipo **Parent**.
2. En Java, los **métodos no estáticos se resuelven en tiempo de ejecución** mediante **polimorfismo**.
3. Por lo tanto, aunque la referencia sea de tipo `Parent`, se llama al **método `display()` de la clase real del objeto**, que es `Child`.

❌ Por qué no las otras:

- **a. Parent** → Se imprimiría solo si no hubiera polimorfismo, pero el método `display()` es sobrescrito.
- **b. Error de ejecución** → No ocurre ningún error en tiempo de ejecución.
- **d. Error de compilación** → El código es válido y compila correctamente.

👉 Resultado de la ejecución:

```
Child
```

22. public class OverloadedMethods {

public OverloadedMethods() {

this(10);

System.out.println("No-argument constructor");

```
    }
    public OverloadedMethods(int x) {
        System.out.println("Constructor with int argument: " + x);
    }
    public void calculate() {
        System.out.println("No-argument calculate method");
    }
    public void calculate(int x) {
        System.out.println("Calculate method with int argument: " + x);
    }
    public static void main(String[] args) {
        OverloadedMethods obj1 = new OverloadedMethods();
        obj1.calculate();
        obj1.calculate(5);
        OverloadedMethods obj2 = new OverloadedMethods(20);
        obj2.calculate();
        obj2.calculate(15);
    }
}
```

a. No-argument constructor Constructor with int argument: 10 No-argument calculate method Calculate method with int argument: 5 Constructor with int argument: 20 Calculate method with int argument: 15 No-argument calculate method

b. No-argument constructor Constructor with int argument: 10 Calculate method with int argument: 5 No-argument calculate method Constructor with int argument: 20 No-argument calculate method Calculate method with int argument: 15

c. Constructor with int argument: 10 No-argument constructor No-argument calculate method Calculate method with int argument: 5 Constructor with int argument: 20 No-argument calculate method Calculate method with int argument: 15

d. Constructor with int argument: 10 No-argument constructor Calculate method with int argument: 5 Constructor with int argument: 20 No-argument calculate method Calculate method with int argument: 15 No-argument calculate method

La respuesta correcta es:

c. ✅

✅ Justificación paso a paso:

1. **Creación de `obj1` con el constructor sin argumentos:**

```
OverloadedMethods obj1 = new OverloadedMethods();
```

- Llama a `this(10)` dentro del constructor sin argumentos → ejecuta:

```
Constructor with int argument: 10
```

- Luego imprime el resto del constructor sin argumentos:

```
No-argument constructor
```

2. **Llamadas a métodos de `obj1`:**

```
obj1.calculate();      // No-argument method
obj1.calculate(5);     // int argument method
```

- Salida:

```
No-argument calculate method
Calculate method with int argument: 5
```

3. **Creación de `obj2` con el constructor con argumento `20`:**

```
OverloadedMethods obj2 = new OverloadedMethods(20);
```

- Ejecuta el constructor con int:

```
Constructor with int argument: 20
```

4. **Llamadas a métodos de `obj2`:**

```
obj2.calculate();      // No-argument method
obj2.calculate(15);    // int argument method
```

- Salida:

```
No-argument calculate method
Calculate method with int argument: 15
```

✅ Salida completa del programa:

```
Constructor with int argument: 10
No-argument constructor
No-argument calculate method
Calculate method with int argument: 5
Constructor with int argument: 20
No-argument calculate method
Calculate method with int argument: 15
```

Por lo tanto, la opción correcta es **c**.

23. Which of the following is not a part of the Java Collections Framework?

a. ArrayList
b. Map
c. Dictionary
d. List

The correct answer is:

c. Dictionary ✅

✅ Explanation:

- **Java Collections Framework (JCF)** includes interfaces and classes like:
  - **List** → e.g., `ArrayList`, `LinkedList`
  - **Set** → e.g., `HashSet`, `TreeSet`
  - **Map** → e.g., `HashMap`, `TreeMap`
  - **Queue**, **Deque**, etc.
- `Dictionary`:
  - Is an **older class** (from Java 1.0) and **not part of the modern Collections Framework**.
  - Replaced by the **Map interface** and its implementations (`HashMap`, `Hashtable`).

❌ Why not the others:

- **ArrayList** → Part of `List` in Collections Framework
- **Map** → Part of Collections Framework
- **List** → Interface in Collections Framework

👉 So, the class **not part of the Java Collections Framework is `Dictionary`**.

24. Cual de los siguientes comandos de Maven se utiliza para compilar un proyecto JAR, pero sin depositarlo en el repositorio local?

a. mvn compile
b. mvn clean
c. mvn install
d. mvn package

La respuesta correcta es:

d. mvn package ✅

✅ Justificación:

En **Maven**, los comandos más relevantes para compilar y empaquetar un proyecto son:

1. **`mvn compile`** → Compila el código fuente del proyecto, pero **no genera el archivo JAR**.
2. **`mvn clean`** → Limpia los archivos generados previamente (target folder).

3. `mvn install` → Compila, empaqueta y además **instala el JAR en el repositorio local**.

4. `mvn package` → Compila y empaqueta el proyecto en un **JAR o WAR**, **pero no lo instala en el repositorio local**.

👉 Entonces, para **generar un JAR sin depositarlo en el repositorio local**, se usa `mvn package`.

25. What is the result type when a short is subtracted from a double in Java?

a. short

b. You cannot subtract a short from a double.

c. int

d. float

e. double

The correct answer is:

e. double ✅

Explanation:

In Java, arithmetic operations follow **type promotion rules**:

1. When operands of different types are used in an arithmetic operation, **the smaller type is promoted to the larger type**.

2. Here:

- `short` is promoted to `double` because the other operand is `double`.

3. Therefore, the result of `double - short` is of type **double**.

Why not the others:

- **short** → Cannot remain `short` because of type promotion.
- **int** → Only if both operands were integer types.
- **float** → `double` has higher precision; result is promoted to `double`.
- **You cannot subtract…** → The operation is valid; promotion ensures it works.

✅ Final answer: **double**

26. public class Test {

```
static boolean b;
public static void main(String[] args) {
    int x = 0;
    if (b) {
        x = 1;
    } else if (b = false) {
        x = 2;
    } else if (b) {
```

```
            x = 3;
        } else {
            x = 4;
        }
        System.out.println("x=" + x);
    }
}
```

What is the result?

a. x = 3
b. x = 0
c. Compilation fails
d. x = 4
e. x = 1

The correct answer is:

d. x = 4 ✅

✅ Explanation:

1. `b` is a **static boolean** and by default initialized to `false`.

```
static boolean b; // default false
```

2. The first `if (b)` → `if (false)` → **skipped**.

3. The second `else if (b = false)` → **assignment**, not comparison:

- `b` is assigned `false`
- The expression evaluates to `false` → **skipped**

4. The third `else if (b)` → `if (false)` → **skipped**
5. Finally, the `else` block executes:

```
x = 4;
```

6. Print statement:

```
x=4
```

✅ **Key points:**

- `b = false` is an **assignment**, not a comparison (`==`).
- Default value of static boolean is `false`.
- The `else` block runs if all previous conditions are false.

✅ Output:

```
x=4
```

27. ¿Cuál de las siguientes herramientas se utiliza principalmente para probar y desarrollar APIs, permitiendo enviar solicitudes HTTP y analizar las respuestas?

a. Git
b. Ninguna de las anteriores.

c. Artifactory
d. Docker
e. Maven

La respuesta correcta es:

b. Ninguna de las anteriores ✅

✅ Justificación:

La pregunta busca una herramienta que permita **probar y desarrollar APIs enviando solicitudes HTTP y analizando las respuestas**.

- Herramientas típicas para esto son **Postman**, **Insomnia**, **SoapUI**, etc.
- Ninguna de las opciones listadas cumple esta función directamente:
    - **Git** → Control de versiones.
    - **Artifactory** → Gestión de repositorios de artefactos.
    - **Docker** → Contenerización de aplicaciones.
    - **Maven** → Gestión de dependencias y compilación de proyectos Java.

👉 Por lo tanto, la opción correcta es **Ninguna de las anteriores**.

28. ¿Qué puede suceder al intentar ejecutar el siguiente código para establecer una conexión a una base de datos?"

```
public class TestJDBC {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/mydb";
```

```
        String user = "user";
        String password = "password";
        try {
            Connection conn = DriverManager.getConnection(url, user,
password);
            System.out.println("Connection established successfully.");
        } catch (SQLException e) {
            System.out.println("Failed to establish connection.");
            e.printStackTrace();
        }
    }
}
```

a) Establece la conexión exitosamente e imprime "Connection established successfully".
b) Lanza una ClassNotFoundException.
c) Lanza una NullPointerException.
d) Falla al establecer la conexión e imprime "Failed to establish connection."

```
29. public class OverloadedExample {
    private String name;
    public OverloadedExample() {
        this("Default");
        System.out.println("No-argument constructor called");
    }
```

```
    public OverloadedExample(String name) {
        this.name = name;
        System.out.println("Constructor with argument called: " + name);
    }
    public void display() {
        System.out.println("Name: " + name);
    }
    public void display(String prefix) {
        System.out.println(prefix + ":" + name);
    }
    public static void main(String[] args) {
        OverloadedExample obj1 = new OverloadedExample();
        obj1.display();
        obj1.display("Hello");
        OverloadedExample obj2 = new OverloadedExample("John");
        obj2.display();
        obj2.display("Hi");
    }
}
```

a.No-argument constructor called Constructor with argument called: Default Name: Default Hello:Default Constructor with argument called: John Name: John Hi:John

b.No-argument constructor called Constructor with argument called: Default Name: Default Hello:Default Constructor with argument called: John Hi:John Name: John

c.Constructor with argument called: Default No-argument constructor called Name: Default Hello:Default Constructor with argument called: John Hi:John Name: John

d.Constructor with argument called: Default No-argument constructor called Name: Default Hello:Default Constructor with argument called: John Name: John Hi:John

30. In Java, which syntax is used to declare a method parameter that accepts a variable number of arguments?

a. []
b. varargs
c. <>
d. ...
e. {}

The correct answer is:

d. ... ✅

✅ Explanation:

In Java, **varargs (variable-length arguments)** allow a method to accept **zero or more arguments** of a specified type.

Syntax example:

```
public void printNumbers(int... numbers) {
    for (int n : numbers) {
        System.out.println(n);
    }
}
```

- `int... numbers` means the method can be called with any number of integers, including none:

```
printNumbers(1, 2, 3);
printNumbers();
```

❌ **Why not the others:**

- **[]** → Used for arrays, not for variable arguments.
- **varargs** → Conceptual name, not valid Java syntax.
- **<>** → Used for generics.
- **{}** → Used for blocks or array initialization.

✅ Correct syntax: `...`

31. En JDBC APX cual es la cantidad máxima de resultados que permite obtener la consulta antes de que se deba de paginar?

1000
2000
5000
4000
3000

32. Which statement about the following code is correct?

class Base {

public Base() {

```
        System.out.println("Base constructor");
    }
    public Base(String message) {
        System.out.println("Base constructor with message: " + message);
    }
}
class Derived extends Base {
    public Derived() {
        super("Hello");
        System.out.println("Derived constructor");
    }
}
public class Test {
    public static void main(String[] args) {
        Derived derived = new Derived();
    }
}
```

a. It will print "Base constructor" followed by "Derived constructor".
b. It will print "Base constructor with message: Hello" followed by "Derived constructor".
c. It will throw a compilation error.
d. It will print "Derived constructor" followed by "Base constructor with message: Hello".

The correct answer is:

b. It will print "Base constructor with message: Hello" followed by "Derived constructor". ✅

✅ Explanation:

1. **Class hierarchy:**

```
class Derived extends Base
```

2. **Derived constructor:**

```
public Derived() {
    super("Hello");
    System.out.println("Derived constructor");
}
```

- `super("Hello")` explicitly calls the **Base constructor that takes a String**.
- This prints:

```
Base constructor with message: Hello
```

3. After calling the base constructor, the rest of the derived constructor executes:

```
Derived constructor
```

❌ Why not the others:

- **a** → Only happens if `super()` (no-arg) is called implicitly.
- **c** → No compilation errors; all constructors are valid.

- **d** → Constructors execute from base → derived, so derived message comes **after** the base constructor.

✅ Output:

```
Base constructor with message: Hello
Derived constructor
```

33. ¿Qué patrón de diseño es el más adecuado para garantizar que una clase tenga solo una instancia y proporciona un punto de acceso global a esta?

a. Strategy Pattern
b. Factory Pattern
c. Singleton Pattern
d. Observer Pattern
e. Command Pattern

La respuesta correcta es:

c. Singleton Pattern ✅

✅ Justificación:

El **patrón Singleton** se utiliza para:

- Asegurar que una clase tenga **solo una instancia** durante toda la ejecución del programa.
- Proporcionar un **punto de acceso global** a esa instancia.

Ejemplo en Java:

```
public class Singleton {
    private static Singleton instance;

    private Singleton() { } // Constructor privado
```

```
    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

❌ Por qué no las otras:

- **Strategy Pattern** → Define una familia de algoritmos intercambiables.
- **Factory Pattern** → Se usa para crear objetos sin exponer la lógica de creación.
- **Observer Pattern** → Permite que un objeto notifique cambios a otros objetos.
- **Command Pattern** → Encapsula una solicitud como un objeto para parametrizar clientes.

✅ Entonces, para **una única instancia y acceso global**, se usa **Singleton Pattern**.

34. ¿Cuál de los siguientes componentes de APX representa una entidad comercial en forma de un Bean?

a. Transacciones
b. Jobs
c. Librerias
d. Todas las opciones son correctas
e. DTOs

La respuesta correcta es:

e. DTOs ✅

✅ Justificación:

En **APX**, un **DTO (Data Transfer Object)**:

- Representa **una entidad comercial** o de negocio.
- Generalmente se implementa como un **Bean**, es decir, una clase con **atributos privados y métodos getter/setter públicos**.
- Se utiliza para **transportar datos** entre capas de la aplicación, sin lógica de negocio compleja.

❌ Por qué no las otras:

- **Transacciones** → Representan operaciones o procesos, no entidades.
- **Jobs** → Procesos o tareas programadas.
- **Librerías** → Conjuntos de funcionalidades reutilizables.
- **Todas las opciones** → Incorrecto; solo los DTOs cumplen la función de representar la entidad comercial como Bean.

✅ Por lo tanto, la opción correcta es **DTOs**.

35. Which of the following are reserved words in Java for exception handling?

a. try, catch, finally, throw, throws
b. final, static, catch, throws

c. throw & throws
d. try, catch, final, throw, throws
e. Error, Exception, RuntimeException, Throwable

The correct answer is:

a. try, catch, finally, throw, throws ✅

✅ Explanation:

In Java, the **reserved keywords for exception handling** are:

1. **try** → Used to define a block of code to test for exceptions.
2. **catch** → Used to handle exceptions thrown in the try block.
3. **finally** → Defines a block that always executes, whether an exception occurs or not.
4. **throw** → Used to explicitly throw an exception.
5. **throws** → Declares the exceptions that a method can throw.

❌ Why not the others:

- **b. final, static, catch, throws** → `final` and `static` are not exception keywords.
- **c. throw & throws** → Incomplete; missing `try`, `catch`, and `finally`.
- **d. try, catch, final, throw, throws** → `final` is not an exception handling keyword.
- **e. Error, Exception, RuntimeException, Throwable** → These are **classes**, not reserved keywords.

✅ Correct set of keywords for exception handling: **try, catch, finally, throw, throws**.

36. ¿Cuál es el propósito principal del archivo pom.xml en un proyecto Maven?

a. Para almacenar la configuración del repositorio del proyecto.
b. Para especificar la versión de Java utilizada en el proyecto.
c. Para definir las dependencias del proyecto, la configuración de compilación y los complementos.
d. Para almacenar la configuración específica del usuario.

La respuesta correcta es:

c. Para definir las dependencias del proyecto, la configuración de compilación y los complementos. ✅

✅ Justificación:

En un proyecto **Maven**, el archivo `pom.xml` (Project Object Model) cumple como **archivo principal de configuración** y permite:

1. **Definir dependencias** → Librerías externas que el proyecto necesita.
2. **Configurar compilación** → Versión de Java, directorios de compilación, etc.
3. **Configurar plugins** → Herramientas adicionales para compilación, pruebas, empaquetado, despliegue, etc.
4. **Definir información del proyecto** → Nombre, versión, descripción, desarrolladores, etc.

❌ Por qué no las otras:

- **a. Para almacenar la configuración del repositorio** → Parcial, no es su propósito principal.

- **b. Para especificar la versión de Java** → Esto se hace dentro del POM, pero es solo una parte.
- **d. Para almacenar la configuración específica del usuario** → Esto se hace en `settings.xml`, no en `pom.xml`.

✅ Por lo tanto, **pom.xml** sirve principalmente para **gestionar dependencias, compilación y plugins del proyecto**.

37. Which tool is used to package Java classes into a JAR (Java Archive) file?

a. jconsole
b. java
c. jps
d. javac
e. jar

The correct answer is:

e. jar ✅

✅ Explanation:

In Java, the `jar` tool is used to **package compiled `.class` files and other resources into a JAR (Java Archive) file**.

Example usage:

```
jar cvf MyApp.jar -C bin/ .
```

- `c` → create a new archive

- `v` → verbose output
- `f` → specify the archive file name
- `-C bin/ .` → include the compiled class files from the `bin` directory

---

❌ **Why not the others:**

- **jconsole** → Java monitoring and management tool.
- **java** → Runs Java programs, does not create JARs.
- **jps** → Lists running Java processes.
- **javac** → Compiles Java source code into `.class` files, but does not package them.

✅ Correct tool to create JARs: **`jar`**.

38. ¿Cuál de los siguientes fragmentos de código generará una ClassCastException en tiempo de ejecución?

```
// Opción a

class A {}

class B extends A {}

public class Test {

    public static void main(String[] args) {

        A obj = new B();
```

```
        A a = (A) obj;
    }
}
// Opción b
class A {}
class B extends A {}
public class Test {
    public static void main(String[] args) {
        B obj = new B();
        A a = (A) obj;
    }
}
// Opción c
class A {}
class B extends A {}
public class Test {
    public static void main(String[] args) {
        A obj = new B();
        B b = (B) obj;
    }
}
```

```
// Opción d

class A {}

class B extends A {}

public class Test {

    public static void main(String[] args) {

        A ob = new A();

        B b = (B) ob;

    }

}
```

a
b
c
d

La respuesta correcta es:

d ✅

✅ Justificación:

1. **Opción a**

```
A obj = new B();
A a = (A) obj;
```

- `obj` ya es un `B`, que **hereda de A**.
- Hacer `(A) obj` es seguro y **no lanza excepción**.

2. **Opción b**

```
B obj = new B();
A a = (A) obj;
```

- `obj` es un `B` que puede ser asignado a `A` porque `B` extiende `A`.
- **No lanza excepción**.

3. **Opción c**

```
A obj = new B();
B b = (B) obj;
```

- `obj` realmente es un `B`, por lo que la conversión `(B) obj` es válida.
- **No lanza excepción**.

4. **Opción d**

```
A ob = new A();
B b = (B) ob;
```

- `ob` **es un A puro**, no un B.
- Intentar hacer `(B) ob` **provoca una `ClassCastException` en tiempo de ejecución**.

✅ Por lo tanto, la opción que genera la excepción es **d**.

39. Which of the following operators are valid for boolean variables in Java?

a. &&
b. |
c. +
d. ||
e. ==

The correct answers are:

a. && ✅
b. | ✅
d. || ✅
e. == ✅

✅ Explanation:

In Java, boolean variables can use the following operators:

1. **Logical AND (`&&`)** → Returns `true` if **both operands** are `true`.
2. **Logical OR (`||`)** → Returns `true` if **at least one operand** is `true`.
3. **Bitwise OR (`|`)** → Can also be applied to booleans; evaluates both operands.
4. **Equality (`==`)** → Compares two boolean values for equality.

❌ Why not `+`:

- `+` is used for numeric addition or String concatenation.
- **Cannot be applied to boolean variables** in Java.

✅ Valid boolean operators: **`&&`**, **`|`**, **`||`**, **`==`**.

40. public class LoopQuestion {

```
    public static void main(String[] args) {
        int count = 0;
        for (int i = 0; i < 10; i++) {
            if (i % 2 == 0) {
                count++;
            }
        }
        System.out.println("Count: " + count);
    }
}
```

a. Compilation fails.
b. Count: 5
c. Count: 4
d. Count: 10
e. Count: 6

The correct answer is:

b. Count: 5 ✅

✅ Explanation:

1. The `for` loop iterates `i` from 0 to 9:

```
for (int i = 0; i < 10; i++)
```

2. Inside the loop, it checks if `i` is **even**:

```
if (i % 2 == 0)
```

3. If true, `count` is incremented:

```
count++;
```

4. Even numbers between 0 and 9 are: **0, 2, 4, 6, 8 → 5 numbers**.
5. Therefore, the output is:

```
Count: 5
```

✅ Output:

```
Count: 5
```

41. What is the output of the following program?

```
public class Test {
   public static void main(String[] args) {
      int x = 0;
      while (x++ < 5) {
         // Empty loop body
      }
      String message = x > 5 ? "greater than" : "false";
```

```
        System.out.println(message + " - " + x);
    }
}
```

a. false than - 6
b. greater than - 6
c. false than - 5
d. Compilation fails.
e. greater than - 5

The correct answer is:

b. greater than - 6 ✅

✅ Explanation:

1. **Initialization:**

```
int x = 0;
```

2. **While loop:**

```
while (x++ < 5) { }
```

- `x++` is **post-increment**, so the condition uses the **current value of x**, then increments it.
- Iterations:
    - x=0 → 0<5 → true → x becomes 1
    - x=1 → 1<5 → true → x becomes 2
    - x=2 → 2<5 → true → x becomes 3

- x=3 → 3<5 → true → x becomes 4
- x=4 → 4<5 → true → x becomes 5
- x=5 → 5<5 → false → loop exits → x becomes 6 after last increment

3. **Ternary operator:**

```
String message = x > 5 ? "greater than" : "false";
```

- x = 6 → x > 5 → true → message = "greater than"

4. **Print statement:**

```
System.out.println(message + " - " + x);
```

- Output:

```
greater than - 6
```

✅ Final Output:

```
greater than - 6
```

42. ¿Qué es el APX CLI?

a. Herramienta Web que permite crear el andamio de un proyecto APX.
b. Herramienta para administrar las dependencias de un proyecto APX.
c. Herramienta web que proporciona a los desarrolladores y administradores gestionar la configuración.
d. Es una interfaz de línea de comandos para realizar operaciones sobre componentes APX.

La respuesta correcta es:

d. Es una interfaz de línea de comandos para realizar operaciones sobre componentes APX. ✅

✅ Justificación:

El **APX CLI** (Command Line Interface) es:

- Una **herramienta de línea de comandos**.
- Permite a los desarrolladores y administradores **realizar operaciones sobre componentes APX**, como:
    - Enviar solicitudes de transacción.
    - Consultar información de componentes.
    - Gestionar proyectos y configuraciones desde la terminal.

❌ Por qué no las otras:

- **a** → Crear el andamio del proyecto se hace generalmente con herramientas de **generación de proyectos**, no con el CLI directamente.
- **b** → La gestión de dependencias se hace con Maven/Gradle, no con APX CLI.
- **c** → Esa función corresponde a la **APX Web Console**, no al CLI.

✅ Por lo tanto, APX CLI es **la interfaz de línea de comandos para trabajar con componentes APX**.

43. Which of the following statements about the Singleton pattern is true?

a. The Singleton pattern allows a class to be instantiated multiple times but provides a global access point to the most recently

b. The Singleton pattern ensures that a class has only one instance and provides a global point of access to it

c. The Singleton pattern is mainly used to facilitate inheritance and polymorphism in object-oriented programming.

d. The Singleton pattern is used to create a collection of objects that can be accessed globally.

The correct answer is:

b. The Singleton pattern ensures that a class has only one instance and provides a global point of access to it ✅

✅ Explanation:

The **Singleton pattern** is a **creational design pattern** that:

1. **Ensures a class has only one instance** during the lifetime of an application.
2. **Provides a global point of access** to that single instance.

Example in Java:

```
public class Singleton {
    private static Singleton instance;

    private Singleton() { } // private constructor

    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

- `getInstance()` ensures there is **only one object** of the class, accessible globally.

❌ **Why not the others:**

- **a** → Incorrect; Singleton does **not allow multiple instances**.
- **c** → Singleton is not used for inheritance/polymorphism.
- **d** → Singleton manages a **single instance**, not a collection of objects.

✅ Key takeaway: Singleton = **one instance + global access**.

44. ¿Cuáles dos de las siguientes opciones son funciones de la siguiente instrucción?

apx deploy local --runtime=online --country=ESP -y

a. Despliega el componente APX en el enfoque APX Online en el lenguaje Español.
b. Compila, empaqueta y ejecuta pruebas del componente en el que está ubicado.
c. Despliega el componente APX en el enfoque APX Online en la región de España.
d. La instrucción tiene un error.

45. ¿Cuál de las siguientes bases de datos NoSQL es conocida por su modelo de documentos, donde los datos se almacenan en formato BSON (Binary JSON)?

a. MongoDB

b. Neo4j
c. Cassandra
d. Redis

La respuesta correcta es:

a. MongoDB ✅

✅ Justificación:

- **MongoDB** es una base de datos **NoSQL orientada a documentos**.
- Almacena los datos en **documentos BSON (Binary JSON)**, lo que permite estructuras flexibles y jerárquicas.
- Soporta consultas dinámicas y escalabilidad horizontal.

❌ Por qué no las otras:

- **Neo4j** → Base de datos **orientada a grafos**, no a documentos.
- **Cassandra** → Base de datos **columnar distribuida**, basada en tablas, no en documentos.
- **Redis** → Base de datos **clave-valor en memoria**, no utiliza documentos BSON.

✅ Por lo tanto, la base de datos NoSQL con **modelo de documentos BSON** es **MongoDB**.

46. ¿Cómo solucionamos un Blob?

Realizar un refactoring del diseño entre las librerías involucradas. La clave es mover el comportamiento de este antipatrón a sus librerías relacionadas, diferenciándose por las

entidades con las que trabajan, reduciendo el acoplamiento entre librerías y simplificando los mantenimientos.

Normalmente es muy complicada de reutilizar y probar. Puede ser ineficiente o introducir excesiva complejidad si se reutiliza para pequeñas partes de su funcionalidad.

Agregar una gran cantidad de métodos “execute”.

Realizar un refactoring del diseño sin las librerías involucradas.

La respuesta correcta es:

Realizar un refactoring del diseño entre las librerías involucradas. La clave es mover el comportamiento de este antipatrón a sus librerías relacionadas, diferenciándose por las entidades con las que trabajan, reduciendo el acoplamiento entre librerías y simplificando los mantenimientos. ✅

✅ Justificación:

El **antipatrón Blob** se refiere a una clase o librería que concentra **demasiadas responsabilidades**, haciendo que:

- Sea difícil de mantener y reutilizar.
- Tenga un **alto acoplamiento** con muchas partes del sistema.
- Introduzca complejidad innecesaria.

Solución:

- **Refactorizar** el diseño **distribuyendo responsabilidades** entre librerías relacionadas.
- **Reducir acoplamiento** moviendo métodos y comportamientos hacia las librerías o clases que realmente los necesitan.
- Esto mejora la **mantenibilidad, legibilidad y testabilidad** del código.

❌ Por qué no las otras:

- **Normalmente es muy complicada de reutilizar…** → Describe el problema, no la solución.
- **Agregar una gran cantidad de métodos "execute"** → Agrava el problema, aumenta la complejidad.
- **Refactorizar sin librerías involucradas** → No resuelve el acoplamiento ni la concentración de responsabilidades.

✅ Clave: **redistribuir responsabilidades y reducir acoplamiento**.

47. ¿Cuál es la principal función del servidor de aplicaciones Jboss?

a. Proporcionar una plataforma para desplegar, ejecutar y gestionar aplicaciones empresariales Java.
b. Ofrecer un sistema de control de versiones para proyectos Java.
c. Actuar como un servicio para el desarrollo de aplicaciones HTML y CSS.
d. Proporcionar un entorno de desarrollo integrado (IDE) para aplicaciones Java.

La respuesta correcta es:

a. Proporcionar una plataforma para desplegar, ejecutar y gestionar aplicaciones empresariales Java. ✅

✅ Justificación:

El **servidor de aplicaciones JBoss (actualmente conocido como WildFly)**:

- Es un **servidor de aplicaciones Java EE / Jakarta EE**.
- Permite **desplegar, ejecutar y gestionar aplicaciones empresariales**.
- Soporta servicios como:
    - Gestión de transacciones.

- Conexiones a bases de datos.
- Mensajería.
- Seguridad y clustering.

❌ Por qué no las otras:

- **b** → Control de versiones se hace con Git, SVN, etc.
- **c** → Desarrollo de HTML/CSS no requiere JBoss.
- **d** → Un IDE sería Eclipse, IntelliJ, NetBeans, no JBoss.

✅ Función principal: **plataforma para aplicaciones empresariales Java**.

48. ¿Cuál de los siguientes comandos de Git se utiliza para crear una nueva rama y cambiarse a ella?

a. git branch nombre_de_rama
b. git checkout -m nombre_de_rama
c. git branch -d nombre_de_rama
d. git checkout -b nombre_de_rama

La respuesta correcta es:

d. git checkout -b nombre_de_rama ✅

✅ Justificación:

- `git checkout -b nombre_de_rama`:

    - Crea una nueva rama llamada `nombre_de_rama`.
    - Cambia automáticamente a esa nueva rama.
- **Otros comandos:**
    - **`git branch nombre_de_rama`** → Solo crea la rama, **no cambia a ella**.
    - **`git checkout -m nombre_de_rama`** → No es un comando válido.
    - **`git branch -d nombre_de_rama`** → Elimina la rama especificada.

✅ Forma correcta de **crear y moverse a una nueva rama**:

```
git checkout -b nombre_de_rama
```

49. ¿Cuáles de las siguientes excepciones deben ser chequeadas?

a. IOException
b. IndexOutOfBoundsException
c. InterruptedException
d. NullPointerException
e. ClassCastException
f. ArithmeticException

Las respuestas correctas son:

a. IOException ✅
c. InterruptedException ✅

✅ Justificación:

En Java, las excepciones se dividen en:

1. **Checked exceptions (excepciones chequeadas):**
    - Deben ser **declaradas o capturadas** usando `throws` o `try-catch`.
    - Ejemplos:
        - `IOException` → ocurre al trabajar con archivos, streams, redes.
        - `InterruptedException` → ocurre al manejar hilos interrumpidos.
2. **Unchecked exceptions (excepciones no chequeadas / runtime exceptions):**
    - No es obligatorio capturarlas ni declararlas.
    - Ejemplos:
        - `IndexOutOfBoundsException`
        - `NullPointerException`
        - `ClassCastException`
        - `ArithmeticException`

✅ Por lo tanto, **solo `IOException` e `InterruptedException` son checked exceptions**.

50. In a Maven proyect which file is used to configure proyect dependecies, build configuration and proyect metadata?

a. package.json
b. settings.xml
c. pom.xml

d. build.gradle
e. apx.yaml

The correct answer is:

c. pom.xml ✅

✅ Explanation:

In a **Maven project**, the `pom.xml` (Project Object Model) file is the central configuration file. It is used to:

1. **Define dependencies** → External libraries required by the project.
2. **Configure build settings** → Compilation, source/target Java versions, directories, plugins.
3. **Specify project metadata** → Name, version, description, developers, etc.
4. **Manage plugins and lifecycle** → Tasks like testing, packaging, deployment.

❌ Why not the others:

- **package.json** → Used in Node.js projects.
- **settings.xml** → Maven user-specific settings, not project dependencies.
- **build.gradle** → Used in Gradle projects.
- **apx.yaml** → Not used for Maven project configuration.

✅ Key takeaway: `pom.xml` **manages dependencies, build configuration, and project metadata in Maven**.

51. ¿Cuál será el resultado de intentar compilar y ejecutar el siguiente fragmento de código?

```
import java.util.*;

public class ComparableTest {

    public static void main(String[] args) {

        List<Item> items = new ArrayList<>();

        items.add(new Item("Apple", 3));

        items.add(new Item("Banana", 2));

        items.add(new Item("Cherry", 1));

        Collections.sort(items);

        for (Item item : items) {

            System.out.println(item.getName() + ": " + item.getPrice());

        }

    }

}


class Item implements Comparable<Item> {

    private String name;

    private int price;

    public Item(String name, int price) {

        this.name = name;

        this.price = price;
```

```
    }

    public String getName() {

        return name;

    }

    public int getPrice() {

        return price;

    }

    @Override

    public int compareTo(Item other) {

        return this.name.compareTo(other.name);

    }

}
```

a. Error de compilación debido a una implementación incorrecta de Comparable.
b. Imprimirá los artículos ordenados por precio.
c. Error de tiempo de ejecución debido a una implementación incorrecta de Comparable.
d. Imprimirá los artículos ordenados por nombre.

La respuesta correcta es:

d. Imprimirá los artículos ordenados por nombre. ✅

✅ Justificación:

1. La clase `Item` implementa correctamente la interfaz `Comparable<Item>`:

```
class Item implements Comparable<Item> {
    ...
    @Override
```

```
    public int compareTo(Item other) {
        return this.name.compareTo(other.name);
    }
}
```

- Esto indica que los objetos `Item` **se compararán según el campo** **`name`**.

2. En `main`, se crea una lista de `Item`s:

```
items.add(new Item("Apple", 3));
items.add(new Item("Banana", 2));
items.add(new Item("Cherry", 1));
```

3. Luego se ordena usando `Collections.sort(items);`

- `Collections.sort` usará el método `compareTo` definido en `Item`.
- Por lo tanto, los elementos se ordenarán **alfabéticamente por** **`name`**.

4. La salida será:

```
Apple: 3
Banana: 2
Cherry: 1
```

❌ **Por qué no las otras:**

- **a y c** → No hay error de compilación ni de ejecución; `Comparable` está correctamente implementado.
- **b** → No se ordena por `price`, sino por `name`.

✅ Conclusión: **el código imprime los artículos ordenados por nombre**.

52. Dado el siguiente código

```
public class Test {
    _____ doMath(long l, short s) {
        return s / (l * 10);
    }
}
```

¿Cuál es el tipo de retorno válido punto flotante más restringido para el método de cálculo?

a. long
b. float
c. short
d. int
e. byte
f. double

La respuesta correcta es:

b. float ✅

✅ Justificación:

1. La operación dentro del método es:

```
s / (l * 10)
```

- `s` es **short**

- `l` es **long**
- `l * 10` → **long**
- `s / (l * 10)` → Como se realiza una **división entre long y short**, Java promueve ambos a **long**, y el resultado es **long** si usamos tipos enteros.

2. El enunciado pide **tipo de retorno de punto flotante más restringido**:

- Para convertir el resultado entero a un tipo de punto flotante:
    - **float** → 32 bits
    - **double** → 64 bits
- El tipo más **restringido** (menos ancho) que puede almacenar el valor en punto flotante es **float**.

❌ Por qué no los otros:

- **long, int, short, byte** → Tipos enteros, no punto flotante.
- **double** → Correcto para punto flotante, pero no es el más restringido (es más ancho que float).

✅ Por lo tanto, el método debería declararse así:

```
public float doMath(long l, short s) {
    return s / (l * 10f); // Convertir a float para que la
división no sea truncada
}
```

53. ¿Cuál de las siguientes anotaciones en JUnit se utiliza para indicar que un método debe ejecutarse antes de cada método de prueba?

a. @Before
b. @Test
c. @JUnit
d. @BeforeClass
e. @After

La respuesta correcta es:

a. @Before ✅

✅ Justificación:

En **JUnit**:

1. `@Before`
    - Se ejecuta **antes de cada método de prueba**.
    - Útil para inicializar variables o configurar el entorno para cada test.
2. `@BeforeClass`
    - Se ejecuta **una sola vez antes de todos los tests** de la clase.
    - Debe aplicarse a métodos `static`.
3. `@Test`
    - Indica que un método es un **caso de prueba**.
4. `@After`
    - Se ejecuta **después de cada método de prueba**.
5. `@JUnit`
    - No existe como anotación.

✅ Clave: `@Before` = **método que se ejecuta antes de cada test**.

54. import java.util.ArrayList;

```
public class Test {

   public static void main(String[] args) {

      String v = new String();

      StringBuffer sb = new StringBuffer();

      class MyList extends ArrayList {

      }

      System.out.println((v instanceof String) &&

            (sb instanceof Object) && (new MyList() instanceof ArrayList));

   }

}
```

What is the result?

a. false
b. true
c. The code runs with no output.
d. Compilation fails.
e. An exception is thrown at runtime.
f. null

The correct answer is:

b. true ✅

✅ **Justification:**

1. **`v instanceof String`**
    - `v` is a `String` → `true`.
2. **`sb instanceof Object`**
    - `sb` is a `StringBuffer`, which extends `Object` → `true`.
3. **`new MyList() instanceof ArrayList`**
    - `MyList` extends `ArrayList` → `true`.
4. All three conditions are combined with `&&`:

```
(v instanceof String) && (sb instanceof Object) && (new MyList() instanceof ArrayList)
```

- `true && true && true` → **true**.

✅ **Key takeaway:**

- The `instanceof` operator returns `true` if the object is an instance of the specified class **or its subclass**.
- All conditions in this code evaluate to `true`, so the program prints:

```
true
```

55. In Java, which of the following is used to call a constructor from another constructor in the same class?

a. super()
b. new()
c. this.<class_name>()
d. this()

The correct answer is:

d. this() ✅

✅ Explanation:

In **Java**, to call **one constructor from another constructor in the same class**, you use:

```
this();
```

or

```
this(parameters);
```

- Must be the **first statement** in the constructor.
- Helps **reuse code** and avoid duplication among constructors.

❌ Why not the others:

- **super()** → Calls a constructor of the **parent class**, not the same class.
- **new()** → Invalid syntax; `new` is used to create objects.
- **this.<class_name>()** → Invalid syntax; Java does not allow this.

✅ Key takeaway: **`this()` = calls another constructor in the same class**.

56. public class Test {

```
    public static void main(String[] args) {
        String str = "a1b2c3d4e5f6";
        String[] parts = str.split("\\D");
        for (String part : parts) {
            System.out.print(part + " ");
        }
    }
}
```

a. 1 2 3 4 5 6
b. Compilation fails.
c. abcdef
d. An exception is thrown at runtime.
e. 123456
f. abcdef

The correct answer is:

a. 1 2 3 4 5 6 ✅

✅ Justification:

1. The string is:

```
String str = "a1b2c3d4e5f6";
```

2. The split uses a **regular expression**:

```
str.split("\\D")
```

- `\D` matches **any non-digit character**.
- `\\D` is the escaped version in Java strings.
- So the string is split **wherever there is a letter**, keeping the digits.

3. The result array `parts` contains:

```
["1", "2", "3", "4", "5", "6"]
```

4. The loop prints each element followed by a space:

```
1 2 3 4 5 6
```

❌ Why not the others:

- **e. 123456** → Missing spaces.
- **c/f. abcdef** → Letters are the split points, not retained.
- **b/d** → No compilation or runtime errors occur.

✅ Key takeaway: **`split("\\D")` extracts all digits from a string, ignoring non-digit characters**.

57. public class Test {

```
    static int co = 0;
    public static void main(String[] args) {
        int x = 1;
        for (test('1'); test('2') && (x <= 2); test('3')) {
            x++;
            test('4');
        }
        System.out.println(" The co value = " + co);
    }
    static boolean test(char num) {
        System.out.print("" + num);
        co++;
        return true;
    }
}
```

a. 12432432 The co value = 8
b. 1234 The co value = 4
c. 14231423 The co value = 8
d. 12341234 The co value = 8
e. 12342342 The co value = 8
f. Compilation fails

The correct answer is:

a. 12432432 The co value = 8 ✅

✅ Justification:

The `for` loop is:

```
for (test('1'); test('2') && (x <= 2); test('3')) {
    x++;
    test('4');
}
```

1. **Initialization:** `test('1')` → prints `1`, increments `co` to 1.
2. **Condition:** `test('2') && (x <= 2)`
    - First iteration: `test('2')` prints `2`, `co = 2`, `x = 1 <= 2` → true → enter loop.
3. **Loop body:**
    - `x++` → `x = 2`
    - `test('4')` → prints `4`, `co = 3`
4. **Increment step:** `test('3')` → prints `3`, `co = 4`
5. **Second iteration condition:**
    - `test('2')` → prints `2`, `co = 5`, `x = 2 <= 2` → true
    - Loop body:
        - `x++` → `x = 3`
        - `test('4')` → prints `4`, `co = 6`
    - Increment step: `test('3')` → prints `3`, `co = 7`
6. **Third iteration condition:**
    - `test('2')` → prints `2`, `co = 8`

- `x = 3 <= 2` → false → loop exits

7. **Output:**

```
12432432 The co value = 8
```

✅ Key takeaway: **`for(init; condition; increment)` executes in this order: init → condition → body → increment, repeatedly**, and method calls inside these parts are executed each time.

58. Which of the following statements about ArrayList is true?

a. ArrayList allows duplicate elements.
b. ArrayList does not maintain the insertion order of elements.
c. ArrayList elements are stored in a tree structure.
d. ArrayList can store primitive types directly.

The correct answer is:

a. ArrayList allows duplicate elements. ✅

✅ Explanation:

1. **ArrayList characteristics in Java:**
    - **Allows duplicates:** Yes, you can add the same element multiple times.
    - **Maintains insertion order:** Yes, elements are stored in the order they are added.
    - **Underlying structure:** ArrayList uses a **dynamic array**, not a tree.
    - **Primitive types:** Cannot store primitives directly; must use their **wrapper classes** (e.g., `Integer` for `int`).

❌ **Why not the others:**

- **b.** → False; ArrayList **does maintain insertion order**.
- **c.** → False; ArrayList is **array-based**, not a tree.
- **d.** → False; primitives must be wrapped (autoboxing handles this).

✅ Key takeaway: **ArrayList preserves order and allows duplicates, but stores objects, not primitives**.

59. import java.time.LocalDateTime;

import java.time.Period;

import java.time.format.DateTimeFormatter;

import java.time.format.FormatStyle;

public class Test {

public static void

main(String[] argv) {

LocalDateTime d = LocalDateTime.of(1997, 5, 15, 23, 55, 30);

Period p = Period.of(1, 2, 3);

d = d.minus(p);

```
        DateTimeFormatter f =
DateTimeFormatter.ofLocalizedDateTime(FormatStyle.SHORT);

        System.out.println(d.format(f));

    }

}
```

a. 15/05/1997

b. The code does not compile

c. 05/15/97 23:55 AM

d. 0.996875

e. A runtime exception is thrown

f. 12/3/1996

g. 12/03/96 23:55

Vamos a analizar el código paso a paso:

```
import java.time.LocalDateTime;
import java.time.Period;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;

public class Test {
    public static void main(String[] argv) {
        LocalDateTime d = LocalDateTime.of(1997, 5, 15, 23, 55,
30);
        Period p = Period.of(1, 2, 3);
        d = d.minus(p);
```

```
        DateTimeFormatter f =
DateTimeFormatter.ofLocalizedDateTime(FormatStyle.SHORT);

        System.out.println(d.format(f));

    }

}
```

1. Fecha original

```
LocalDateTime d = LocalDateTime.of(1997, 5, 15, 23, 55, 30);
```

👉 `d = 1997-05-15T23:55:30`

2. Periodo

```
Period p = Period.of(1, 2, 3);
```

👉 Esto representa **1 año, 2 meses y 3 días**.

3. Restar el periodo

```
d = d.minus(p);
```

- Restar **1 año** → `1996-05-15`
- Restar **2 meses** → `1996-03-15`
- Restar **3 días** → `1996-03-12`

👉 Resultado:
`1996-03-12T23:55:30`

4. Formateo

```
DateTimeFormatter f =
DateTimeFormatter.ofLocalizedDateTime(FormatStyle.SHORT);

System.out.println(d.format(f));
```

- `FormatStyle.SHORT` depende de la **localización**.
  En formato en **US locale**, se mostraría:
  👉 `3/12/96, 11:55 PM`

En formato **EU (ej. España/México)**, sería:
👉 `12/03/96 23:55`

✅ Respuesta correcta:

g. 12/03/96 23:55

60. ¿Cuál es el tipo de resultado cuando se resta un short de un double en Java?

a. int

b. No se puede restar un short de un double.

c. float

d. short

e. double

Muy bien, lo vemos con calma 👇

📌 Reglas de promoción numérica en Java

Cuando realizas operaciones aritméticas entre tipos primitivos en Java, el compilador **convierte automáticamente los operandos al tipo más "grande" (con mayor precisión)** de la operación.

El orden de "jerarquía" es:
`byte → short → int → long → float → double`

- Si uno de los operandos es `double`, **el otro se convierte automáticamente en `double`**.

- El resultado final será también `double`.

📌 Ejemplo práctico

```
short s = 5;

double d = 10.5;

var r = d - s;

System.out.println(r);        // 5.5

System.out.println(((Object)r).getClass().getName());
```

Salida:

```
5.5

java.lang.Double
```

👉 El `short` se promueve a `double`, y el resultado es `double`.

✅ Respuesta correcta:

e. double

🎯 Justificación:

En Java, cuando se resta un `short` de un `double`, el `short` se convierte implícitamente a `double` antes de la operación. Por lo tanto, el resultado también es de tipo **double**.

61. ¿Cuál de los siguientes métodos de Mockito se utiliza para verificar que un método de un mock ha sido llamado un número específico de veces?

a. doThrow()

b. verify()

c. doReturn()

d. Ninguna opción es correcta.

e. when() & thenReturn()

Vamos a analizar 👇

En **Mockito**, los métodos más usados son:

- `when(...).thenReturn(...)` → para definir qué debe devolver un mock cuando se llama a cierto método.
- `doReturn(...).when(...)` o `doThrow(...).when(...)` → para simular comportamientos (devolver algo o lanzar excepciones).
- `verify(...)` → para **verificar interacciones** con el mock, como cuántas veces se llamó un método.

Ejemplo:

```
verify(miMock, times(3)).miMetodo();
```

👉 Verifica que `miMetodo()` fue invocado exactamente **3 veces** sobre `miMock`.

✅ **Respuesta correcta:**
b. verify()

62. ¿Cuál de las siguientes afirmaciones es correcta sobre la aserción assertEquals en JUnit?

a. Se utiliza para verificar que una colección contiene un elemento específico.

b. Se utiliza para verificar que dos valores son iguales.

c. Se utiliza para verificar que una condición es verdadera.

d. Se utiliza para verificar que dos objetos referencian la misma instancia.

Vamos con la explicación 👇

En **JUnit**, las aserciones más comunes son:

- `assertEquals(expected, actual)` → **verifica que dos valores (u objetos, comparando con `equals()`) sean iguales**.
- `assertTrue(condition)` → verifica que una condición sea verdadera.
- `assertSame(expected, actual)` → verifica que dos referencias apunten a la **misma instancia**.
- Para verificar que una colección contenga un elemento se usaría otra librería (por ejemplo Hamcrest con `assertThat`), **no `assertEquals`**.

✅ **Respuesta correcta:**
b. Se utiliza para verificar que dos valores son iguales.

63. import java.util.Arrays;

```
public class Test {

    public static void main(String[] args) {

        String[] strings = { "2", "3", "1", "4" };

        Arrays.sort(strings, 1, 4);

        System.out.println(Arrays.toString(strings));
```

}

}

What is the given code fragment's output?

a. [2, 3, 1, 4]

b. [2, 1, 3, 4]

c. [1, 2, 3, 4]

d. [2, 4, 1, 3]

e. [4, 3, 2, 1]

Analicemos el código 👇

```
String[] strings = { "2", "3", "1", "4" };
Arrays.sort(strings, 1, 4);
System.out.println(Arrays.toString(strings));
```

Paso 1: Arreglo original

```
["2", "3", "1", "4"]
```

Paso 2: Uso de `Arrays.sort(strings, 1, 4)`

- `Arrays.sort(array, fromIndex, toIndex)` → ordena **desde `fromIndex` (incluido) hasta `toIndex` (excluido)**.
- Aquí: `fromIndex = 1`, `toIndex = 4` → se ordenan los elementos en posiciones **1, 2, 3** (`"3"`, `"1"`, `"4"`).
- El elemento en índice `0` (`"2"`) **queda igual**.

Paso 3: Orden lexicográfico de `{"3", "1", "4"}`

En orden ascendente (String): `"1"`, `"3"`, `"4"`.

Paso 4: Arreglo final

```
["2", "1", "3", "4"]
```

✅ **Respuesta correcta:**
b. [2, 1, 3, 4]

64. class Exc1 extends Exception {}

class Exc2 extends Exception {}

public class Test {

static void a() throws Exc1 {

try {

System.out.print("Try ");

} catch (Exc1 e) {

System.out.print("Catch ");

} finally {

System.out.println("Finally ");

}

}

```
    static void b() throws Exc1, Exc2 {
        throw new Exc2();
    }
    public static void main(String args[]) {
        try {
            a();
            b();
        } catch (Exc1 e) {
            System.out.println("Caught1");
        } catch (Exc2 ex) {
            System.out.print("Caught2");
        }
    }
}
```

What is the result?

a. Try Finally Caught2

b. La compilación falla

c. Try

d. Try Caught2

e. Try finally

65. En Java, ¿cuál de las siguientes instrucciones es usada para llamar a un constructor desde otro constructor en la misma clase?

a. this.<class_name>()

b. super()

c. new()

d. this()

En Java:

- `super()` → llama al **constructor de la superclase**.
- `new()` → crea un **nuevo objeto**.
- `this.<class_name>()` → ❌ **no existe** en Java.
- `this()` → ✅ se utiliza para llamar a **otro constructor de la misma clase**.

Ejemplo:

```
class Persona {
    String nombre;
    int edad;

    Persona() {
        this("Desconocido", 0);  // llama al otro constructor
    }
```

```
    Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }
}
```

✅ **Respuesta correcta:**
d. this()

66. ¿Cuáles son las utilerías que pueden ser usadas en APX Batch?

a. JDBC Utility

b. AP Connector

c. Merge/Sort

d. Neo4J

e. Document Manager

67. What will be the output of the following code snippet?

```
import java.io.Closeable;

import java.io.IOException;

class Resource implements AutoCloseable {
```

```
@Override
    public void close() {
        System.out.print("Resource closed ");
    }
    public void use() {
        System.out.print("Resource used ");
    }
}
public class Test {
    public static void main(String[] args) {
        try (Resource resource = new Resource()) {
            resource.use();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

a. Compilation error.

b. Resource used Resource closed.

c. Resource closed Resource used.

d. Runtime error.

Vamos a analizarlo paso a paso 👇

Código:

```
class Resource implements AutoCloseable {

    @Override
    public void close() {
        System.out.print("Resource closed ");
    }

    public void use() {
        System.out.print("Resource used ");
    }
}

public class Test {
    public static void main(String[] args) {
        try (Resource resource = new Resource()) {
            resource.use();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
```

```
}
```

🔎 Análisis

1. `Resource` implementa `AutoCloseable`, lo cual permite usarlo en un **try-with-resources**.
    - ✅ Esto compila correctamente.
2. En el bloque `try`:
    - Se crea el recurso → `new Resource()`.

Se llama a `resource.use()` → imprime:

```
Resource used
```

    -
3. Al salir del bloque `try`, automáticamente se llama al método `close()` del recurso:

Imprime:

```
Resource closed
```

    -
4. No se lanza ninguna excepción → el `catch` no se ejecuta.

🖨️ Salida final en consola:

```
Resource used Resource closed
```

✅ **Respuesta correcta:**
b. Resource used Resource closed.

68. En el contexto del patrón de diseño de Comando (Command Pattern), ¿cuál es el propósito de la clase invocadora (Invoker)?

a. Crear objetos que representan acciones y sus parámetros.

b. Definir una interfaz común para todas las clases que representan comandos.

c. Mantener la lógica de los objetos comando y ejecutar el comando solicitado en el momento adecuado.

d. Ejecutar un comando y, opcionalmente, almacenar y deshacer el comando si es necesario.

Vamos a analizar el **Command Pattern** paso a paso 👇

Componentes principales del patrón de diseño Command:

1. **Command (Comando)**
    - Define una **interfaz común** con un método `execute()`.
    - Representa la acción a realizar.
2. **ConcreteCommand (Comando Concreto)**
    - Implementa la interfaz `Command`.
    - Conoce el **receptor** y llama a sus métodos para ejecutar la acción.
3. **Receiver (Receptor)**
    - Es el objeto que sabe cómo realizar la acción concreta.
4. **Invoker (Invocador)**
    - ✅ **Mantiene y controla los objetos comando**.
    - Decide **cuándo ejecutar un comando**, llamando a `execute()` del comando.
    - Puede almacenar comandos para permitir **deshacer o repetir acciones**.

5. **Client (Cliente)**
    - Crea los objetos `ConcreteCommand` y los asocia con un `Receiver`.
    - Le pasa los comandos al `Invoker`.

Pregunta: propósito de la clase invocadora (Invoker)

- **Opción a**: ❌ Esto lo hace el **Client**.
- **Opción b**: ❌ Esto lo hace la **interfaz Command**, no el invocador.
- **Opción c**: ✅ Correcto.

  Mantiene la lógica de los objetos comando y ejecuta el comando solicitado en el momento adecuado.

- **Opción d**: ⚠️ Parcialmente correcto; el invocador **puede** almacenar y deshacer comandos, pero su propósito principal es **ejecutar comandos en el momento adecuado**.

✅ Respuesta correcta:

c. Mantener la lógica de los objetos comando y ejecutar el comando solicitado en el momento adecuado.

Justificación:
El `Invoker` no sabe cómo se realiza la acción, solo sabe **cuándo y qué comando ejecutar**. La lógica concreta de la acción reside en el `Receiver` y el `ConcreteCommand`.

69.¿Cuál es el propósito principal del patrón de paginación en el desarrollo de aplicaciones web?

Mejorar la seguridad de la aplicación al evitar ataques de inyección SQL.

Incrementar la portabilidad del software al utilizar estándares de codificación universal.

Facilitar la integración con servicios externos mediante el uso de API RESTful.

Optimizar el rendimiento y la experiencia del usuario al dividir grandes conjuntos de datos en páginas más pequeñas.

Vamos a analizarlo:

El **patrón de paginación** en aplicaciones web se utiliza principalmente cuando tienes **grandes volúmenes de datos**.

- En lugar de cargar todos los registros a la vez (lo que puede consumir mucha memoria y hacer lenta la aplicación), la paginación:
    - Divide los datos en **páginas más pequeñas**.
    - Permite que el usuario vea solo una parte del conjunto de datos a la vez.
    - Mejora el **rendimiento** y la **experiencia de usuario**.

Las otras opciones:

- Seguridad (`inyección SQL`) → no es el objetivo principal de la paginación.
- Portabilidad → tampoco se relaciona con paginación.
- Integración con APIs REST → aunque la paginación se usa en APIs, su propósito principal sigue siendo **optimizar rendimiento**, no integrar servicios.

✅ **Respuesta correcta:**
Optimizar el rendimiento y la experiencia del usuario al dividir grandes conjuntos de datos en páginas más pequeñas.

70. public class NestedLoopTest {

```
    public static void main(String[] args) {
        for (int i = 1; i <= 3; i++) {
            for (int j = 1; j <= 2; j++) {
                System.out.println("i."+i + "," + "j."+j);
            }
        }
    }
}
```

a)

i.1,j.1

i.1,j.2

i.2,j.1

i.2,j.2

i.3,j.1

i.3,j.2

b)

i.1,j.1

i.2,j.2

i.3,j.3

c)

i.1,j.1

i.1,j.2

i.1,j.3

i.2,j.1

i.2,j.2

i.2,j.3

d)

i.1,j.1

i.2,j.1

i.3,j.1

a

b

c

d

Analicemos el código paso a paso 👇

```
for (int i = 1; i <= 3; i++) {        // i = 1, 2, 3
    for (int j = 1; j <= 2; j++) {    // j = 1, 2
        System.out.println("i."+i + "," + "j."+j);
    }
}
```

- **Iteraciones:**

1. **i = 1**
    - j = 1 → imprime `i.1,j.1`
    - j = 2 → imprime `i.1,j.2`
2. **i = 2**
    - j = 1 → imprime `i.2,j.1`
    - j = 2 → imprime `i.2,j.2`
3. **i = 3**
    - j = 1 → imprime `i.3,j.1`
    - j = 2 → imprime `i.3,j.2`

- **Salida final:**

```
i.1,j.1
i.1,j.2
i.2,j.1
i.2,j.2
i.3,j.1
i.3,j.2
```

✅ **Respuesta correcta:**
a

71. ¿Cuál es la salida del siguiente código?

```
import java.time.LocalDate;

import java.time.Month;

public class Test {

    public static void main(String[] argv) {

        LocalDate date = LocalDate.of(1997, Month.MAY, 50);

        System.out.println(date.getYear() + "-" + date.getMonth()

            + "-" + date.getDayOfMonth());

    }

}
```

a. 1997-MAY-15

b. 1997-JUN-15

c. Se genera una excepción en tiempo de ejecución.

d. El código no compila.

e. 14-JUN-1997

Analicemos el código paso a paso 👇

```
LocalDate date = LocalDate.of(1997, Month.MAY, 50);
```

- `LocalDate.of(int year, Month month, int dayOfMonth)`
- El **día 50 de mayo** no existe.
- Java **no ajusta automáticamente** días que excedan el mes, a diferencia de algunas librerías.

Por lo tanto, al intentar crear esta fecha, **se lanza una excepción en tiempo de ejecución**:

```
java.time.DateTimeException: Invalid date 'MAY 50'
```

-

✅ **Respuesta correcta:**
c. Se genera una excepción en tiempo de ejecución.

72. Which of the following lines of code correctly uses a method from the String class to replace all occurrences of the character 'a' with 'e' in a given string in Java?

a. String result = "banana".replace('a', 'e');

b. String result = "banana".change('a', 'e');

c. String result = "banana".modify('a', 'e');

d. String result = "banana".substitute('a', 'e');

e. String result = "banana".swap('a', 'e');

En Java, la clase `String` tiene el método **`replace`** para reemplazar caracteres o secuencias de caracteres:

```
String result = "banana".replace('a', 'e');
System.out.println(result);  // imprime "benene"
```

Las otras opciones (`change`, `modify`, `substitute`, `swap`) **no existen** en la clase `String`.

✅ **Respuesta correcta:**
a. String result = "banana".replace('a', 'e');

73. Consider the following Java Code:

```
class Animal {
    public void makeSound() {
        System.out.println("Sound");
    }
    public void makeSound(String sound) {
        System.out.println(sound);
    }
}
class Dog extends Animal {
```

```
    @Override
    public void makeSound() {
        System.out.println("Bark");
    }
    public void makeSound(int times) {
        for (int i = 0; i < times; i++) {
            System.out.println("Bark");
        }
    }
}
public class Test {
    public static void main(String[] args) {
        Animal animal = new Animal();
        Animal dog = new Dog();
        animal.makeSound();
        animal.makeSound("Roar");
        dog.makeSound();
```

```
        dog.makeSound("Growl");

    }

}
```

What will be the output of the above code?

a. Sound Roar Bark Roard

b. Sound Roar Bark Growl

c. Sound Roar Bark Bark

d. Sound Roar Sound Growl

Vamos a analizar el código paso a paso 👇

Clases

```
class Animal {

    public void makeSound() { System.out.println("Sound"); }

    public void makeSound(String sound) {
System.out.println(sound); }

}


class Dog extends Animal {

    @Override

    public void makeSound() { System.out.println("Bark"); }

    public void makeSound(int times) { for (int i=0;i<times;i++)
System.out.println("Bark"); }

}
```

- `Animal` tiene:
    - `makeSound()` → imprime `"Sound"`
    - `makeSound(String)` → imprime el string recibido
- `Dog` **sobrescribe** `makeSound()`
    - `makeSound()` → imprime `"Bark"`
    - Tiene un **overload** `makeSound(int)` → imprime `"Bark"` varias veces

Main

```
Animal animal = new Animal();

Animal dog = new Dog();


animal.makeSound();          // Animal → makeSound() → "Sound"

animal.makeSound("Roar");    // Animal → makeSound(String) → "Roar"


dog.makeSound();             // Dog sobrescribe makeSound() →
"Bark"

dog.makeSound("Growl");      // Animal tiene makeSound(String) →
"Growl"
```

Importante:

- Aunque `dog` es de tipo `Animal`, el objeto real es `Dog`.
- Para **overriding**, se llama al método de la clase real (`Dog`) → `makeSound()`.
- Para **overloading** (`makeSound(String)`), se decide por el **tipo de referencia** → `Animal` → `makeSound(String)` de `Animal`.

Salida final:

```
Sound
Roar
Bark
Growl
```

✅ **Respuesta correcta:**
b. Sound Roar Bark Growl

74. APX proporciona a las aplicaciones las utilidades para ejecutar una consulta solo a base de datos relacional. Seleccione una:

Verdadero

Falso

La afirmación es **falsa**.

Justificación:

- APX no se limita únicamente a bases de datos relacionales.
- Proporciona utilidades que permiten interactuar con **diferentes tipos de fuentes de datos**, incluyendo bases relacionales, documentos, y otras fuentes.
- Por lo tanto, no es correcto decir que solo se ejecutan consultas a bases relacionales.

✅ **Respuesta:**
Falso

75. Resultado de la operacion 9 | 4

13 --> System.out.println(9| 4);

Vamos a analizarlo paso a paso:

La operación | en Java es el **OR bit a bit**.

- Números en binario:

```
9  -> 1001
4  -> 0100
```

- Operación OR (|): se pone 1 si **alguno de los bits** es 1:

```
1001
0100
----
1101  -> 13 en decimal
```

✅ Resultado:

```
13
```

76. Which of the following are valid ways to initialize a boolean variable in Java?

a. boolean flag = Boolean.FALSE;

b. boolean flag = true;

c. boolean flag = 0;

d. boolean flag = false;

e. boolean flag = "false";

77. Which is the primary purpose of the pom.xml file in a Maven project?

a. To specify the Java version used in the project.

b. To manage the project's repository settings.

c. To define the project's dependencies, build configuration, and plugins.

d. To store user-specific configuration.

En **Maven**, el archivo `pom.xml` (Project Object Model) es **el corazón del proyecto** y tiene los siguientes propósitos:

- Definir **dependencias** que el proyecto necesita.
- Configurar el **proceso de construcción (build)**.
- Especificar **plugins** para tareas adicionales (compilación, empaquetado, tests, etc.).

Las otras opciones son parciales o incorrectas:

- a) La versión de Java puede especificarse, pero no es el propósito principal.
- b) Repositorios pueden definirse, pero no es lo central.
- d) Configuración de usuario se guarda en `settings.xml`, no en `pom.xml`.

✅ **Respuesta correcta:**
c. To define the project's dependencies, build configuration, and plugins.

78. public class Test {

```
    public static void main(String[] args) {
        try {
            mf();
            System.out.print("A");
        } catch (Exception e) {
            System.out.print("B");
        } finally {
            System.out.print("C");
        }
        System.out.print("D");
    }
    static void mf() {
    }
}
```

What is the result?

a. ABCD

b. ACD

c. AC

d. D

e. compilation fails

f. BCD

Analicemos el código paso a paso 👇

```
public class Test {
    public static void main(String[] args) {
        try {
            mf();
            System.out.print("A");
        } catch (Exception e) {
            System.out.print("B");
        } finally {
            System.out.print("C");
        }
        System.out.print("D");
    }

    static void mf() {
    }
}
```

Paso 1: Llamada a `mf()`

- `mf()` **no lanza ninguna excepción**.
- Por lo tanto, no se entra al bloque `catch`.

Paso 2: Bloque `try`

- Después de `mf()`, se imprime `"A"` → salida parcial: `"A"`

Paso 3: Bloque `finally`

- Siempre se ejecuta → imprime `"C"` → salida parcial: `"AC"`

Paso 4: Después del `try-catch-finally`

- Se imprime `"D"` → salida final: `"ACD"`

✅ **Respuesta correcta:**
b. ACD

79. En Java, ¿qué sintaxis es usada para declarar un parámetro en un método que acepte un número variable de argumentos?

a. <>

b. ...

c. []

d. varargs

e. {}

En Java, para declarar un parámetro que acepte un **número variable de argumentos** se utiliza la sintaxis `...`, conocida como **varargs**.

Ejemplo:

```
public void imprimirNumeros(int... nums) {
    for (int n : nums) {
        System.out.println(n);
    }
}
```

- Aquí, `nums` puede recibir cualquier cantidad de enteros (`0, 1, o más`).

✅ **Respuesta correcta:**
b. ...

80. ¿Qué es un Job en el contexto de Spring Batch?

a. Una clase que gestiona la persistencia de datos en la base de datos.

b. Una entidad que encapsula todo el proceso por lotes, configurada a través de un archivo XML o una configuración basada en Java.

c. Un componente que ejecuta tareas individuales dentro de un Step.

d. Una interfaz que define métodos para realizar operaciones CRUD en los datos del JobRepository.

En **Spring Batch**, los conceptos principales son **Job** y **Step**:

- **Job**
    - ✅ Encapsula **todo un proceso por lotes**.
    - Puede contener **uno o varios Steps**.
    - Se configura mediante **XML** o **Java Configuration**.

- **Step**
    - Representa una **tarea individual** dentro del Job.

Las otras opciones:

- a) ❌ Gestionar persistencia de datos lo hace el `ItemReader`/`ItemWriter`.
- c) ❌ Ejecuta tareas individuales → eso es un **Step**, no el Job.
- d) ❌ Interfaz para operaciones CRUD → eso lo hace `JobRepository`, no el Job.

✅ **Respuesta correcta:**
b. Una entidad que encapsula todo el proceso por lotes, configurada a través de un archivo XML o una configuración basada en Java.

81. Give

```
public class Test {

   public static void main(String[] args) {

      float f = 3.14f;

      new Test().dolt(f);

   }

   void dolt(Float f) {

      System.out.println("Float");

   }
```

```
    void doIt(double d) {
        System.out.println("double");
    }
    void doIt(float... fs) {
        System.out.println("varargs");
    }
}
```

What is the result?

a. Compilations fails.

b. An exception is thrown at runtime.

c. The output is not predictable.

d. Float

e. varargs

f. double

Vamos a analizar la llamada a `doIt(f)` paso a paso 👇

Código clave:

```
float f = 3.14f;
new Test().doIt(f);
```

Métodos disponibles en `Test`:

1. `void doIt(Float f)` → acepta **objeto Float** (boxing).

2. `void doIt(double d)` → acepta **primitivo double** (widening).
3. `void doIt(float... fs)` → acepta **varargs de float**.

Resolución de sobrecarga

Java sigue un **orden de preferencia** cuando hay sobrecarga:

1. **Exact match**
2. **Widening primitive conversion**
3. **Boxing**
4. **Varargs**

- Tenemos `float f` (primitivo).
- Revisando las opciones:
    1. `Float f` → requiere **boxing** (de `float` a `Float`).
    2. `double d` → requiere **widening** (`float` → `double`).
    3. `float... fs` → varargs (menos preferido que boxing o widening).

Regla: Para sobrecarga, **widening tiene prioridad sobre boxing**.

- Por lo tanto, `doIt(double d)` será elegido.

✅ Salida

```
double
```

Respuesta correcta:
f. double

82. ¿Cuál de los siguientes componentes de APX encapsula la lógica empresarial y el acceso a los datos, además publica su interfaz para que pueda ser usada por terceros?

a. Librerías

b. DTOs

c. Jobs

d. Todas las opciones son correctas.

e. Transacciones

Analicemos cada opción en el contexto de **APX**:

- **Librerías** → Contienen código reutilizable, pero no necesariamente exponen lógica empresarial para terceros.
- **DTOs (Data Transfer Objects)** → Solo transportan datos, **no encapsulan lógica**.
- **Jobs** → Ejecutan procesos batch, **no necesariamente exponen interfaz para terceros**.
- **Transacciones** → Gestionan la atomicidad de operaciones, **no encapsulan lógica empresarial**.

El componente que sí **encapsula la lógica empresarial, maneja acceso a datos y expone una interfaz para terceros** son **las librerías de APX que implementan los servicios o procesos reutilizables**.

✅ **Respuesta correcta:**
a. Librerías

83. Which of the following statements accurately describe the relationships that can exist between classes in Java?

Inheritance represents an "is-a" relationship where one class derives from another class.

Composition represents a "has-a" relationship where one class contains an instance of another class.

Inheritance should be preferred over composition to promote code reuse and flexibility.

Both inheritance and composition can be used together to model complex relationships.

Usage (or association) represents a "uses-a" relationship where one class uses methods or instances of another class.

Composition should be preferred over inheritance to promote code reuse and flexibility.

Analicemos cada afirmación paso a paso:

1. **Inheritance represents an "is-a" relationship where one class derives from another class.** ✅
    - Correcto. La herencia define que un objeto **"es un"** tipo de otro (por ejemplo, `Dog` **is-a** `Animal`).
2. **Composition represents a "has-a" relationship where one class contains an instance of another class.** ✅
    - Correcto. La composición indica que un objeto **"tiene un"** otro objeto como parte de su estado.
3. **Inheritance should be preferred over composition to promote code reuse and flexibility.** ❌
    - Incorrecto. La **composición** suele ser preferida sobre herencia para mayor flexibilidad y desacoplamiento.
4. **Both inheritance and composition can be used together to model complex relationships.** ✅
    - Correcto. En diseños complejos se pueden combinar ambas técnicas.
5. **Usage (or association) represents a "uses-a" relationship where one class uses methods or instances of another class.** ✅
    - Correcto. Una asociación indica que una clase **usa** otra, sin poseerla necesariamente.

6. **Composition should be preferred over inheritance to promote code reuse and flexibility.** ✅

    - Correcto. Es una buena práctica de diseño: "**favor composition over inheritance**".

✅ Resumen de afirmaciones correctas:

1, 2, 4, 5, 6

84. import java.io.Closeable;

import java.io.IOException;

class Resource implements AutoCloseable {

@Override

public void close() {

System.out.print("Resource closed ");

}

public void use() {

System.out.print("Resource used ");

}

}

```
public class Test {

    public static void main(String[] args) {

        try (Resource resource = new Resource()) {

            resource.use();

        } catch (Exception e) {

            e.printStackTrace();

        }

    }

}
```

a. Compilation error.

b. Resource used Resource closed.

c. Resource closed Resource used.

d. Runtime error.

Analicemos el código paso a paso 👇

```
class Resource implements AutoCloseable {
    @Override
    public void close() {
        System.out.print("Resource closed ");
    }

    public void use() {
```

```
        System.out.print("Resource used ");
    }
}

public class Test {
    public static void main(String[] args) {
        try (Resource resource = new Resource()) {
            resource.use();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

🔹 Análisis

1. `Resource` implementa `AutoCloseable`, por lo que puede usarse en un **try-with-resources**.
2. Dentro del `try`, se llama a `resource.use()` → imprime `"Resource used "`.
3. Al finalizar el bloque `try`, se llama automáticamente a `close()` → imprime `"Resource closed "`.
4. No se lanza ninguna excepción, por lo que el `catch` no se ejecuta.

🔹 Salida final

```
Resource used Resource closed
```

✅ **Respuesta correcta:**
b. Resource used Resource closed.

85.¿Cuál es el comando en Windows para listar el contenido de un directorio/carpeta?

a. ls

b. list

c. mv

d. dir

e. show

En **Windows**, el comando para listar el contenido de un directorio es:

- `dir` → muestra archivos y subcarpetas del directorio actual.

Las otras opciones:

- `ls` → comando de Linux/macOS.
- `list`, `show` → no existen en CMD.
- `mv` → se usa para mover o renombrar archivos en Linux/Windows PowerShell, no para listar.

✅ **Respuesta correcta:**
d. dir

86. What are the modifiers that a nested class can have?

a. public

b. final

c. protected

d. abstract

e. static

f. private

g. strictfp

h. All the options are correct.

En Java, una **nested class** (clase anidada) puede tener varios modificadores dependiendo de su tipo:

1. **Member class (nested non-static class)**:
    - `public`, `protected`, `private`, `abstract`, `final`, `strictfp`
2. **Static nested class**:
    - `public`, `protected`, `private`, `abstract`, `final`, `strictfp`, `static`

Por lo tanto, **todas las opciones listadas son válidas para algún tipo de nested class**.

✅ **Respuesta correcta:**
h. All the options are correct.

87. import java.util.*;

public class ComparableTest {

public static void main(String[] args) {

```
        List<Item> items = new ArrayList<>();
        items.add(new Item("Apple", 3));
        items.add(new Item("Banana", 2));
        items.add(new Item("Cherry", 1));
        Collections.sort(items);
        for (Item item : items) {
            System.out.println(item.getName() + ": " + item.getPrice());
        }
    }
}
class Item implements Comparable<Item> {
    private String name;
    private int price;
    public Item(String name, int price) {
        this.name = name;
        this.price = price;
    }
```

```
    public String getName() {
        return name;
    }
    public int getPrice() {
        return price;
    }
    @Override
    public int compareTo(Item other) {
        return this.name.compareTo(other.name);
    }
}
```

Apple: 3 Banana: 2 Cherry: 1

Compilation error

Apple: 3 Cherry: 1 Banana: 2

Cherry: 1 Banana: 2 Apple: 3

Analicemos el código paso a paso 👇

Código clave

```
Collections.sort(items);
```

- La lista `items` contiene objetos `Item` que implementan `Comparable<Item>`.
- El método `compareTo` está definido como:

```
@Override
public int compareTo(Item other) {
    return this.name.compareTo(other.name);
}
```

- Esto ordena los `Item` **lexicográficamente por nombre** (`String.compareTo`).

Lista original

```
"Apple"  -> 3
"Banana" -> 2
"Cherry" -> 1
```

Orden lexicográfico

```
Apple, Banana, Cherry
```

Salida final

```
Apple: 3
Banana: 2
Cherry: 1
```

✅ Respuesta correcta:
Apple: 3 Banana: 2 Cherry: 1

88. What will be the output of the following code snippet?

```
class A {
    public A() {
        System.out.println("A's constructor");
    }
}
class B extends A {
    public B() {
        super();
        System.out.println("B's constructor");
    }
}
class C extends B {
    public C() {
        System.out.println("C's constructor");
```

```
    }
}
public class Test {
    public static void main(String[] args) {
        C c = new C();
    }
}
```

A's constructor B's constructor C's constructor

C's constructor A's constructor B's constructor

A's constructor C's constructor B's constructor

What will be the output of the following code snippet?

Analicemos el flujo de construcción de objetos en Java paso a paso 👇

Clases:

```
class A {
    public A() { System.out.println("A's constructor"); }
}
```

```
class B extends A {

    public B() {

        super(); // Llama al constructor de A

        System.out.println("B's constructor");

    }

}



class C extends B {

    public C() { System.out.println("C's constructor"); }

}
```

Flujo de creación de `new C()`:

1. Se llama al constructor de `C`.

2. Antes de ejecutar su cuerpo, **Java invoca automáticamente el constructor de la superclase** (`B`).

3. Constructor de `B`:
    - Llama explícitamente a `super()` → invoca constructor de `A`.
    - Constructor de `A` imprime → `"A's constructor"`
    - Luego se imprime `"B's constructor"`
4. Finalmente, se ejecuta el constructor de `C` → imprime `"C's constructor"`

Salida final:

```
A's constructor

B's constructor

C's constructor
```

✅ **Respuesta correcta:**
A's constructor B's constructor C's constructor

89. What will be the output of the following code snippet?

public class StringConcatenationTest {

```
    public static void main(String[] args) {

        String str1 = "Hello";

        String str2 = "World";

        String str3 = str1 + " " + str2;

        String str4 = str1.concat(" ").concat(str2);

        String str5 = new StringBuilder().append(str1).append("
").append(str2).toString();

        System.out.print(str3.equals(str4) + " ");

        System.out.print(str4.equals(str5) + " ");

        System.out.print((str3 == str4) + " ");

        System.out.println(str4 == str5);

    }

}
```

a. true true true true

b. Compilation fails.

c. true true false false

d. false false false false

e. false true false true

f. false false true true

Analicemos el código paso a paso 👇

```
String str1 = "Hello";
String str2 = "World";

String str3 = str1 + " " + str2;                    // concatenación con +
String str4 = str1.concat(" ").concat(str2);       // concatenación con concat()
String str5 = new StringBuilder().append(str1).append(" ").append(str2).toString(); // StringBuilder
```

Comparaciones

1. `str3.equals(str4)` → ✅ **true**
    - Ambos contienen `"Hello World"`, `equals` compara **contenido**, no referencias.
2. `str4.equals(str5)` → ✅ **true**
    - Contenido igual `"Hello World"`.
3. `str3 == str4` → ❌ **false**
    - `==` compara **referencias**, no contenido.
    - `str3` y `str4` son distintos objetos en memoria.
4. `str4 == str5` → ❌ **false**
    - `str5` fue creado con `StringBuilder.toString()` → nueva referencia.

Salida final

```
true true false false
```

✅ **Respuesta correcta:**
c. true true false false

90.

```
import java.sql.Connection;

import java.sql.DriverManager;

import java.sql.ResultSet;

import java.sql.SQLException;

import java.sql.Statement;

public class TestJDBC {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/testdb";

        String user = "testuser";

        String password = "testpassword";

        try {

            Connection conn = DriverManager.getConnection(url, user, password);

            Statement stmt = conn.createStatement();

            ResultSet rs = stmt.executeQuery("SELECT name FROM users WHERE
id = 1");
```

```
            while (rs.next()) {

                System.out.println(rs.getString("name"));

            }

        } catch (SQLException e) {

            e.printStackTrace();

        }

    }

}
```

a. It will throw a NullPointerException.

b. It will print the name of the user with id 1.

c. It will print "Connection established successfully."

d. It will throw a SQLException.

Analicemos el código paso a paso 👇

- El código usa **JDBC** para conectarse a una base de datos MySQL:

```
Connection conn = DriverManager.getConnection(url, user, password);

Statement stmt = conn.createStatement();

ResultSet rs = stmt.executeQuery("SELECT name FROM users WHERE id =
1");

while (rs.next()) {

    System.out.println(rs.getString("name"));

}
```

- `DriverManager.getConnection(...)` establece la conexión.
- `stmt.executeQuery(...)` ejecuta la consulta.
- `rs.next()` recorre el `ResultSet`.
- `rs.getString("name")` imprime el valor de la columna `"name"` del usuario con `id = 1`.

✅ Suponiendo que la base de datos existe, la tabla `users` existe y tiene un usuario con `id = 1`, el código **imprimirá el nombre del usuario**.

- **No imprime mensajes de éxito de conexión.**
- **No lanza NullPointerException** a menos que `rs` sea nulo, lo cual no ocurre.
- **No lanza SQLException** si la conexión y la consulta son correctas.

✅ **Respuesta correcta:**
b. It will print the name of the user with id 1.

91. ¿Cuáles de las siguientes son herramientas necesarias para desarrollar componentes APX?

a. Git

b. Docker

c. Artifactory

d. Maven

e. JDK

92. public class Test {

```
    public void op2(String s) {
        s.toUpperCase();
    }
    public void op3() {
        String msg = new String("message");
        op2(msg);
        System.out.println(msg);
    }
    public static void main(String[] args) {
        new Test().op3();
    }
}
```

a. MESSAGE

b. An exception is thrown at runtime.

c. Compilation fails.

d. message

e. null

Analicemos el código paso a paso 👇

```
public void op2(String s) {

    s.toUpperCase();

}
```

- `String` en Java es **inmutable**.
- `s.toUpperCase()` devuelve un **nuevo String**, pero no modifica `s` en sí.
- Como no se asigna el resultado, `s` sigue apuntando al mismo valor original.

```
public void op3() {

    String msg = new String("message");

    op2(msg);

    System.out.println(msg);

}
```

- `msg` se pasa a `op2`.
- `op2` llama a `s.toUpperCase()` pero **no cambia `msg`**.
- Por lo tanto, `System.out.println(msg);` imprime el **valor original**: `"message"`.

✅ **Respuesta correcta:**
d. message

93. ¿Cuál de las siguientes afirmaciones confirma correctamente los paquetes que usa el siguiente código?

```
import java.sql.Connection;

import java.sql.DriverManager;

import java.sql.ResultSet;

import java.sql.SQLException;

import java.sql.Statement;

public class TestJDBC {

    public static void main(String[] args) {

        String url = "jdbc:mysql://localhost:3306/mydb";
```

```
String user = "user";

String password = "password";

Connection conn = null;

Statement stmt = null;

ResultSet rs = null;

try {

    conn = DriverManager.getConnection(url, user, password);

    stmt = conn.createStatement();

    rs = stmt.executeQuery("SELECT name FROM users WHERE id = 1");

    while (rs.next()) {

        System.out.println(rs.getString("name"));

    }

} catch (SQLException e) {

    e.printStackTrace();

} finally {

    // Close resources here

}
```

```
    }
}
```

a. rs.close();

   stmt.close();

   conn.close();

b. if (rs != null) rs.close();

   if (conn != null) conn.close();

   if (stmt != null) stmt.close();

c. if (rs != null) rs.close();

   if (stmt != null) stmt.close();

   if (conn != null) conn.close();

d. stmt.close();

   conn.close();

   rs.close();

a

b

c

d

Para cerrar correctamente los recursos JDBC en el bloque `finally`, **deben cerrarse en el orden inverso de su apertura**:

1. `ResultSet` → `rs.close()`
2. `Statement` → `stmt.close()`
3. `Connection` → `conn.close()`

Además, siempre se debe verificar que cada recurso **no sea nulo** para evitar `NullPointerException`.

Opción correcta:

```
if (rs != null) rs.close();
if (stmt != null) stmt.close();
if (conn != null) conn.close();
```

✅ **Respuesta correcta:**

C

94. public class Test implements Runnable {

private int i;

@Override

public void run() {

if (i % 10 == 0) {

i++;

```
        }
        for (int x = 0; x < 10; x++, i++) {
            if (x == 4) {
                Thread.yield();
            }
            System.out.println(i + " ");
        }
    }
    public static void main(String[] args) {
    Test n = new Test();
    for (int x = 0; x < 100; x++) {
        new Thread(n).start();
    }
 }
}
```

¿Cuál es la respuesta verdadera?

O a. La salida nunca puede contener el valor 297.

O b. La salida nunca puede contener el valor 1020.

O c. La salida nunca puede contener el valor 1130.

O d. La salida nunca puede contener el valor 30.

Analicemos el código paso a paso y cómo funciona la concurrencia en Java 👇

Código clave

```
public class Test implements Runnable {
    private int i;

    @Override
    public void run() {
        if (i % 10 == 0) { i++; }

        for (int x = 0; x < 10; x++, i++) {
            if (x == 4) { Thread.yield(); }
            System.out.println(i + " ");
        }
    }

    public static void main(String[] args) {
        Test n = new Test();
        for (int x = 0; x < 100; x++) {
            new Thread(n).start();
        }
    }
}
```

Observaciones importantes

1. La variable `i` es **compartida por todos los hilos** (`Test n = new Test()` se usa para los 100 hilos).

2. **No hay sincronización**, por lo que el incremento `i++` **no es atómico**.

3. Cada hilo ejecuta 10 incrementos (`for`), y puede empezar en cualquier momento relativo a los otros hilos.

4. `i` comienza en `0`. El primer hilo incrementa `i` antes de entrar al `for` si `i % 10 == 0`. Esto asegura que `i` nunca será múltiplo exacto de 10 al inicio del `for`.

Comportamiento de `i`

- Cada hilo intenta incrementar `i` 10 veces.

- Como no hay sincronización, **cualquier valor puede ser alcanzado** por las operaciones intercaladas, excepto los **valores que violen la condición inicial `i % 10 == 0`** al inicio de un hilo.

- **Condición `i % 10 == 0`**:

    - Si `i % 10 == 0`, se hace `i++` → el `for` empieza con `i = 1` modulo 10.

    - Por lo tanto, **ningún hilo imprimirá un valor múltiplo de 10 al inicio del `for`**.

    - Valores como `30, 40, 50...` pueden **ser impresos más tarde** debido a incrementos, pero el valor exacto `30` **al inicio de un hilo nunca ocurrirá**.

Conclusión

- La salida **puede contener cualquier número**, excepto que **ningún hilo puede empezar con un múltiplo exacto de 10** (`i % 10 == 0`) debido al `if` inicial.

- Por lo tanto, el valor **30 nunca se imprime exactamente en la salida** como primer valor de algún hilo.

✅ **Respuesta correcta:**
d. La salida nunca puede contener el valor 30.

95. public class OverloadedStaticMethods {

```
public static void display() {
    System.out.println("Display with no arguments");
}
public static void display(int num) {
    System.out.println("Display with integer: " + num);
}
public static void display(String str) {
    System.out.println("Display with string: " + str);
}
public static void main(String[] args) {
    OverloadedStaticMethods.display();
```

```
        OverloadedStaticMethods.display(5);

        OverloadedStaticMethods.display("Hello");

    }

}
```

a.

Display with no arguments

Display with string:

Display with integer: Hello

b.

Error en compilación

c.

Error en tiempo de ejecución

d.

Display with no arguments

Display with integer: 5

Display with string: Hello

a

b

c

d

Vamos a analizar el código:

```
public class OverloadedStaticMethods {

    public static void display() {
        System.out.println("Display with no arguments");
    }

    public static void display(int num) {
        System.out.println("Display with integer: " + num);
    }

    public static void display(String str) {
        System.out.println("Display with string: " + str);
    }

    public static void main(String[] args) {
        OverloadedStaticMethods.display();
        OverloadedStaticMethods.display(5);
        OverloadedStaticMethods.display("Hello");
    }
}
```

Explicación:

1. **Sobrecarga de métodos estáticos** → en Java es válida.
    - No hay error de compilación.
    - No hay error en ejecución.
2. Llamadas en `main`:
    - `OverloadedStaticMethods.display();` → invoca el método sin argumentos
      → imprime: **Display with no arguments**
    - `OverloadedStaticMethods.display(5);` → invoca el método con `int`
      → imprime: **Display with integer: 5**
    - `OverloadedStaticMethods.display("Hello");` → invoca el método con `String`
      → imprime: **Display with string: Hello**

✅ Salida final:

```
Display with no arguments
Display with integer: 5
Display with string: Hello
```

👉 **Respuesta correcta:**
d.

96. ¿Cuál de las siguientes opciones se utiliza principalmente para la integración continua en el desarrollo del sofware?

a. Jenkins

b. Artifactory

c. Jira

d. SonarQube

e. Bitbucket

La pregunta es:

¿Cuál de las siguientes opciones se utiliza principalmente para la integración continua en el desarrollo del software?

Análisis de cada opción:

- **a. Jenkins** → ✅ Es una de las herramientas más utilizadas para **integración continua (CI)** y automatización de pipelines.

- **b. Artifactory** → Es un repositorio de artefactos (binarios, librerías, dependencias), no una herramienta de CI.

- **c. Jira** → Es una herramienta de gestión de proyectos y seguimiento de tareas.

- **d. SonarQube** → Se usa para análisis estático de código y control de calidad, no para integración continua.

- **e. Bitbucket** → Es un sistema de control de versiones basado en Git, aunque incluye pipelines, su propósito principal es el control de código fuente.

✅ Respuesta correcta:

a. Jenkins

97. import java.util.ArrayList;

import java.util.List;

public class Test {

    public static void main(String[] args) {

        List<String> list = new ArrayList<>();

        list.add("Apple");

        list.add("Banana");

        list.add("Cherry");

        printList(list);

        System.out.println("Size: " + list.size());

        System.out.println("Contains 'Banana': " + list.contains("Banana"));

        System.out.println("Element at index 1: " + list.get(1));

    }

    private static void printList(List<String> list) {

        for (String element : list) {

```
            System.out.println(element);

        }

    }

}
```

a. Apple Banana Cherry Size: 3 Contains 'Banana': true Element at index 1: Banana

b. Apple Banana Size: 2 Contains 'Banana': false Element at index 1: Banana

c. Apple Banana Size: 2 Contains 'Banana': true Element at index 1: Banana

d. Apple Banana Cherry Size: 3 Contains 'Banana': true Element at index 1: Cherry

Vamos a analizar el código paso a paso:

```
List<String> list = new ArrayList<>();
list.add("Apple");
list.add("Banana");
list.add("Cherry");
```

👉 La lista contiene:
`["Apple", "Banana", "Cherry"]`

Luego se imprime con `printList(list);`
Resultado:

```
Apple
Banana
Cherry
```

Después:

```
System.out.println("Size: " + list.size());
```

👉 El tamaño es **3**.

```
System.out.println("Contains 'Banana': " +
list.contains("Banana"));
```

👉 `contains("Banana")` es **true**.

```
System.out.println("Element at index 1: " + list.get(1));
```

👉 El índice `1` corresponde a **"Banana"**.

✅ Salida final:

```
Apple

Banana

Cherry

Size: 3

Contains 'Banana': true

Element at index 1: Banana
```

Respuesta correcta:

a. Apple Banana Cherry Size: 3 Contains 'Banana': true Element at index 1: Banana ✅

98. Todo el acceso a los datos debe estar encapsulado en una transacción para facilitar la reutilización y control de acceso

Seleccione una:

verdadero

Falso

99. Given:

```
interface IFun {

int calculate(int i1, int i2);

}

public class Test {

public static void main(String[] args) {

IFun f = (i1, i2) -> i1 + i2; // line 1

int i1=1, i2 = 2; // line 2

IFun g = (i1, i2) -> i1 * i2; // line 3

System.out.println(g.calculate(i1, ii)); // line 4

}

}
```

What will be the output of the above code?

a. Compilation fails on line 4.

b. Compilation fails on line 2.

c. 2

d. Compilation fails on line 1.

e. Compilation fails on line 3.

f. 3

Let's analyze step by step:

```
interface IFun {
    int calculate(int i1, int i2);
}

public class Test {
    public static void main(String[] args) {

        IFun f = (i1, i2) -> i1 + i2; // line 1 ✅ valid lambda

        int i1=1, i2 = 2; // line 2 ✅ local variables declared

        IFun g = (i1, i2) -> i1 * i2; // line 3 ❌ ERROR
        // The lambda parameters i1, i2 shadow the local variables i1, i2 already defined above.
        // This is not allowed → "variable i1 is already defined in method main".

        System.out.println(g.calculate(i1, ii)); // line 4
```

```
            // This also has an error: variable "ii" does not exist.
        }
    }
```

Errors:

- **Line 3** → Compilation fails because lambda parameter names (`i1, i2`) cannot be the same as local variables `i1, i2` declared in the same scope.
- **Line 4** → Even if line 3 were fixed, it uses `ii` which is undefined → another compilation error.

👉 But since the **first compilation error appears at line 3**, the correct answer is:

e. Compilation fails on line 3. ✅

100. ¿Cuál de las siguientes ramas en Gitflow se utiliza para aplicar correcciones rápidas directamente en producción?

a. master

b. feature

c. hotfix

d. release

e. develop

En **Gitflow**, la rama que se utiliza para aplicar correcciones rápidas directamente en producción es:

✅ **c. hotfix**

📌 Explicación:

- **master** → contiene el código en producción estable.
- **develop** → rama principal de desarrollo.
- **feature** → para desarrollar nuevas funcionalidades.
- **release** → para preparar una nueva versión estable antes de pasarla a producción.
- **hotfix** → se crea a partir de **master** para corregir errores críticos detectados en producción, y luego se fusiona tanto en **master** como en **develop**.

👉 Por lo tanto, la respuesta correcta es **c. hotfix**.

101.

```
import java.util.Arrays;

import java.util.List;

import java.util.stream.Collectors;

public class StreamTest {

    public static void main(String[] args) {

        List<String> items = Arrays.asList("apple", "banana", "cherry", "date");

        List<String> result = items.stream()

                                .filter(s -> s.startsWith("a"))

                                .collect(Collectors.toList());

        System.out.println(result);
```

}

}

a. [apple, banana]

b. [banana, cherry]

c. [apple]

d. [apple, cherry]

El programa usa **Streams** para filtrar los elementos de la lista que empiezan con `"a"`.

Lista original:

```
["apple", "banana", "cherry", "date"]
```

Filtro aplicado:

```
s -> s.startsWith("a")
```

👉 Solo `"apple"` cumple la condición.

Por lo tanto, la salida será:

✅ **c. [apple]**

102. class ABC {

public String met() {

return "snippet 1";

}

}

public class ABCD extends ABC {

// Insert code here

}

Which line of code, when inserted at the commented line, causes the compiler to fail?

a. public String met() { return "4"; }

b. public char met(int i) { return '3';}

c. public String met(int i) { return "5"; }

d. public char met() { return '2'; }

Veamos:

La clase base:

```
class ABC {
    public String met() {
        return "snippet 1";
    }
}
```

En la subclase `ABCD`, cualquier método con **el mismo nombre pero distinta firma** se considera **sobrecarga (overload)**.
Solo si tiene **misma firma** pero tipo de retorno incompatible, habrá error de compilación.

Opciones:

a.

```
public String met() { return "4"; }
```

✅ Esto **sobrescribe** correctamente (mismo tipo de retorno `String`) → Compila bien.

b.

```
public char met(int i) { return '3'; }
```

✅ Firma distinta (`int` parámetro) → Sobrecarga válida.

c.

```
public String met(int i) { return "5"; }
```

✅ Firma distinta (`int` parámetro) → Sobrecarga válida.

d.

```
public char met() { return '2'; }
```

❌ Firma igual (ningún parámetro), pero tipo de retorno distinto (`char` en lugar de `String`). En Java, el retorno debe ser igual o covariante. `char` **no es compatible con** `String` → Error de compilación.

✅ **Respuesta correcta: d. public char met() { return '2'; }**

103. class CA {

CA() {

System.out.print("middle");

}

```
    CA(String s) {
        System.out.print(s);
    }
}
class CB extends CA {
}
public class Test {
    public static void main(String[] args) {
        new CA("message ");
        new CB();
        System.out.print(" end");
    }
}
```

What is the result?

a. message middle end

b. An exception is thrown at runtime.

c. Compilation fails

d. message end middle

Vamos paso a paso 🚀

Código:

```
class CA {
    CA() {
        System.out.print("middle");
    }

    CA(String s) {
        System.out.print(s);
    }
}

class CB extends CA {
}

public class Test {
    public static void main(String[] args) {
        new CA("message ");
        new CB();
        System.out.print(" end");
    }
}
```

Análisis:

1. `new CA("message ");`

- Llama al constructor `CA(String s)`.
- Imprime **`message`** .

2. `new CB();`
   - No tiene constructor definido → usa el **constructor por defecto**.
   - En Java, el constructor de una subclase llama **implícitamente** a `super()`.
   - Eso invoca `CA()` → imprime **`middle`**.
3. `System.out.print(" end");`
   - Imprime **`end`**.

Resultado final:

```
message middle end
```

✅ **Respuesta correcta: a. message middle end**

104. Una transacción APX es la unidad aplicativa que se ejecutará en APX Batch. Seleccione una:

Verdadero

Falso

105. ¿Cuál es la función principal del JDK (Java Development Kit)?

a. Permitir la edición de archivos JHTML y JCSS.

b. Servir como un servidor web para aplicaciones Java.

c. Proporcionar un entorno de ejecución para aplicaciones Java.

d. Ofrecer herramientas necesarias para compilar, depurar y ejecutar aplicaciones Java.

e. Ninguna opción es correcta.

La **función principal del JDK (Java Development Kit)** es:

✅ **d. Ofrecer herramientas necesarias para compilar, depurar y ejecutar aplicaciones Java.**

📌 Explicación:

- El **JDK** incluye:
    - **Compilador (`javac`)**
    - **Intérprete/ejecutor (`java`)**
    - Herramientas de depuración y documentación (`javadoc`, `jdb`, etc.)
- Permite **desarrollar aplicaciones Java**, mientras que el **JRE** solo permite **ejecutarlas**.

106. ¿Cuál es el propósito del método setSeverity() en el manejo de una transacción APX?

a. Validar los datos proporcionados por la transacción.

b. Establecer la gravedad de la notificación generada por la transacción

c. Determinar el tipo de transacción a ejecutar

d. Configurar los parámetros de entrada de la transacción.

El propósito del método `setSeverity()` en una transacción APX es:

✅ **b. Establecer la gravedad de la notificación generada por la transacción**

📌 Explicación:

- En APX, las transacciones pueden generar **notificaciones** o **mensajes de log** durante su ejecución.
- `setSeverity()` permite definir el **nivel de importancia o severidad** de estas notificaciones, como:
    - Informativo
    - Advertencia
    - Error
- Esto ayuda a controlar cómo se reportan y manejan los eventos dentro del sistema.

107. import java.util.function.*;

public class Test {

public static void main(String[] args) {

Predicate<String> p = (s) -> s.length() > 5;

Function<String, Integer> f = String::length;

Consumer<String> c = (s) -> System.out.println(s.toUpperCase());

Supplier<String> s = () -> "Hello, world!";

String testString = "functional";

System.out.println(p.test(testString));

```
        System.out.println(f.apply(testString));

        c.accept(testString);

        System.out.println(s.get());

    }

}
```

What will be the output of the above code?

a. false 9 FUNCTIONAL Hello, World!

b. A runtime exception is thrown.

c. Compilation fails.

d. true 9 FUNCTIONAL Hello, World!

e. true 10 FUNCTIONAL Hello, World!

Vamos a analizar el código paso a paso:

Código clave:

```
Predicate<String> p = (s) -> s.length() > 5;
Function<String, Integer> f = String::length;
Consumer<String> c = (s) -> System.out.println(s.toUpperCase());
Supplier<String> s = () -> "Hello, world!";
String testString = "functional";
```

1. **`p.test(testString)`**
    - `testString = "functional"` → longitud = 10

- `10 > 5` → `true`

2. **`f.apply(testString)`**
   - Devuelve la longitud de `"functional"` → **10**
3. **`c.accept(testString)`**
   - Imprime `"FUNCTIONAL"` (mayúsculas)
4. **`s.get()`**
   - Devuelve `"Hello, world!"`

Salida final:

```
true
10
FUNCTIONAL
Hello, world!
```

✅ **Respuesta correcta:**
d. true 9 FUNCTIONAL Hello, World! ❌

⚠️ Observa que la longitud de `"functional"` es **10**, no 9. Por lo tanto la opción correcta sería **true 10 FUNCTIONAL Hello, World!**

Respuesta correcta: e. true 10 FUNCTIONAL Hello, World!

108. public class Test {

public static void main(String[] args) {

```
        String a = "tic";

        String b = "tac";

        a.substring(0, 2);

        a.concat("toc");

        System.out.println(a += "game" + b.toUpperCase());

    }

}
```

What is the result?

a. ticgameTAC

b. titocTAC

c. tictactoc

d. ticgametac

e. Compilation fails.

Vamos a analizar el código paso a paso:

```
String a = "tic";

String b = "tac";


a.substring(0, 2);  // Se crea un nuevo String "ti" pero **no se asigna** → a sigue siendo "tic"

a.concat("toc");    // Se crea un nuevo String "tictoc" pero **no se asigna** → a sigue siendo "tic"
```

```
System.out.println(a += "game" + b.toUpperCase());
```

- `b.toUpperCase()` → `"TAC"`
- `"game" + b.toUpperCase()` → `"gameTAC"`
- `a += "gameTAC"` → `a = "ticgameTAC"`

✅ Salida final:

`ticgameTAC`

Respuesta correcta:

a. ticgameTAC

109. Which of the following packages in Java do not need to be explicitly imported in a Java program?

a. java.net

b. java.lang

c. java.function

d. java.util

e. java.io

f. javax.sql

En Java, el paquete **`java.lang`** se importa automáticamente en todos los programas, por lo que **no es necesario importarlo explícitamente**.

- **b. java.lang ✅**

📌 Explicación:

- Contiene clases fundamentales como `String`, `Math`, `Integer`, `Object`, `System`, etc.
- Todos los demás paquetes (`java.net`, `java.util`, `java.io`, `java.function`, `javax.sql`) **sí deben importarse explícitamente** si se usan.

Respuesta correcta:

b. java.lang

110. Which of the following statements correctly describes an immutable object in Java?

a. An immutable object must implement the Cloneable interface.

b. Immutable objects cannot be created using the new keyword.

c. Immutable objects can have setter methods to change their fields.

d. An immutable object does not allow any changes to its state after it is created.

e. An immutable object can have its fields modified after it is created.

La descripción correcta de un **objeto inmutable en Java** es:

✅ **d. An immutable object does not allow any changes to its state after it is created.**

📌 Explicación:

- Un **objeto inmutable** no permite modificar sus campos una vez creado.
- Clases como `String` o `Integer` son ejemplos de objetos inmutables en Java.

- No tiene métodos `setter` que cambien sus campos internos, y sus campos suelen ser `final`.

Las otras opciones son incorrectas:

- **a.** No es obligatorio implementar `Cloneable`.
- **b.** Se pueden crear con `new`.
- **c. y e.** Tendrían la posibilidad de modificar el estado, lo cual rompe la inmutabilidad.

111. What will be the output of the following code snippet?

```
public class EnumTest {
    enum Day {
        MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY,
SUNDAY
    }
    public static void main(String[] args) {
        for (Day day : Day.values()) {
            System.out.print(day + " ");
        }
    }
}
```

a. Compilation error.

b. MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY

c. MONDAY TUESDAY WEDNESDAY THURSDAY FRIDAY SATURDAY SUNDAY

d. MONDAY TUESDAY WEDNESDAY THURSDAY FRIDAY SATURDAY

El código define un **enum `Day`** con todos los días de la semana y luego recorre sus valores usando `Day.values()`.

```
for (Day day : Day.values()) {
    System.out.print(day + " ");
}
```

- `Day.values()` devuelve un array con todos los valores del enum.
- `System.out.print(day + " ")` imprime cada día seguido de un espacio.
- No hay comas ni errores de compilación.

Salida:

```
MONDAY TUESDAY WEDNESDAY THURSDAY FRIDAY SATURDAY SUNDAY
```

Respuesta correcta:

c. MONDAY TUESDAY WEDNESDAY THURSDAY FRIDAY SATURDAY SUNDAY ✅

112. enum Days {

MONDAY, TUESDAY, WEDNESDAY, THURSDAY

}

```
public class Test {

    public static void main(String[] args) {

        int y = 1;

        Days d = Days.TUESDAY;

        switch (d) {

            case MONDAY: y++;

            case TUESDAY: y += 10;

            case WEDNESDAY: y += 100;

            case THURSDAY:  y += 1000;

        }

        System.out.println(y);

    }

}
```

a. 11

b. 111

c. Compilation fails.

d. An exception is thrown at runtime.

e. 1111

En Java, cuando se usa un **`switch`** sobre un enum, **cada `case` debe terminar con `break`** para evitar el *fall-through*.

Código:

```
int y = 1;
Days d = Days.TUESDAY;

switch (d) {
    case MONDAY: y++;
    case TUESDAY: y += 10;
    case WEDNESDAY: y += 100;
    case THURSDAY: y += 1000;
}
System.out.println(y);
```

Análisis:

- `d = Days.TUESDAY`
- **Se ejecuta desde `TUESDAY` hasta el final** (porque no hay `break`):
    1. `y += 10` → `y = 11`
    2. `y += 100` → `y = 111`
    3. `y += 1000` → `y = 1111`

Resultado final:

`1111`

✅ **Respuesta correcta: e. 1111**

113. Cuando se asigna un byte a un carácter en Java, ¿cuál es el tipo de resultado?

a. char

b. int

c. byte

d. short

e. No puedes asignar un byte a un carácter.

114. Dado:

```
class Test {
    static void m(int a) {
        a += 10;
    }
    public static void main(String[] args) {
        int a = 20;
        m(a);
```

Test.m(a);

System.out.println(a);

}

}

¿Cuál es el resultado?

a. 10

b. 40

c. 20

d. Compilation fails.

e. An exception is throw at run time.

f. 30

Analicemos paso a paso:

```
class Test {
    static void m(int a) {
        a += 10;
    }

    public static void main(String[] args) {
        int a = 20;
        m(a);        // Llama a m con a = 20 → dentro de m, a = 30,
pero es **pasado por valor**
        Test.m(a);   // Lo mismo, a sigue siendo 20 fuera del
método
        System.out.println(a);
```

```
    }
}
```

- En Java, los **tipos primitivos se pasan por valor**.
- Modificar `a` dentro del método `m` **no afecta** a `a` en `main`.

Resultado final:

```
20
```

✅ **Respuesta correcta: c. 20**

115. abstract class Animal {

```
    void speak() {
        System.out.println("Animals make noise");
    }
    public static void main(String[] args) {
        Animal animal = new Dog();
        animal.speak();
    }
}
```

```
class Dog extends Animal {

    protected void speak() {

        System.out.println("The Dog barks");

    }

}

public class Test {

    public static void main(String... strings) {

        Animal.main(strings);

    }

}
```

a. The Dog barks

b. Compilation fails.

c. Animals make noise

d. A runtime exception is thrown.

Analicemos el código paso a paso:

```
abstract class Animal {
    void speak() {
        System.out.println("Animals make noise");
    }
```

```
    public static void main(String[] args) {
        Animal animal = new Dog();
        animal.speak();
    }
}

class Dog extends Animal {
    protected void speak() {
        System.out.println("The Dog barks");
    }
}
```

1. La clase `Dog` **hereda de** `Animal`.
2. `Animal.speak()` tiene **default (package-private)**.
3. `Dog.speak()` intenta sobrescribir `speak()` con **protected**.

📌 En Java, no se puede reducir la visibilidad al sobrescribir un método.

- `default` → `protected` es **válido** porque aumenta la visibilidad.
- Por lo tanto, **no hay error de compilación por la visibilidad**.

4. `Animal.main()` crea un `Dog` y llama a `animal.speak()`.

- Esto es **polimorfismo**: se invoca el método de la subclase (`Dog`).

Salida final:

```
The Dog barks
```

✅ **Respuesta correcta: a. The Dog barks**

116.

```
public class Test {
    public static void main(String[] args) {
        String str = "Hello";
        modify(str);
        System.out.println(str)
        StringBuffer sbf = new StringBuffer("Hello");
        modify(sbf);
        System.out.println(sbf.replace(2, sbf.length(), "o"));
        StringBuilder sbd = new StringBuilder("Hello");
        modify(sbd);
        System.out.println(sbd.reverse());
    }
    private static void modifyString(String s) {
        s = s.concat(" World");
```

```
    }

    private static void modifyStringBuffer(StringBuffer sb) {

        sb.append(" World");

    }

    private static void modifyStringBuilder(StringBuilder sb) {

        sb.append(" World");

    }

}
```

a. Hello Hello World Hello World

b. Hello Hello Hello

c. Hello Heo dlroW olleH

d. Helio World Heo Hello World

e. Hello Heo World Hello

Vamos a analizar el código paso a paso. Primero, observemos que los métodos llamados en `main` son:

```
modify(str);

modify(sbf);

modify(sbd);
```

Pero los métodos definidos son `modifyString`, `modifyStringBuffer` y `modifyStringBuilder`.

❌ Esto provoca un **error de compilación**, porque **no existen métodos llamados `modify`**.

Explicación:

1. `String str = "Hello";`
    - Si llamara a `modifyString(str);` → no cambia `str` porque `String` es inmutable y se pasa por valor.
2. `StringBuffer sbf = new StringBuffer("Hello");`
    - Si llamara a `modifyStringBuffer(sbf);` → se modifica el contenido del `StringBuffer`.
3. `StringBuilder sbd = new StringBuilder("Hello");`
    - Si llamara a `modifyStringBuilder(sbd);` → se modifica el contenido del `StringBuilder`.

Conclusión:

- Debido a que los nombres de los métodos llamados **no coinciden** con los definidos, el código **no compila**.

✅ **Respuesta correcta: c. Compilation fails.**

117. ¿Cuál es la salida del siguiente código?

```
public class Test {

   public void run() {
```

```
        try {
            System.out.print("Running ");
            throw new Exception();
        } catch (Exception e) {
            System.err.print("Problem");
        } finally {
            System.out.print("Stopping");
        }
    }
    public static void main(String[] args) {
        new Test().run();
    }
}
```

a. Running Stopping Problem

b. Running Problem Stopping

c. Running Problem

d. La compilación falla

e. Problem Stopping

f. Stopping Problem

Analicemos el código paso a paso:

```
public void run() {
    try {
        System.out.print("Running ");
        throw new Exception();
    } catch (Exception e) {
        System.err.print("Problem");
    } finally {
        System.out.print("Stopping");
    }
}
```

1. **Try:**
    - Se imprime `"Running "` en **salida estándar** (`System.out`).
    - Se lanza una excepción `Exception`.
2. **Catch:**
    - La excepción es capturada.
    - Se imprime `"Problem"` en **salida de error** (`System.err`).
3. **Finally:**
    - Se imprime `"Stopping"` en **salida estándar** (`System.out`).

Salida:

- En **stdout**: `Running Stopping`

- En **stderr**: `Problem`

Si se considera la combinación de ambas salidas (como usualmente se hace en ejercicios teóricos):

```
Running Problem Stopping
```

✅ **Respuesta correcta: b. Running Problem Stopping**

118. Which two options are true about the default constructor?

a. The default always invokes the no-arg superclass constructor.

b. The compiler creates a default constructor if the class does not have a no-arg constructor defined.

c. The default constructor is always public.

d. The default constructor is c

reated only when the class has no constructors defined.

e. The default constructor inherits the arguments of the superclass constructor.

119. ¿Cuál es la nomenclatura correcta para un componente de tipo librería?

a. TESTCO01

b. TESTRO01-MX-01

c. TESTRO01

d. TESTRO01-01-MX

En APX, la nomenclatura estándar para una **librería** sigue el patrón:

```
TESTRO01
```

- `TEST` → prefijo del proyecto
- `RO` → indica que es **librería (RO = Reusable Object / Library)**
- `01` → número secuencial del componente

✅ **Respuesta correcta: c. TESTRO01**

120. boolean x = true;

boolean y = false;

boolean result = x && y || !x;

System.out.println(result);

a. Runtime error

b. False

c. True

d. Compilation error

Analicemos paso a paso la expresión:

```
boolean x = true;
boolean y = false;
```

```
boolean result = x && y || !x;
```

1. `x && y` → `true && false` → **false**
2. `!x` → `!true` → **false**
3. `x && y || !x` → `false || false` → **false**

✅ **Respuesta correcta: b. False**

121. Which file is used to configure user-specific settings in Maven?

a. build.xml

b. settings.xml

c. user.xml

d. pom.xml

En Maven, el archivo que se utiliza para **configurar ajustes específicos del usuario** (como repositorios locales, credenciales, proxies, etc.) es:

```
settings.xml
```

- Se encuentra típicamente en el directorio:
  `~/.m2/settings.xml` (Linux/macOS) o
  `C:\Users\<usuario>\.m2\settings.xml` (Windows)
- **pom.xml** define el proyecto, sus dependencias y su build, pero **no es específico del usuario**.

✅ **Respuesta correcta: b. settings.xml**

122. ¿Qué significa que un cambio en el software sea retrocompatible?

a. El cambio puede romper la funcionalidad existente, requiriendo ajustes en el código que depende del software.

b. El cambio garantiza que el software siga funcionando sin necesidad de modificaciones en el código que depende de él.

c. El cambio introduce nuevas funcionalidades que no afectan el comportamiento existente del software.

d. El cambio corrige errores menores y realiza mejoras de rendimiento.

La definición de **retrocompatibilidad** (backward compatibility) se refiere a que **las nuevas versiones de un software siguen funcionando con el código, datos o interfaces existentes sin necesidad de cambios adicionales**.

Por lo tanto:

✅ **Respuesta correcta: b. El cambio garantiza que el software siga funcionando sin necesidad de modificaciones en el código que depende de él.**

123. Given the following classes, which line of code correctly creates an instance of the Dog class?

```
class Animal {

    // ...

}

class Dog extends Animal {
```

```
    // ...

}
```

a. Dog dog = new Dog();

b. Animal dog = new Dog();

c. Dog dog = new Animal();

d. Animal dog = new Animal();

Analicemos las opciones:

- `Dog` extiende de `Animal`.
- Se puede asignar una instancia de **subclase** a una referencia de **superclase**, pero no al revés.

Opciones:

a. `Dog dog = new Dog();` ✅ Correcto, instancia de `Dog` asignada a referencia `Dog`.
b. `Animal dog = new Dog();` ✅ También correcto, instancia de `Dog` asignada a referencia de `Animal` (polimorfismo).
c. `Dog dog = new Animal();` ❌ Incorrecto, no se puede asignar superclase a subclase.
d. `Animal dog = new Animal();` ❌ Correcto para crear un `Animal`, pero no es instancia de `Dog`.

✅ **Respuestas correctas: a y b**

124. ¿Qué declaraciones inicializan correctamente una variable booleana en Java?

a. boolean d = (4 - 14);

b. boolean f = "true";

c. boolean c = (7 >= 7);

d. boolean e = (10 > 5 && 2 < 1);

e. boolean b = (15 = 2);

f. boolean a = (3 < 6);

En Java, una **variable booleana** solo puede ser inicializada con un **valor booleano literal** (`true` o `false`) o con una **expresión que resulte en boolean**.

Analicemos cada opción:

- **a.** `boolean d = (4 - 14);` ❌ Incorrecto, `4 - 14` es un entero, no booleano.
- **b.** `boolean f = "true";` ❌ Incorrecto, `"true"` es un `String`, no un booleano literal.
- **c.** `boolean c = (7 >= 7);` ✅ Correcto, `7 >= 7` es una expresión booleana (`true`).
- **d.** `boolean e = (10 > 5 && 2 < 1);` ✅ Correcto, `10 > 5 && 2 < 1` es boolean (`true && false = false`).
- **e.** `boolean b = (15 = 2);` ❌ Incorrecto, `=` es asignación, no comparación (`==`).
- **f.** `boolean a = (3 < 6);` ✅ Correcto, `3 < 6` es boolean (`true`).

✅ **Respuestas correctas: c, d, f**

125. En APX CLI, ¿cual opcion permite crear un proyecto Maven con una unidad de despliegue de tipo librería?

a. apx init dto -u="TEST" (c="001") d="New DTO domain="domain subpackage class name="Example010" -y

b. apx init du-online-u="TEST" -n="online-test" d="New Deployment Unit Online" -y

c. apx add dto (-c="010") -domain="customer" -d="Test of DTO" -subpackage="" class name="ExampleDTO" -y

d. apx init du-lib-u="TEST" (-c="000") d="New Deployment Unit Library" -y

e. apx init du-batch-u="TEST" (-n=) "batch-test" -d="New Deployment Unit Batch" -y

En APX CLI, para **crear un proyecto Maven con una unidad de despliegue de tipo librería**, se utiliza la opción que indica explícitamente `du-lib` (Deployment Unit Library).

Analizando las opciones:

- **a.** Se usa para DTOs → ❌
- **b.** `du-online` → unidad de despliegue online → ❌
- **c.** Añadir DTO → ❌
- **d.** `apx init du-lib-u="TEST"` → correcto, crea un **Deployment Unit Library** → ✅
- **e.** `du-batch` → unidad de despliegue batch → ❌

✅ **Respuesta correcta: d. apx init du-lib-u="TEST" (-c="000") d="New Deployment Unit Library" -y**

126. ¿Qué es ENTORNO LOCAL APX?

a. Ninguna opción es correcta.

b. Es un entorno productivo para desplegar los componentes APX.

c. Herramienta para gestionar proyectos en Java.

d. Es un entorno para que los desarrolladores prueben y depuren sus transacciones y job.

e. Herramienta de línea de comandos para crear componentes APX.

En APX, el **ENTORNO LOCAL** se refiere al entorno donde los desarrolladores **pueden probar y depurar sus transacciones y jobs** antes de desplegarlos a entornos de integración o producción.

✅ **Respuesta correcta: d. Es un entorno para que los desarrolladores prueben y depuren sus transacciones y job.**

¿Cuál de las siguientes afirmaciones describe mejor un Step en el contexto de Spring Batch?

a. Una clase que gestiona la configuración de la base de datos utilizada por un trabajo por lotes.

b. Una interfaz que define los métodos para realizar operaciones CRUD en los datos del trabajo por lotes.

c. Un objeto de dominio que encapsula una fase independiente y secuencial de un trabajo por lotes.

d. Un componente que se encarga exclusivamente de la validación de datos en un trabajo por lotes.

En Spring Batch, un **Step** representa **una fase o etapa independiente de un Job** que se ejecuta de manera secuencial o condicional. Cada Step puede incluir lectura, procesamiento y escritura de datos.

✅ **Respuesta correcta:**
c. Un objeto de dominio que encapsula una fase independiente y secuencial de un trabajo por lotes.

127. ¿Qué métodos de anulación (Override) son válidos para las siguientes clases?

```
class Parent {

    void display() {

        System.out.println("Parent");
```

```
    }

}

class Child extends Parent {

    // Override here

}
```

a. static void display() { System.out.println("Child"); }

b. private void display() { System.out.println("Child"); }

c. void display() { System.out.println("InChild"); }

d. public void display() { System.out.println("Child"); }

Para que un método sea **válidamente sobrescrito (override)** en Java, se deben cumplir estas reglas:

1. El método debe tener **el mismo nombre, tipo de retorno y parámetros**.
2. No puede reducir la **visibilidad** del método (por ejemplo, de `public` a `protected`).
3. No puede ser **`static` ni `private`**, porque esos métodos **no son heredables** en el sentido de polimorfismo.

Analizando las opciones:

- **a. `static void display()`** ❌ No se puede sobrescribir un método estático.
- **b. `private void display()`** ❌ Un método privado no puede sobrescribirse, solo se declara oculto dentro de la subclase.
- **c. `void display()`** ✅ Correcto, mismo nivel de visibilidad (`package-private`) y mismo tipo de retorno.
- **d. `public void display()`** ✅ Correcto, aumenta la visibilidad de `package-private` a `public`, lo cual es permitido.

✅ **Respuestas correctas: c y d**

128. Which of the following class declarations is not correct?

a. abstract class Person ()

b. final class Person ()

c. class Person ()

d. abstract final class Person ()

e. public abstract class Person ()

Analizando cada declaración en Java:

- **a.** `abstract class Person ()` ❌ Incorrecto por los paréntesis `()`. La sintaxis correcta no lleva `()`.
- **b.** `final class Person ()` ❌ Igual que arriba, los paréntesis no son válidos en la declaración de clase.
- **c.** `class Person ()` ❌ Los paréntesis no son válidos al declarar una clase.
- **d.** `abstract final class Person ()` ❌ Incorrecto por **ser abstract y final al mismo tiempo** (no puede instanciarse ni heredarse) y por los paréntesis.
- **e.** `public abstract class Person ()` ❌ Nuevamente, los paréntesis no son válidos en la declaración.

En realidad, **ninguna de estas declaraciones es correcta** porque todas incluyen `()` en la declaración de la clase, lo cual es sintácticamente incorrecto. Además, la combinación `abstract final` es conceptualmente inválida.

✅ **Respuesta correcta: d. abstract final class Person ()**
(es la más claramente incorrecta por ser abstract y final simultáneamente, además de la sintaxis inválida)

129. Given the following code snippet, what will be the output of the program?

String s1 = "text";

String s2 = new String("text");

String s3 = "text";

String s4 = s2.intern();

System.out.println(s1 == s3);

System.out.println(s1 == s2);

System.out.println(s1 == s4);

a. false false true

b. true true true

c. true false false

d. true false true

Analicemos el código paso a paso:

```
String s1 = "text";                 // String literal → se guarda
en el string pool

String s2 = new String("text");     // Nueva instancia en heap,
diferente del pool

String s3 = "text";                 // Misma literal que s1 →
apunta al pool
```

```
String s4 = s2.intern();              // Devuelve la referencia del
string en el pool
```

Ahora evaluamos las comparaciones con `==` (compara referencias):

1. `s1 == s3` → **true**, ambos apuntan al mismo objeto en el string pool.
2. `s1 == s2` → **false**, s2 apunta a un objeto diferente en el heap.
3. `s1 == s4` → **true**, s4 es el string internado del pool, coincide con s1.

✅ **Respuesta correcta:**
d. true false true

130. Which method in java.lang.Object should be overridden if the hashcode() method is overridden?

a. clone()

b. compareTo()

c. copy()

d. equals()

e. sort()

En Java, existe una **regla fundamental**: si sobrescribes el método `hashCode()`, también debes sobrescribir `equals()`. Esto asegura que los objetos que son considerados iguales por `equals()` tengan el mismo valor de hash, lo cual es esencial para colecciones como `HashMap` y `HashSet`.

✅ **Respuesta correcta:**
d. equals()

131. ¿Cuál será el resultado de compilar y ejecutar el siguiente fragmento de código en Java?

```
public class StaticBlockTest {

    static {

        System.out.println("Static block 1");

    }

    {

        System.out.println("Instance block 1");

    }

    public StaticBlockTest() {

        System.out.println("Constructor");

    }

    static {

        System.out.println("Static block 2");

    }

    {

        System.out.println("Instance block 2");
```

```
    }

    public static void main(String[] args) {

        new StaticBlockTest();

        new StaticBlockTest();

    }

}
```

Opción a:

Static block 1

Static block 2

Instance block 1

Constructor

Instance block 2

Instance block 1

Constructor

Instance block 2

Opción b:

Error de compilación.

Opción c:

Static block 1

Static block 2

Instance block 1

Instance block 2

Constructor

Instance block 1

Instance block 2

Constructor

Opción d:

Instance block 1

Instance block 2

Constructor

Instance block 1

Instance block 2

Constructor

Static block 1

Static block 2

a

b

c

d

En Java, **los bloques estáticos** se ejecutan **una sola vez al cargar la clase**, en el orden en que aparecen.
Los **bloques de instancia** se ejecutan **cada vez que se crea un objeto**, **antes del constructor**, en el orden en que aparecen en la clase.

Analizando el código:

1. Se cargan los bloques estáticos:

```
Static block 1
Static block 2
```

2. Se crea la primera instancia:

```
Instance block 1
Instance block 2
Constructor
```

3. Se crea la segunda instancia:

```
Instance block 1
Instance block 2
Constructor
```

✅ Por lo tanto, la salida correcta es:

```
Static block 1
Static block 2
Instance block 1
Instance block 2
Constructor
Instance block 1
Instance block 2
Constructor
```

Respuesta correcta: c

132. ¿Cuál es el comando en Bash para cambiar el directorio actual a uno especificado?

a. goto

b. cd

c. move

d. sh

e. chdir

La instrucción correcta para cambiar el directorio en Bash es:

`cd` (change directory).

Por ejemplo:

```
cd /home/usuario/documentos
```

Respuesta correcta: b

133. Which three of the following options are considered bad practices in software development in Java?

a. Ignoring exceptions instead of handling them properly.

b. Using empty and undocumented catch blocks.

c. Duplicating code.

d. Not testing code that is difficult to unit test.

e. Using proper naming conventions.

Las tres opciones que representan **malas prácticas** en desarrollo de software en Java son:

a. Ignoring exceptions instead of handling them properly.

- Ignorar excepciones puede ocultar errores y causar fallos difíciles de diagnosticar.

b. Using empty and undocumented catch blocks.

- Bloques `catch` vacíos dejan errores sin tratar y dificultan el mantenimiento del código.

c. Duplicating code.

- La duplicación de código reduce la mantenibilidad y aumenta la probabilidad de errores.

Las otras opciones **d** y **e** no son malas prácticas:

- **d**: “Not testing code that is difficult to unit test” es una advertencia, pero no necesariamente una práctica común.
- **e**: “Using proper naming conventions” es buena práctica, no mala.

Respuesta correcta: a, b, c

134. ¿Qué clase puede proveer las siguientes capacidades?: no permitir valores duplicados, mantener el orden en el cual los elementos fueron insertados, proveer iteración rápida y acceso aleatorio eficiente.

a. TreeSet

b. LinkedHashSet

c. HashSet

d. ArrayList

e. LinkedHashMap

La descripción dada combina varias características:

- **No permitir valores duplicados:** esto lo hacen las colecciones tipo `Set`.
- **Mantener el orden de inserción:** esto lo hace `LinkedHashSet` (a diferencia de `HashSet`, que no garantiza orden).
- **Iteración rápida y acceso aleatorio eficiente:** `LinkedHashSet` permite iteración predecible y relativamente eficiente, aunque acceso aleatorio directo (`get(index)`) no es tan eficiente como en un `List`.

Por lo tanto, la clase que **mejor cumple con estos requisitos** es:

Respuesta correcta: b. LinkedHashSet ✅

135. ¿Cuántas veces se ejecutará la instrucción println más interna en el siguiente código?

public class NestedForLoopTest {

```
    public static void main(String[] args) {

        for (int i = 0; i < 4; i++) {

            for (int j = 0; j < 3; j++) {

                for (int k = 0; k < 2; k++) {

                    System.out.println("Hello");

                }

            }

        }

    }

}
```

a. 6 veces

b. 12 veces

c. 24 veces

d. 36 veces

Para calcular cuántas veces se ejecuta la instrucción `println` más interna, multiplicamos las iteraciones de los tres bucles anidados:

- `i` de 0 a 3 → 4 iteraciones
- `j` de 0 a 2 → 3 iteraciones
- `k` de 0 a 1 → 2 iteraciones

Total de ejecuciones: **4 × 3 × 2 = 24**

Respuesta correcta: c. 24 veces ✅

136. Given:

```
interface I {
    void method();
}
public class C implements I {
// method declarations
}
```

Which method can be declared within the C class to make it compile?

a. public abstract void method();

b. protected abstract void method() { }

c. private void method() {}

d. void method() {}

e. None of the options is valid.

137. ¿Cuál de las siguientes opciones es verdadera acerca de Spring Batch?

a. Marco de trabajo basado en jobs y steps.

b. Es una abstracción que representa la salida de un Step.

c. Es una abstracción que representa la recuperación de la entrada de un Step.

d. Es una abstracción que representa el procesamiento comercial de un Item.

e. Marco de trabajo para lotes diseñado para permitir el desarrollo de aplicaciones por lotes.

138. Dado el siguiente código:

```
import java.util.function.Predicate;

class Foo {

    int i;

    Foo(int i) {

        this.i = i;

    }

}

public class Test {

    static boolean compare(Foo foo, Predicate<Foo> predicate) {

        return predicate.test(foo);

    }

public static void main(String[] args) {
```

```
    Predicate<Foo> p = f -> f.i > 0; // line 1

    p = p.or(f -> f.i > 0).and(f -> f.i < 0); // line 2

    boolean b = compare(new Foo(1), p); // line 3

    System.out.println(b);

}

}
```

¿Cuál será la salida del código anterior?

a. Compilación falla en la línea 1.

b. Compilación falla en la línea 3.

c. true

d. false

e. Compilación falla en la línea 2.

Analicemos cuidadosamente el código paso a paso:

1. Se define la clase `Foo` con un atributo `int i`.
2. En `main`, se crea un `Predicate<Foo>`:

```
Predicate<Foo> p = f -> f.i > 0; // line 1
```

Esto es válido. ✅

3. Luego se combina el predicate con operadores lógicos:

```
p = p.or(f -> f.i > 0).and(f -> f.i < 0); // line 2
```

- `p.or(f -> f.i > 0)` crea un nuevo Predicate. ✅
- `.and(f -> f.i < 0)` se aplica sobre ese Predicate. ✅

Todo esto es **válido sintácticamente** y compilará correctamente.

4. Finalmente se llama al método `compare` con un objeto `Foo(1)`:

```
boolean b = compare(new Foo(1), p); // line 3
```

- Evaluamos el predicate combinado: `(f.i > 0 OR f.i > 0) AND (f.i < 0)`
- Para `f.i = 1`:
    - `(1 > 0 OR 1 > 0) = true`
    - `(1 < 0) = false`
    - `true AND false = false`

Por lo tanto, el resultado es **false**.

Respuesta correcta:

d. false

139. public class Test {

public static void main(String... arguments) {

final char a = 'A', d = 'D';

char grade = 'B';

```
        switch (grade) {
            case a:
            case 'B':
                System.out.print("B");
            case 'C':
                System.out.print("C");
                break;
            case d:
            case 'F':
                System.out.print("F");
        }
    }
}
```

a. B

b. Compilation fails

c. C

d. A

e. AC

f. BC

Analicemos el código paso a paso:

1. Se definen variables finales `a = 'A'` y `d = 'D'`. ✅
    - En un `switch`, se permite usar **constantes `final`** como `case labels`.
2. La variable `grade = 'B'`.
3. Evaluando el `switch(grade)`:

```
switch (grade) {
    case a:         // 'A'
    case 'B':       // coincide con 'B'
        System.out.print("B");
    case 'C':
        System.out.print("C");
        break;
    case d:         // 'D'
    case 'F':
        System.out.print("F");
}
```

- `grade` es `'B'`, así que entra en `case 'B'`.
- Se imprime `"B"`.
- No hay `break` después, así que **cae por "fall-through"** al siguiente `case 'C'`.
- Se imprime `"C"`.
- Luego encuentra `break` y sale del `switch`.

Salida final:

BC

Respuesta correcta:

f. BC

140. public class Test {

String value = "A";

{

value += "B";

}

{

value += "C";

}

public Test() {

value += "x";

}

public Test(String s) {

value += s;

```
    }

    public static void main(String[] args) {

        Test m = new Test("M");

        m = new Test();

        System.out.println(m.value);

    }

}
```

a. ABCXM

b. ABCMABCB

c. ABCM

d. ABCX

e. An exception is thrown

f. the code does not compile

Analicemos paso a paso la ejecución del código:

1. La clase `Test` tiene un campo `value` inicializado como `"A"`.
2. Hay dos bloques de inicialización de instancia:

```
{
    value += "B";
}
{
    value += "C";
```

```
}
```

Estos bloques se ejecutan **antes del constructor**, en el orden en que aparecen.

3. Constructores:

```
public Test() {
    value += "x";
}

public Test(String s) {
    value += s;
}
```

4. En `main`:

```
Test m = new Test("M");
```

- Se ejecuta el constructor `Test(String s)`.
- Pasos de inicialización:
    1. `value = "A"`
    2. Primer bloque: `value += "B"` → `"AB"`
    3. Segundo bloque: `value += "C"` → `"ABC"`
    4. Constructor `Test(String s)`: `value += "M"` → `"ABCM"`

Luego:

```
m = new Test();
```

- Se ejecuta el constructor sin argumentos.
- Pasos de inicialización:
    1. `value = "A"` (nuevo objeto)
    2. Primer bloque: `value += "B"` → `"AB"`
    3. Segundo bloque: `value += "C"` → `"ABC"`
    4. Constructor `Test()`: `value += "x"` → `"ABCx"`

Finalmente:

```
System.out.println(m.value);
```

- Imprime `"ABCx"`.

✅ **Respuesta correcta:**

d. ABCX

141. Which statement is true about the following code?

```
abstract class Shape {
    public abstract void draw();
    public void printShape() {
        System.out.println("This is a shape");
    }
```

```
}

class Circle extends Shape {

    @Override

    public void draw() {

        System.out.println("Drawing a circle");

    }

}

public class Test {

    public static void main(String[] args) {

        Circle circle = new Circle();

        circle.draw();

        circle.printShape();

    }

}
```

a. The program will print "Drawing a circle" followed by "This is a shape".

b. Compilation error because the Shape class is abstract.

c. The program will print "This is a shape" followed by "Drawing a circle".

d. Compilation error because the Circle class does not implement the draw method.

Let's analyze the code:

1. `Shape` is an **abstract class** with:
    - An abstract method `draw()`
    - A concrete method `printShape()`
2. `Circle` **extends Shape** and **overrides** `draw()` correctly.
3. In `main`, we create a `Circle` instance and call:

```
circle.draw();       // prints "Drawing a circle"
circle.printShape(); // prints "This is a shape"
```

✅ There are **no compilation errors**, and the methods execute as expected.

Correct answer:
a. The program will print "Drawing a circle" followed by "This is a shape".

142. ¿Cuál es un antipatrón de Binary Large Object (BLOB)?

Una librería compuesta de sola clase con gran cantidad de funcionalidad.

"The Blob" normalmente es muy complicada de reutilizar y probar, por lo cual, la solución es realizar un "refactoring" del rediseño entre las librerías involucradas.

La opción correcta es:

Una librería compuesta de sola clase con gran cantidad de funcionalidad.

Este es un **antipatrón BLOB** porque concentra demasiada responsabilidad en una sola clase, lo que la hace difícil de mantener, reutilizar y probar. La recomendación es **refactorizar** la clase, distribuyendo la funcionalidad en varias clases más cohesivas.

143. Which of the following code snippets will result in a compilation error when implementing the Vehicle interface?

```
interface Vehicle {

    void start();

    void stop();

}
```

Opción a:

```
public class Scooter implements Vehicle {

    public void start() {

        System.out.println("Scooter starts");

    }

    public void stop() {

        System.out.println("Scooter stops");

    }

}
```

Opción b:

```
public class Bike implements Vehicle {
```

```
    public void start() {
        System.out.println("Bike starts");
    }
    void stop() {
        System.out.println("Bike stops");
    }
}
```

Opción c:

```
public class Truck implements Vehicle {
    public void start() {
        System.out.println("Truck starts");
    }
    public void stop() {
        System.out.println("Truck stops");
    }
    public void load() {
        System.out.println("Truck loads");
```

```
    }
}
```

Opción d:

```
public class Car implements Vehicle {
    public void start() {
        System.out.println("Car starts");
    }
    public void stop() {
        System.out.println("Car stops");
    }
}
```

Opción a

Opción b

Opción c

Opción d

La opción correcta es:

Opción b ✅

Justificación:
En la clase `Bike`, el método `stop()` tiene el modificador de acceso **default** (`void stop()`), mientras que en la interfaz `Vehicle` todos los métodos son **públicos por defecto**.

Al implementar un método de interfaz, **debe declararse como `public`**, de lo contrario el compilador lanzará un error.

Las demás opciones (a, c, d) implementan correctamente todos los métodos públicos de la interfaz.

144. ¿Cuál de los siguientes fragmentos de código implementa correctamente Comparator para ordenar una lista de objetos Person por su edad?

```
import java.util.Comparator;

class Person {

    private String name;

    private int age;

    public Person(String name, int age) {

        this.name = name;

        this.age = age;

    }

    public int getAge() {

        return age;

    }

    public String getName() {
```

```
        return name;
    }
}
```

Opción a)

```
import java.util.Comparator;
public class AgeComparator implements Comparator<Person> {
    @Override
    public int compare(Person p1, Person p2) {
        return p1.getAge() - p2.getAge();
    }
}
```

Opción b)

```
import java.util.Comparator;
public class AgeComparator implements Comparator<Person> {
    @Override
    public int compare(Person p1, Person p2) {
        return p1.getName().compareTo(p2.getName());
```

```
    }
}
```

Opción c)

```
import java.util.Comparator;

public class AgeComparator implements Comparator<Person> {

    @Override

    public int compare(Person p1, Person p2) {

        return Integer.compare(p1.getAge(), p2.getAge());

    }

}
```

Opción d)

```
import java.util.Comparator;

public class AgeComparator implements Comparator<Person> {

    @Override

    public int compare(Person p1, Person p2) {

        return p2.getAge() - p1.getAge();

    }
```

}

a

b

c

d

145. public class Test {

___ doMath (long l, short s) {

return s / (l * 10);

}

}

What is the narrowest valid return type for the calculate Method?

a. float

b. short

c. int

d. long

e. double

f. byte

The expression in the method is:

```
s / (l * 10)
```

- `s` is **short**, `l` is **long**, and `10` is an **int** literal.
- `l * 10` → **long × int = long**
- `s / (l * 10)` → **short / long = long** (the result of dividing a smaller type by a long is promoted to long).

So the **narrowest valid return type** is:

d. long ✅

146. In Java, which class is the superclass of all errors and exceptions?

a. Object

b. RuntimeException

c. Exception

d. Error

e. Throwable

In Java, both **exceptions** and **errors** are subclasses of the **`Throwable`** class.

- `Exception` and `Error` both extend `Throwable`.
- `Object` is the root of all classes but not specifically for errors/exceptions.
- `RuntimeException` is a subclass of `Exception`.

✅ Correct answer: **e. Throwable**

147. Todo el acceso a los datos debe estar encapsulado en una biblioteca para facilitar la reutilización y el control de acceso

verdadero

Falso

La afirmación es ✅ **Verdadero**.

En buenas prácticas de desarrollo:

- Todo el acceso a los datos debe estar **encapsulado en una capa o biblioteca** (por ejemplo, un *Data Access Layer* o DAO).
- Esto permite **reutilización**, **mantenimiento**, y sobre todo un mejor **control de acceso** (seguridad, validación, logging).
- Si el acceso a datos se dispersa por toda la aplicación, se dificulta el control y la reutilización.

👉 Respuesta correcta: **Verdadero** ✅

148. ¿Cuál es la característica que comparten Array y Arraylist?

Pueden almacenar valores nulos.

Ambos procesos ocurren en tiempo constante.

Es posible que tengan valores duplicados.

Array y ArrayList no garantizan la presencia de elementos ordenados.

La característica que **sí comparten** tanto un **Array** como un **ArrayList** en Java es:

✅ **Es posible que tengan valores duplicados.**

Justificación:

- **Array**: puede contener elementos repetidos (duplicados) y también valores `null` si es de tipo referencia.
- **ArrayList**: al ser una implementación de `List`, también permite **duplicados** y valores `null`.

- **Orden**: ambos **mantienen el orden de inserción**, así que la opción "no garantizan la presencia de elementos ordenados" es **falsa**.
- **Tiempo constante**: no todos los procesos son en tiempo constante (por ejemplo, insertar en medio de un `ArrayList` no lo es).

👉 **Respuesta correcta: "Es posible que tengan valores duplicados." ✅**

149.

```
abstract class Animal {
    void speak() {
        System.out.println("Animals make noise");
    }
    public static void main(String[] args) {
        Animal animal = new Dog();
        animal.speak();
    }
}
class Dog extends Animal {
    protected void speak() {
        System.out.println("The Dog barks");
```

```
    }
}
public class Test {
    public static void main(String... strings) {
        Animal.main(strings);
    }
}
```

a. Animals make noise

b. A runtime exception is thrown

c. Compilation fails

d. The Dog barks

150. ¿Cuál será el resultado del siguiente fragmento de código?

```
public class OverloadedConstructorsTest {
    private String name;
    private int age;
    public OverloadedConstructorsTest() {
        this("Unknown", 0);
```

```
        System.out.println("Default constructor called");
    }
    public OverloadedConstructorsTest(String name) {
        this(name, 0);
        System.out.println("Constructor with name called");
    }
    public OverloadedConstructorsTest(String name, int age) {
        this.name = name;
        this.age = age;
        System.out.println("Constructor with name and age called");
    }
    public static void main(String[] args) {
    OverloadedConstructorsTest obj1 = new OverloadedConstructorsTest();
    OverloadedConstructorsTest obj2 = new OverloadedConstructorsTest("Alice");
    OverloadedConstructorsTest obj3 = new OverloadedConstructorsTest("Bob", 30);
    }
}
```

Opción a:

Default constructor called

Constructor with name and age called

Constructor with name called

Constructor with name and age called

Constructor with name and age called

Opción b:

Constructor with name and age called

Default constructor called

Constructor with name called

Constructor with name and age called

opción c:

Constructor with name and age called

Default constructor called

Constructor with name and age called

Constructor with name called

Constructor with name and age called

opción d:

Default constructor called

Constructor with name called

Constructor with name and age called

a

b

c

d

Perfecto, vamos paso a paso 👇

Código analizado

```
public OverloadedConstructorsTest() {
    this("Unknown", 0);
    System.out.println("Default constructor called");
}

public OverloadedConstructorsTest(String name) {
    this(name, 0);
    System.out.println("Constructor with name called");
}

public OverloadedConstructorsTest(String name, int age) {
    this.name = name;
    this.age = age;
    System.out.println("Constructor with name and age called");
}
```

Ejecución en `main`

```
OverloadedConstructorsTest obj1 = new OverloadedConstructorsTest();

OverloadedConstructorsTest obj2 = new
OverloadedConstructorsTest("Alice");

OverloadedConstructorsTest obj3 = new
OverloadedConstructorsTest("Bob", 30);
```

Caso 1 → `new OverloadedConstructorsTest();`

1. Llama al constructor **sin parámetros**.

2. Ese constructor llama a `this("Unknown", 0)` → constructor con nombre y edad.
   → imprime **"Constructor with name and age called"**

3. Después sigue la línea del constructor por defecto.
   → imprime **"Default constructor called"**

📌 **Salida parcial:**

```
Constructor with name and age called

Default constructor called
```

Caso 2 → `new OverloadedConstructorsTest("Alice");`

1. Llama al constructor con `String name`.

2. Este llama a `this(name, 0)` → constructor con nombre y edad.
   → imprime **"Constructor with name and age called"**

3. Luego continúa y ejecuta su propio `System.out.println`.
   → imprime **"Constructor with name called"**

📌 **Salida parcial:**

```
Constructor with name and age called
Constructor with name called
```

Caso 3 → `new OverloadedConstructorsTest("Bob", 30);`

1. Llama directamente al constructor con `(String, int)`.
2. Ese imprime "**Constructor with name and age called**".

📌 **Salida parcial:**

```
Constructor with name and age called
```

✅ Salida completa final

```
Constructor with name and age called
Default constructor called
Constructor with name and age called
Constructor with name called
Constructor with name and age called
```

Respuesta correcta:

c

151. ¿Cuál de las siguientes opciones es una herramienta de comprensión y gestión de proyectos basada en el concepto de modelo de objetos de proyecto (POM)?

a. Git

b. IDE

c. Artifactory

d. Maven

e. Docker

La respuesta correcta es:

d. Maven ✅

Justificación:

- **Maven** es una herramienta de gestión y comprensión de proyectos Java (aunque puede usarse con otros lenguajes), basada en el concepto de **POM (Project Object Model)**.
    - El archivo `pom.xml` describe la configuración del proyecto, sus dependencias, plugins, ciclo de vida de construcción, etc.
    - Facilita la **compilación, empaquetado, gestión de dependencias y despliegue**.
- **Git** → Sistema de control de versiones.
- **IDE** → Entorno de desarrollo (Eclipse, IntelliJ, NetBeans).
- **Artifactory** → Repositorio de artefactos (gestiona dependencias).
- **Docker** → Plataforma de contenedores.

👉 Solo **Maven** está basado directamente en el concepto de **POM**.

152. import java.util.Arrays;

import java.util.Comparator;

```
class SortLongs implements Comparator<Long> {
    @Override
    public int compare(Long a, Long b) {
        return b.compareTo(a);
    }
}
public class Test {
    public static void main(String[] args) {
        Long[] values = {20L, 70L, 50L, 30L};
        SortLongs sort = new SortLongs();
        Arrays.sort(values, sort);
        System.out.println(Arrays.toString(values));
    }
}
```

What is the result?

a. An ArrayIndexOutOfBoundsException is thrown.

b. [20, 70, 50, 30]

c. [20, 30, 50, 70]

d. [70, 50, 30, 20]

e. Compilation fails.

La clave está en analizar el **Comparator**:

```
@Override
public int compare(Long a, Long b) {
    return b.compareTo(a);
}
```

- `b.compareTo(a)` → ordena en **orden descendente** (el mayor va primero).
- El arreglo inicial es: `{20L, 70L, 50L, 30L}`.
- Después de aplicar `Arrays.sort(values, sort)`, el resultado será:

```
[70, 50, 30, 20]
```

✅ **Respuesta correcta:**
d. [70, 50, 30, 20]

153. public class NestedForLoopTest {

public static void main(String[] args) {

int[][] matrix = {

{1, 2, 3},

```
            {4, 5, 6},
            {7, 8, 9}
        };
        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[i].length; j++) {
                if (i == j) {
                    System.out.print(matrix[i][j] + " ");
                }
            }
        }
    }
}
```

a. 147

b. 123456789

c. 159

d. 159246837

Vamos a analizarlo 👇

El código:

```
int[][] matrix = {
```

```
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        if (i == j) {
            System.out.print(matrix[i][j] + " ");
        }
    }
}
```

Paso a paso:

- El `if (i == j)` imprime solo los **elementos de la diagonal principal** de la matriz.
- La matriz es:

```
1 2 3
4 5 6
7 8 9
```

- Elementos con `i == j`:
    - `matrix[0][0] = 1`
    - `matrix[1][1] = 5`
    - `matrix[2][2] = 9`

Salida:

```
1 5 9
```

✅ **Respuesta correcta:**
c. 159

154. En el contexto de Maven. ¿Cuál es la función principal del archivo settings.xml?

a. Ninguna opción es correcta.

b. Configurar la información del repositorio local y remoto, así como las credenciales y perfiles de usuario.

c. Especificar las dependencias del proyecto.

d. Definir la estructura de directorios del proyecto.

e. Generar informes de construcción y documentación.

En **Maven**, el archivo `settings.xml` **no es parte del proyecto**, sino de la configuración de Maven en la máquina local (ubicado típicamente en `~/.m2/settings.xml`).

Su función principal es:

- Definir el **repositorio local**.
- Configurar **repositorios remotos** adicionales.
- Guardar **credenciales** (usuario/contraseña) para repositorios privados.
- Configurar **perfiles de usuario** (que permiten personalizar el comportamiento de Maven para distintos entornos).

✅ **Respuesta correcta:**
b. Configurar la información del repositorio local y remoto, así como las credenciales y perfiles de usuario.

155. abstract class Animal {

```
    private void speak() {
        System.out.println("Animals make noise");
    }
    public static void main(String[] args) {
        Animal animal = new Dog();
        animal.speak();
    }
}
class Dog extends Animal {
    protected void speak() {
        System.out.println("The Dog barks");
    }
}
```

```
public class Test {

    public static void main(String... strings) {

        Animal.main(strings);

    }

}
```

a. Animals make noise

b. Compilation fails.

c. The Dog barks

d. A runtime exception is thrown.

Vamos a analizar el código paso a paso 👇

Código:

```
abstract class Animal {
    private void speak() {
        System.out.println("Animals make noise");
    }

    public static void main(String[] args) {
        Animal animal = new Dog();
        animal.speak();
    }
}

class Dog extends Animal {
```

```
    protected void speak() {
        System.out.println("The Dog barks");
    }
}

public class Test {
    public static void main(String... strings) {
        Animal.main(strings);
    }
}
```

Puntos importantes:

1. En **`Animal`**, el método `speak()` es **`private`**, por lo que:
    - **No se hereda** en la subclase `Dog`.
    - **No se puede sobrescribir** (override).
    - Sólo puede ser llamado **dentro de la clase `Animal`**.

Dentro de `Animal.main`, se hace:

```
Animal animal = new Dog();
animal.speak();
```

2. Aquí se invoca **el método `private speak()` de `Animal`**, no el de `Dog`.
   Esto es porque **la resolución de métodos privados ocurre en tiempo de compilación** (estático), no dinámico.

3. El método `Dog.speak()` nunca será llamado en este programa.
   Aunque parece sobrescribir, en realidad **es un método nuevo** con otra visibilidad, sin relación con el `private` de `Animal`.

Resultado:

El programa imprimirá:

```
Animals make noise
```

✅ **Respuesta correcta:**
a. Animals make noise

156. ¿Cuál de las siguientes afirmaciones sobre la palabra clave throw es verdadera?

a. Se utiliza para detectar excepciones generadas por otros métodos.

b. Se utiliza para declarar que un método puede generar una excepción.

c. Se utiliza para definir el código de limpieza que se debe ejecutar.

d. Se utiliza para generar una excepción manualmente.

La pregunta es sobre la palabra clave **`throw`** en Java.

Opciones:

a. ❌ *Se utiliza para detectar excepciones generadas por otros métodos.*
👉 Eso lo hace `try-catch`, no `throw`.

b. ❌ *Se utiliza para declarar que un método puede generar una excepción.*
👉 Eso lo hace `throws`, no `throw`.

c. ❌ *Se utiliza para definir el código de limpieza que se debe ejecutar.*
👉 Eso lo hace `finally`.

d. ✅ *Se utiliza para generar una excepción manualmente.*
👉 Correcto. Ejemplo:

```
throw new IllegalArgumentException("Valor inválido");
```

✅ Respuesta correcta:

d. Se utiliza para generar una excepción manualmente.

157. ¿Cuál de las siguientes anotaciones es verdadera acerca de la siguiente jerarquía de clases?

```
class Vehicle {

    String type = "Vehicle";

}

class Car extends Vehicle {

    String type = "Car";

}

class ElectricCar extends Car {

    String type = "ElectricCar";
```

}

a) ElectricCar myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: ElectricCar

b) Car myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: Vehicle

c) ElectricCar myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: Vehicle

d) ElectricCar myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: Car

En Java, **las variables de instancia no se polimorfizan**. Esto significa que **el tipo de referencia determina qué campo se accede**, no el tipo del objeto en tiempo de ejecución. Solo los métodos se comportan de forma polimórfica.

Veamos las opciones:

```
class Vehicle { String type = "Vehicle"; }

class Car extends Vehicle { String type = "Car"; }

class ElectricCar extends Car { String type = "ElectricCar"; }
```

Opción a)

```
ElectricCar myCar = new ElectricCar();

System.out.println(myCar.type); // ElectricCar
```

✅ Correcta. La referencia es `ElectricCar`, accede al campo `type` de `ElectricCar`.

Opción b)

```
Car myCar = new ElectricCar();

System.out.println(myCar.type); // Vehicle
```

❌ Incorrecta. Accede al campo `type` de **Car**, no Vehicle.

Opción c)

```
ElectricCar myCar = new ElectricCar();

System.out.println(myCar.type); // Vehicle
```

❌ Incorrecta. Accede al campo `type` de `ElectricCar`.

Opción d)

```
ElectricCar myCar = new ElectricCar();

System.out.println(myCar.type); // Car
```

❌ Incorrecta. Igual que opción c, el campo es de `ElectricCar`.

✅ Respuesta correcta:

a) ElectricCar myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: ElectricCar

158. Which of the following statements correctly identifies checked and unchecked exceptions in Java?

a. SQLException is an unchecked exception, and ArrayIndexOutOfBoundsException is a checked exception.

b. IOException is a checked exception, and NullPointerException is an unchecked exception.

c. RuntimeException is a checked exception, and FileNotFoundException is an unchecked exception.

d. ArithmeticException is a checked exception, and ClassNotFoundException is an unchecked exception.

La distinción clave en Java es:

- **Checked exceptions**: Deben ser declaradas o capturadas (`try-catch`) en tiempo de compilación. Ejemplo: `IOException`, `SQLException`.
- **Unchecked exceptions**: No requieren declaración ni captura, ocurren en tiempo de ejecución. Generalmente son subclases de `RuntimeException`. Ejemplo: `NullPointerException`, `ArrayIndexOutOfBoundsException`.

Analizando las opciones:

- **a.** Incorrecta: `SQLException` **es checked**, `ArrayIndexOutOfBoundsException` **es unchecked**.
- **b.** ✅ Correcta: `IOException` es **checked**, `NullPointerException` es **unchecked**.
- **c.** Incorrecta: `RuntimeException` es **unchecked**, `FileNotFoundException` es **checked**.
- **d.** Incorrecta: `ArithmeticException` es **unchecked**, `ClassNotFoundException` es **checked**.

Respuesta correcta:

b. IOException is a checked exception, and NullPointerException is an unchecked exception.

159. ¿Cuáles de las siguientes opciones son verdaderas acerca de APX?

a. Ayuda a aumentar el uso de Mainframe.

b. Ninguna opción es correcta.

c. Ofrece una alternativa confiable para el desarrollo de transacciones no acopladas al canal.

d. Biblioteca de métodos para crear un punto de contacto entre las tecnologías Frontend y Backend.

e. Extensión de la Arquitectura Backend que brinda las mismas capacidades en el mundo distribuido

Analizando las opciones:

- **a.** Incorrecta. APX no tiene como objetivo principal aumentar el uso de Mainframe.
- **b.** Incorrecta. Sí existen opciones correctas.
- **c.** ✅ Correcta. APX permite desarrollar transacciones desacopladas del canal, facilitando la reutilización y portabilidad.
- **d.** Incorrecta. APX no es únicamente una biblioteca de métodos para integrar Frontend y Backend; su alcance es más amplio.
- **e.** ✅ Correcta. APX es una extensión de la arquitectura backend que permite trasladar capacidades del mundo mainframe a entornos distribuidos.

Respuesta correcta:

c y e

160. ¿Cuáles son los tipos de componentes en APX batch?

a. Librerías

b. Jobs

c. Todas las opciones son correctas

d. DTOs

e. Transacciones

161. What is the output of the following program?

```
public class Test {

    public static void main(String[] args) {

        int x = 0;

        while (x++ < 5) {

        }

        String message = x > 5 ? "greater than" : "smaller than";

        System.out.println(message + " - " + x);

    }

}
```

a. smaller than - 6

b. greater than - 6

c. smaller than - 5

d. Compilation fails.

e. greater than - 5

Let's analyze the code carefully:

```
int x = 0;
while (x++ < 5) {
    // empty loop
}
```

1. `x` starts at 0.
2. The condition `x++ < 5` uses the **post-increment**, meaning the comparison uses the current value of `x` before incrementing.
    - Iteration 1: `0 < 5` → true → `x` becomes 1
    - Iteration 2: `1 < 5` → true → `x` becomes 2
    - Iteration 3: `2 < 5` → true → `x` becomes 3
    - Iteration 4: `3 < 5` → true → `x` becomes 4
    - Iteration 5: `4 < 5` → true → `x` becomes 5
    - Iteration 6: `5 < 5` → false → loop exits → `x` is incremented to 6

After the loop, `x = 6`.

```
String message = x > 5 ? "greater than" : "smaller than";
```

- Since `x = 6`, `x > 5` is **true** → `message = "greater than"`.

```
System.out.println(message + " - " + x);
```

- Prints: `"greater than - 6"` ✅

Answer:
b. greater than - 6

162. public class Test implements Runnable {

```
boolean flag = true;

@Override

public synchronized void run() {

    if (flag) {

        flag = false;

        System.out.print("1 ");

        try {

            this.wait();

        } catch (Exception e) {

        }

        System.out.print("2 ");
```

```
        } else {
            flag = true;
            System.out.print("3 ");
            try {
                Thread.sleep(2000);
            } catch (Exception e) {
            }
            System.out.print("4 ");
            notify();
        }
    }
    public static void main(String[] args) {
        Test w = new Test();
        new Thread(w).start();
        new Thread(w).start();
    }
}
```

Which of the following will be the output?

a. The code outputs 1 3 4.

b. The code outputs 1 2 3 4.

c. The code outputs 3 4 1.

d. The code outputs 1 3 4 2.

e. The code never completes.

f. The code runs to completion.

163. ¿Cuáles son palabras reservadas para manejo de excepciones en Java?

a. throw & throws

b. Error, Exception, RuntimeException, Throwable

c. final, static, catch, throws

d. try, catch, finally, throw, throws

e. try, catch, final, throw, throws

La respuesta correcta es:

d. try, catch, finally, throw, throws ✅

Justificación:

- **try**: bloque donde se coloca el código que puede generar una excepción.
- **catch**: bloque que maneja la excepción lanzada en el `try`.
- **finally**: bloque que siempre se ejecuta, haya o no excepción, normalmente usado para liberar recursos.
- **throw**: se utiliza para lanzar una excepción de forma explícita.

- **throws**: se utiliza en la declaración de un método para indicar que puede lanzar ciertas excepciones.

Las otras opciones incluyen palabras que no son reservadas de manejo de excepciones o clases de excepciones (que no son palabras reservadas).

164. ¿Cuál de las siguientes implementaciones de código utiliza correctamente una clase genérica en Java?

a)

```
public class Box {
    private Object value;
    public Box(Object value) {
        this.value = value;
    }
    public Object getValue() {
        return value;
    }
}
```

b)

```
public class Box {
    private int value;
    public Box(int value) {
        this.value = value;
    }
    public int getValue() {
        return value;
    }
}
```

c)

```
public class Box<T extends Number> {
    private T value;
    public Box(T value) {
        this.value = value;
    }
    public T getValue() {
        return value;
```

```
    }
}
```

d)

```
public class Box<T> {
    private T value;
    public Box(T value) {
        this.value = value;
    }
    public T getValue() {
        return value;
    }
}
```

a)

b)

c)

d)

165. ¿Cuál de las siguientes opciones son capacidades ofrecidas en APX?

a. Todas las opciones son correctas.

b. Procesamiento transaccional.

c. Procesamiento Batch.

d. Cold Deployment.

e. Sin integración con servicios de seguridad.

La respuesta correcta es:

b. Procesamiento transaccional y **c. Procesamiento Batch**.

Justificación:

- APX ofrece **procesamiento transaccional** y **procesamiento por lotes (Batch)** para gestionar operaciones de manera eficiente tanto en línea como en procesos masivos.
- No todas las opciones listadas son capacidades de APX:
    - **Cold Deployment** no es una característica específica de APX.
    - **Sin integración con servicios de seguridad** no es una capacidad, sino más bien una limitación, y APX sí puede integrarse con mecanismos de seguridad según la implementación.

Por lo tanto, la opción **a. Todas las opciones son correctas** sería **incorrecta**.

166. hat will happen when you try to compile the following code?

```
class Parent {

   protected void show() {

      System.out.println("Parent show");

   }
```

```
}

class Child extends Parent {

    @Override

    void show() {

        System.out.println("Child show");

    }

}

public class Test {

    public static void main(String[] args) {

        Child child = new Child();

        child.show();

    }

}
```

a. It will print "Parent show".

b. It will print "Child show".

c. Compilation error.

d. It will print "Parent show" followed by "Child show".

The correct answer is:

c. Compilation error. ✅

Explanation:

- In Java, when overriding a method, the **access level cannot be more restrictive** than the method in the superclass.
- In the code:
    - `Parent.show()` is **protected**.
    - `Child.show()` is **default (package-private)** because no access modifier is specified.
- Since default access is **more restrictive** than protected, the compiler will throw an **error**.

To fix it, you could declare `Child.show()` as `protected` or `public`.

167. ¿Cuáles de las siguientes opciones son archivos configurados por APX CLI al agregar JDBC Utility?

a. library-arc-osgi.xml

b. library-arc.xml

c. config.xml

d. settings.xml

e. pom.xml

168. Con la "APX Operation Console", sólo se pueden ver grafos de dependencias para las librerías. Seleccione una:

Verdadero

Falso

Falso ✅

Justificación:
La **APX Operation Console** no se limita únicamente a mostrar grafos de dependencias de librerías. También permite visualizar **dependencias entre otros tipos de componentes**, como transacciones, jobs y DTOs, así como el estado y ejecución de componentes dentro del entorno APX. Por lo tanto, su alcance es más amplio que solo las librerías.

169. ¿Cual opción es verdadera acerca de "APX Operation Console"?

a. Herramienta web que proporciona a los desarrolladores y administradores gestionar la configuración, verificación de componentes de la aplicación o busqueda de jobs ejecutados o en ejecución

b. Herramienta de línea de comandos para inicializar proyectos de tipo APX.

c. Herramienta para administrar las dependencias de un proyecto APX.

d. Herramienta para agregar los parámetros de entrada y salida de una transacción o job APX.

e. Ninguna opción es correcta.

La opción correcta es:

a. Herramienta web que proporciona a los desarrolladores y administradores gestionar la configuración, verificación de componentes de la aplicación o búsqueda de jobs ejecutados o en ejecución ✅

Justificación:
La **APX Operation Console** es una **herramienta web** diseñada para que desarrolladores y administradores puedan **visualizar, monitorear y gestionar** los componentes del entorno APX, incluyendo transacciones, jobs y librerías, así como consultar el estado de ejecución y dependencias de estos componentes. Las otras opciones describen funciones de otras herramientas o acciones específicas que no corresponden a la consola.

170. ¿Cuáles son los tipos de componentes en APX?

a. Librerías

b. Jobs

c. Todas las opciones son correctas

d. DTOs

e. Transacciones

La respuesta correcta es:

c. Todas las opciones son correctas ✅

Justificación:
En **APX**, los tipos de componentes incluyen:

- **Librerías:** Contienen código reutilizable y utilidades.
- **Jobs:** Procesos batch que ejecutan tareas de manera programada o por lotes.
- **DTOs (Data Transfer Objects):** Objetos que transportan datos entre capas de la aplicación.
- **Transacciones:** Unidades de ejecución de negocio que se invocan desde el frontend o batch.

Por lo tanto, todas las opciones mencionadas son tipos válidos de componentes en APX.

171. ¿Qué significa que un cambio en el software sea retrocompatible?

a. El cambio garantiza que el software siga funcionando sin necesidad de modificaciones en el código que depende de él.

b. El cambio puede romper la funcionalidad existente, requiriendo ajustes en el código que depende del software.

c. El cambio corrige errores menores y realiza mejoras de rendimiento.

d. El cambio introduce nuevas funcionalidades que no afectan el comportamiento existente del software.

La respuesta correcta es:

a. El cambio garantiza que el software siga funcionando sin necesidad de modificaciones en el código que depende de él. ✅

Justificación:
Un cambio **retrocompatible** (o *backward compatible*) significa que las aplicaciones, librerías o sistemas que dependían de la versión anterior del software continuarán funcionando correctamente sin necesidad de modificaciones. Esto asegura que las nuevas versiones no rompan la funcionalidad existente de los consumidores del software.

172. ¿Qué comando en Bash se utiliza para copiar archivos de un lugar a otro?

a. copy

b. mv

c. cp

d. zip

e. duplicate

La respuesta correcta es:

c. cp ✅

Justificación:
En Bash (y en la mayoría de shells de Unix/Linux), el comando `cp` se utiliza para **copiar archivos o directorios** de una ubicación a otra. Por ejemplo:

```
cp archivo1.txt /ruta/destino/
```

- `mv` sirve para mover o renombrar archivos.
- `copy` y `duplicate` **no existen** como comandos estándar en Bash.
- `zip` se usa para comprimir archivos, no para copiarlos.

173. ¿Cuál es el propósito principal de un objeto de transferencia de datos (DTO) en el diseño de software?

a. Encapsular la lógica empresarial dentro de la aplicación.

b. Transferir datos entre diferentes capas o niveles de una aplicación.

c. Manejar operaciones de base de datos directamente.

d. Proporcionar una interfaz de usuario para la entrada de datos.

La respuesta correcta es:

b. Transferir datos entre diferentes capas o niveles de una aplicación. ✅

Justificación:
Un **DTO (Data Transfer Object)** se utiliza para **transportar datos** entre distintas capas de una aplicación, como la capa de presentación, la capa de negocio y la capa de persistencia, sin exponer la lógica de negocio ni el estado interno de los objetos de dominio.

- No encapsula lógica empresarial (eso lo hacen los objetos de dominio o servicios).
- No maneja operaciones de base de datos directamente (eso es responsabilidad de DAOs o repositorios).
- No proporciona interfaces de usuario; solo transporta información.

174. ¿Cuál es la función principal de JobRepository en Spring Batch?

a. Configurar y gestionar los recursos necesarios para la ejecución de un trabajo.

b. Proporcionar un mecanismo de persistencia para todas las operaciones CRUD relacionadas con JobLauncher, Job y Step.

c. Ejecutar los trabajos (Jobs) en paralelo para mejorar el rendimiento.

d. Realizar la validación de los datos antes de procesarlos en un trabajo.

La respuesta correcta es:

b. Proporcionar un mecanismo de persistencia para todas las operaciones CRUD relacionadas con JobLauncher, Job y Step. ✅

Justificación:
En **Spring Batch**, el **JobRepository** es el componente encargado de **almacenar el estado y la metadata** de la ejecución de jobs y steps. Esto incluye información sobre:

- Ejecuciones de jobs (`JobExecution`)
- Ejecuciones de steps (`StepExecution`)
- Parámetros de job y step

Permite que Spring Batch realice **reinicios, reintentos y auditorías** de manera confiable, asegurando la persistencia de los resultados y el seguimiento del progreso de los jobs.

Otros roles mencionados en las opciones, como ejecutar trabajos en paralelo o validar datos, son responsabilidad de otros componentes, no del `JobRepository`.

175. class Ot {

```
    enum Colors {

        RED, GREEN, BLUE, YELLOW, BLACK

    };

}

public class Test {

    public static void main(String[] String) {

        for (Ot.Colors c : Ot.Colors.values()) {

            if (Ot.Colors.RED.equals(c)) {
```

```
            System.out.print("red ");

        }

        if (c == Ot.Colors.GREEN) {

            System.out.print("green ");

        }

        if (c.equals("BLUE")) {

            System.out.print("blue ");

        }

    }

  }

}
```

What is the result?

a. red blue

b. green blue

c. red green blue

d. red green

e. An exception is thrown at runtime.

f. Compilation fails.

La respuesta correcta es:

d. red green ✅

Justificación:

1. `Ot.Colors.RED.equals(c)` → true cuando `c` es `RED` → imprime `"red "`
2. `c == Ot.Colors.GREEN` → true cuando `c` es `GREEN` → imprime `"green "`
3. `c.equals("BLUE")` → **falso**, porque `c` es un valor del enum `Colors` y `"BLUE"` es un `String`. Comparar un enum con un String con `equals` devuelve `false` sin lanzar excepción.

Por lo tanto, solo se imprimen `"red green"`.

176. ¿Cuál es el archivo generado por el comando 'apx config'?

a. settings.yaml

b. config.xml

c. pom.xml

d. settings.xml

e. config.yaml

177. ¿Cuál es la nomenclatura correcta para un componente de tipo transacción?

a. TESTT001-01-MX

b. TESTT001-MX-01

c. TESTTT01-MX-01

d. TEST01-001-MX

La respuesta correcta es:

a. TESTT001-01-MX ✅

Justificación:
En APX, la nomenclatura de una **transacción** sigue un patrón estandarizado:

- `TESTT001` → Código único del componente, donde la letra **T** indica que es una transacción.
- `01` → Número secuencial o versión del componente.
- `MX` → Indicador de país o entorno específico.

Este formato asegura consistencia y facilita la identificación de transacciones dentro de un proyecto APX.

178. Which of the following are valid ways to initialize an array in Java?

a. int[] arr = (1, 2, 3);

b. int[] arr = {1, 2, 3};

c. int[] arr = new int[] {1, 2, 3};

d. int[] arr = new int[3];

The valid ways to initialize an array in Java are:

b. `int[] arr = {1, 2, 3};` ✅
c. `int[] arr = new int[] {1, 2, 3};` ✅
d. `int[] arr = new int[3];` ✅

Explanation:

- **b)** Uses the shorthand array initializer.
- **c)** Uses the explicit `new` keyword with an initializer.
- **d)** Declares an array of length 3 with default values (0 for `int`).

- **a)** `int[] arr = (1, 2, 3);` is invalid syntax in Java.

179. ¿Cuál es la nomenclatura y la ruta de creación de un archivo que sigue el formato [UUAA]T[CODE]-[VERSION]-[COUNTRY].xml en un proyecto?

a. Ninguna opción es correcta.

b. [CODE]T[UUAA]-[VERSION]-[COUNTRY].xml y se crea en la ruta src/main/java/.

c. [UUAA]T[CODE]-[VERSION]-[COUNTRY].xmly se crea en la ruta src/main/resources/.

d. [VERSION]-[UUAA]T[CODE]-[COUNTRY].xml y se crea en la ruta src/main/resources/.

e. [COUNTRY]-[VERSION]-[CODE]T[UUAA].xml y se crea en la ruta src/test/resources/.

La respuesta correcta es:

c. `[UUAA]T[CODE]-[VERSION]-[COUNTRY].xml` y se crea en la ruta `src/main/resources/`. ✅

Justificación:

- En proyectos APX/Maven, los archivos de configuración y recursos que siguen un formato específico como `[UUAA]T[CODE]-[VERSION]-[COUNTRY].xml` se colocan en la carpeta de **resources**, ya que no son archivos de código fuente sino recursos que la aplicación necesita en tiempo de ejecución.

- La nomenclatura `[UUAA]T[CODE]-[VERSION]-[COUNTRY].xml` sigue el patrón estándar de identificación de transacciones o componentes APX.

180. What are the types of exceptions in Java?

a. Checked

b. Unchecked

c. Object

d. All the options are correct.

e. Throwable

181. Dado el siguiente código:

```
interface Omega {}

class Gamma {}

class Epsilon extends Gamma implements Omega {}

public class Test {

    public static void main(String[] args) {

        Gamma g = new Epsilon();

        Omega o = (Omega) g;  // Corrected line

        Epsilon e = (Epsilon) g; // Corrected line

        Omega m= e;

    }

}
```

¿Cuál es el resultado de la ejecución del programa?

a. An exception is thrown at runtime.

b. No option is correct.

c. Compilation fails.

d. The code runs with no output.

The correct answer is:

d. The code runs with no output. ✅

Explanation:

- `Gamma g = new Epsilon();` → `g` references an `Epsilon` object. ✅
- `Omega o = (Omega) g;` → This cast is valid because `Epsilon` implements `Omega`. ✅
- `Epsilon e = (Epsilon) g;` → This cast is also valid because `g` actually refers to an `Epsilon`. ✅
- `Omega m = e;` → Simple assignment; `Epsilon` implements `Omega`. ✅

There are **no compilation errors** and **no runtime exceptions**, and since there are no print statements, the program executes **without output**.

182. ¿Cuáles son los protocolos físicos que admite en APX?

a.SQL

b.JMS

c.Kermit

d.REST

e.HTTP

183. Which of the following are unchecked exceptions?

a. ArrayIndexOutOfBoundsException

b. IOException

c. RuntimeException

d. NumberFormatException

e. IllegalArgumentException

f. InterruptedException

The **unchecked exceptions** in Java are those that **inherit from `RuntimeException`**. Checked exceptions must be declared in a `throws` clause, while unchecked exceptions do not.

From the given options:

- **ArrayIndexOutOfBoundsException** ✅ (subclass of `RuntimeException`)
- **IOException** ❌ (checked exception)
- **RuntimeException** ✅ (the base class of all unchecked exceptions)
- **NumberFormatException** ✅ (subclass of `IllegalArgumentException` → `RuntimeException`)
- **IllegalArgumentException** ✅ (subclass of `RuntimeException`)
- **InterruptedException** ❌ (checked exception)

Answer: a, c, d, e

184. ¿Cual de las siguientes opciones de vida de Maven se encarga de compilar el código fuente del proyecto?

a. compile

b. package

c. test

d. validate

La **fase `compile`** en el ciclo de vida de Maven se encarga de **compilar el código fuente del proyecto**.

- **compile:** Compila el código fuente principal del proyecto.
- **package:** Empaqueta el proyecto compilado en un formato distribuible, como JAR o WAR.
- **test:** Ejecuta las pruebas unitarias del proyecto.
- **validate:** Valida que el proyecto esté correctamente configurado y que toda la información necesaria esté disponible.

Respuesta: a. compile

185. ¿Cuál de las siguientes herramientas incluidas en el JDK se utiliza para monitorear y gestionar aplicaciones?

a. VM

b. javac

c. jar

d. jconsole

e. java

La herramienta correcta es `jconsole`.

- **jconsole**: Es una herramienta incluida en el JDK que permite **monitorear y gestionar aplicaciones Java en tiempo de ejecución**, incluyendo uso de memoria, hilos y CPU, mediante JMX (Java Management Extensions).
- **javac**: Compila código fuente Java.
- **jar**: Empaqueta archivos en un archivo JAR.
- **java**: Ejecuta aplicaciones Java.
- **VM**: No es una herramienta específica del JDK.

Respuesta: d. jconsole

186. ¿Cuál de las siguientes herramientas proporciona análisis de calidad del código, detectando errores, vulnerabilidades y malas prácticas en el código fuente?

a. Bitbucket

b. Jenkins

c. Jira

d. SonarQube

e. Artifactory

La herramienta correcta es **SonarQube**.

- **SonarQube**: Proporciona **análisis de calidad del código**, detectando errores, vulnerabilidades, “code smells” y malas prácticas en proyectos de software.
- **Bitbucket**: Sistema de control de versiones basado en Git.
- **Jenkins**: Herramienta de integración continua y automatización de builds.
- **Jira**: Plataforma de gestión de proyectos y seguimiento de incidencias.
- **Artifactory**: Repositorio de artefactos y gestión de dependencias.

Respuesta: d. SonarQube

187. Dado el siguiente código:

```
interface I {

    void method();

}

public class C implements I {
```

```
    // declaraciones de métodos

}
```

¿Qué método puede ser declarado dentro de la clase C para que compile?

a. protected abstract void method() { }

b. private void method() { }

c. public abstract void method();

d. Ninguna de las opciones.

e. void method() { }

188. ¿Cuál de los siguientes métodos de Mockito se utilizan conjuntamente para definir el comportamiento de un mock cuando se llama a un método específico?

a. doReturn()

b. Ninguna opción es correcta.

c. doThrow()

d. when() & thenReturn()

e. verify()

En **Mockito**, para definir el comportamiento esperado de un mock al llamar a un método específico, se utiliza la combinación de **`when()`** y **`thenReturn()`** (o `thenThrow()`, según corresponda).

Ejemplo:

```
when(mockedList.get(0)).thenReturn("first");
```

- `when()` indica **qué método del mock estamos interceptando**.

- `thenReturn()` indica **qué valor debe devolver** cuando se llame ese método.

Otras opciones:

- `doReturn()` y `doThrow()` se usan en ciertos casos de métodos `void` o spies, pero no son la combinación estándar para métodos normales.
- `verify()` se usa para **verificar** que un método fue llamado, no para definir comportamiento.

Respuesta correcta: d. when() & thenReturn()

189. abstract class Animal {

public void speak() {

System.out.println("Animals make noise");

}

public static void main(String[] args) {

Animal animal = new Dog();

animal.speak();

}

}

class Dog extends Animal {

```
    protected void speak() {
        System.out.println("The Dog barks");
    }
}
public class Test {
    public static void main(String... strings) {
        Animal.main(strings);
    }
}
```

a. Compilation fails.

b. The Dog barks

c. A runtime exception is thrown.

d. Animals make noise

La respuesta correcta es: **a. Compilation fails.** ✅

Justificación:

- En Java, al sobrescribir un método de una clase padre, el modificador de acceso **no puede ser más restrictivo**.
- El método `speak()` en `Animal` es `public`.
- En `Dog`, se declara como `protected`, que es **más restrictivo que public**, lo cual **provoca un error de compilación**.

Si se intentara compilar, el compilador marcaría:

```
attempting to assign weaker access privileges; was public
```

-

190. Which of the following lines of code correctly appends a string to a StringBuilder object in Java?

a. StringBuilder sb = new StringBuilder("Hello"); sb.add(" World");

b. StringBuilder sb = new StringBuilder("Hello"); sb.append(" World");

c. StringBuilder sb = "Hello"; sb += " World";

d. StringBuilder sb = new StringBuilder("Hello"); sb.insert(" World");

La respuesta correcta es: **b.** ✅

Justificación:

- En Java, `StringBuilder` tiene el método **`append()`** para agregar contenido al final del objeto.
- Las otras opciones son incorrectas:
  - **a.** `add()` no existe para `StringBuilder`.
  - **c.** No se puede asignar un `String` directamente a un `StringBuilder` ni usar `+=`.
  - **d.** `insert()` requiere un índice como primer argumento, no solo la cadena.

191. The image shows a multiple-choice question about the java -jar command, which is used to execute Java programs packaged in JAR (Java Archive) files. The question asks which of the following statements about the command are true. The options are:

a. The java -jar command can dynamically add libraries to the classpath using the -cp or -classpath options.

b. The java -jar command ignores the CLASSPATH environment variable and command-line classpath settings.

c. The java -jar command can execute a JAR file without specifying the main class if the JAR file contains multiple main classes.

d. The java -jar command requires the JAR file to have a META-INF/MANIFEST.MF file specifying the main class.

e. The java -jar command can only be used to execute JAR files that are executable JARs, i.e., those that contain the META-INF/MANIFEST.MF file with the Main-Class attribute.

Las opciones correctas son: **b, d y e** ✅

Justificación:

- **b.** Correcto: `java -jar` **ignora el CLASSPATH** y cualquier `-cp` o `-classpath` en la línea de comandos; solo usa el classpath definido dentro del JAR (si se especifica en el `MANIFEST.MF`).
- **d.** Correcto: Para ejecutar un JAR con `java -jar`, el JAR debe tener un **`META-INF/MANIFEST.MF`** que indique la clase principal (`Main-Class`).
- **e.** Correcto: Solo los **JARs ejecutables** (con atributo `Main-Class` en el manifiesto) pueden ejecutarse con `java -jar`.

Incorrectas:

- **a.** Falso: `-cp` o `-classpath` son **ignoradas** con `java -jar`.
- **c.** Falso: Si el JAR tiene múltiples clases `main`, no se puede ejecutar sin definir cuál es la principal en el manifiesto.

192. Which two statements are true?

a. An interface cannot be extended by another interface.

b. An interface can be extended by an abstract class.

c. An abstract class cannot be extended by an abstract class.

d. An abstract class can implement an interface.

e. An abstract class can be extended by a concrete class.

The correct options are: **d** and **e** ✅

Justification:

- **d.** True: An abstract class **can implement an interface**, either fully or partially, leaving some methods abstract.
- **e.** True: An abstract class **can be extended by a concrete class**, which must implement all abstract methods.

Incorrect options:

- **a.** False: An interface **can be extended** by another interface.
- **b.** False: An abstract class **cannot extend an interface**; it can only implement it.
- **c.** False: An abstract class **can be extended** by another abstract class.

193. ¿Cuál es el propósito principal de la anotación @Test en JUnit?

a. Ninguna opción es correcta.

b. Marcar un método como un método de prueba.

c. Ejecutar un método antes de todos los métodos de prueba en una clase.

d. Ejecutar un método después de todos los métodos de prueba en una clase.

e. Ignorar un método de prueba.

La respuesta correcta es: **b. Marcar un método como un método de prueba.** ✅

Justificación:
En JUnit, la anotación `@Test` se utiliza para indicar que un método es un **caso de prueba** que debe ejecutarse como parte de la suite de pruebas. Esto permite que el framework ejecute automáticamente ese método durante la ejecución de pruebas unitarias.

- **c.** y **d.** corresponden a `@BeforeAll` y `@AfterAll`.

- **e.** corresponde a `@Disabled`.

194. Una transacción es un conjunto de operaciones que se ejecutan como un solo elemento. Una transacción debe tener propiedades ACID.

Verdadero

Falso

Verdadero ✅

Justificación:
Una **transacción** en sistemas de información es un conjunto de operaciones que se ejecutan como una unidad indivisible. Para garantizar **consistencia, confiabilidad y manejo de fallos**, una transacción debe cumplir las propiedades **ACID**:

- **Atomicidad (Atomicity):** todas las operaciones se completan o ninguna se aplica.
- **Consistencia (Consistency):** el sistema pasa de un estado válido a otro estado válido.
- **Aislamiento (Isolation):** las transacciones concurrentes no interfieren entre sí.
- **Durabilidad (Durability):** los cambios realizados por una transacción confirmada se mantienen incluso en caso de fallos.

195. ¿Cuál es la rama principal de Gitflow en la que se integran las nuevas funcionalidades antes de lanzarlas a producción?

a. feature

b. release

c. hotfix

d. develop

e. master

196. ¿Cuál de los siguientes comandos de Maven se utiliza para compilar un proyecto y crear un archivo JAR pero sin instalarlo en el repositorio local?

a. mvn install

b. mvn package

c. mvn clean

d. mvn compile

Respuesta: b. **mvn package** ✅

Justificación:

- `mvn compile`: compila el código fuente del proyecto, pero no genera el JAR.
- `mvn package`: compila y empaqueta el proyecto en un archivo JAR o WAR según la configuración del `pom.xml`, **sin instalarlo en el repositorio local**.
- `mvn install`: compila, empaqueta y además instala el artefacto en el **repositorio local de Maven**.
- `mvn clean`: elimina los archivos compilados y empaquetados previamente (limpieza del proyecto).

197. Which design pattern allows you to compose objects into tree structures to represent part-whole hierarchies, letting clients treat individual objects and compositions uniformly?

a. Builder

b. Composite

c. Singleton

d. Abstract Factory

e. Facade

Respuesta: b. **Composite** ✅

Justificación:
El **patrón Composite** permite construir estructuras de árbol donde los objetos individuales y las composiciones de objetos se tratan de manera uniforme. Esto es útil para representar jerarquías **parte-todo**, como archivos y carpetas en un sistema de archivos.

- **Builder:** Se enfoca en construir objetos complejos paso a paso.
- **Singleton:** Garantiza una única instancia de una clase.
- **Abstract Factory:** Proporciona una interfaz para crear familias de objetos relacionados sin especificar sus clases concretas.
- **Facade:** Simplifica el acceso a un conjunto de interfaces complejas proporcionando una interfaz unificada.

198. ¿Cuál es el significado del acrónimo CRUD en el contexto del desarrollo de aplicaciones?

a. Create, Read, Update, Delete.

b. Compile, Run, Update, Display.

c. Connect, Retrieve, Upload, Download.

d. Create, Retrieve, Upgrade, Delete.

Respuesta: a. **Create, Read, Update, Delete** ✅

Justificación:
CRUD es un acrónimo que representa las **cuatro operaciones básicas** que se pueden realizar sobre datos en aplicaciones de software:

- **Create:** Crear nuevos registros o datos.
- **Read:** Leer o consultar datos existentes.

- **Update:** Modificar datos existentes.
- **Delete:** Eliminar datos existentes.

Estas operaciones son fundamentales en el diseño de sistemas de gestión de bases de datos y aplicaciones que manipulan información persistente.

199. import java.util.stream.Stream;

import java.util.Objects;

class Person {

```
    public String name;
    Person(String name) {
        this.name = name;
    }
    @Override
    public int hashCode() {
        return Objects.hash(name);
    }
    @Override
```

```
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null)
            return false;
        if (getClass() != obj.getClass())
            return false;
        Person other = (Person) obj;
        return Objects.equals(name, other.name);
    }
}
public class Test {
    public static void main(String[] args) {
        Person p1 = new Person("John");
        Person p2 = new Person("Jane");
        Person p3 = new Person("Jane");
        Person p4 = new Person("Mark");
```

```
        Stream<Person> stream = Stream.of(p1, p2, p3, p4);

        long count = stream.filter(p -> p.name.startsWith("J")).distinct().count();

        System.out.println(count);

    }

}
```

What will be the output of the above code?

a. 2

b. A runtime exception is thrown

c. 0

d. 3

e. 4

f. 1

g. Compilation fails

The correct answer is: **a. 2** ✅

Explanation:

1. A `Stream` is created with four `Person` objects: `"John"`, `"Jane"`, `"Jane"`, and `"Mark"`.
2. The `filter(p -> p.name.startsWith("J"))` keeps only the persons whose names start with `"J"`, so the stream now has: `"John"`, `"Jane"`, `"Jane"`.
3. The `distinct()` method removes duplicates. Since `Person` overrides `equals()` and `hashCode()`, the two `"Jane"` objects are considered equal. After `distinct()`, the stream contains: `"John"`, `"Jane"`.
4. `count()` then counts the remaining elements: 2.

Output:

2

200. ¿Cuál es la principal función de JFrog Artifactory en un entorno de desarrollo de software?

a. Proporcionar un entorno de desarrollo integrado (IDE) para aplicaciones Java.

b. Ofrecer un sistema de control de versiones para proyectos de software.

c. Actuar como un servidor web para alojar sitios HTML y CSS.

d. Gestionar y almacenar artefactos de software, como dependencias y bibliotecas, en un repositorio.

La respuesta correcta es: **d. Gestionar y almacenar artefactos de software, como dependencias y bibliotecas, en un repositorio.** ✅

Explicación:
JFrog Artifactory es un **repositorio de artefactos** que se utiliza para almacenar, versionar y distribuir componentes de software, como **librerías, dependencias, JARs, paquetes npm, Docker images**, entre otros. Facilita la integración continua y la entrega continua (CI/CD), asegurando que los equipos de desarrollo tengan acceso a versiones controladas y confiables de los artefactos necesarios para sus proyectos.

201. Which of the following lines of code correctly defines the main method in a Java program?

a. static void main(String args[]) {}

b. public static int main(String[] args) {}

c. public static void main(String... String) {}

d. public static void main(String[] args) {}

e. private static void main(String[] args) {}

202. hat will be the result of attempting to compile and run the following code snippet?

```
public class RecursiveConstructorTest {
    private int value;
    public RecursiveConstructorTest() {
        this(10);
        System.out.println("No-argument constructor called");
    }
    public RecursiveConstructorTest(int value) {
        this();
        this.value = value;
        System.out.println("Constructor with int argument called: " + value);
    }
    public static void main(String[] args) {
        RecursiveConstructorTest obj = new RecursiveConstructorTest();
    }
}
```

a. Compilation error due to recursive constructor calls.

b. Runtime error due to infinite recursion.

c. No-argument constructor called. Constructor with int argument called: 10.

d. Constructor with int argument called: 10. No-argument constructor called.

La respuesta correcta es:

a. Compilation error due to recursive constructor calls. ✅

Explicación:

- En Java, un constructor puede llamar a otro constructor de la **misma clase** usando `this(...)`.
- Sin embargo, **no se permite la recursión infinita entre constructores**, ya que ambos constructores (`RecursiveConstructorTest()` y `RecursiveConstructorTest(int value)`) se llaman mutuamente sin una condición de terminación.
- Esto provoca un **error de compilación**, específicamente: *constructor call must be the first statement* y detecta la recursión infinita.

203. abstract class Vehicle {

public abstract void start();

public void stop() {

System.out.println("Vehicle stopped");

}

}

```
class Car extends Vehicle {
    @Override
    public void start() {
        System.out.println("Car started");
    }
}
class Bike extends Vehicle {
    @Override
    public void start() {
        System.out.println("Bike started");
    }
}
public class Test {
    public static void main(String[] args) {
        Vehicle car = new Car();
        Vehicle bike = new Bike();
        car.start();
```

```
        car.stop();

        bike.start();

        bike.stop();

    }

}
```

What will be the output of the following code?

Car started Vehicle stopped Bike started Vehicle stopped

Vehicle stopped Car started Vehicle stopped Bike started

Bike started Vehicle stopped Car started Vehicle stopped

Car started Vehicle stopped Vehicle stopped Bike started

204. ¿Cuáles son los tipos de componentes en APX Online?

a. Librerías

b. Jobs

c. Todas las opciones son correctas

d. DTOs

e. Transacciones

205. ¿Cuál es el patrón de diseño que permite a un objeto cambiar su estado interno?

State Pattern

Correcto. ✅

El **State Pattern** (Patrón de Estado) es un patrón de diseño de comportamiento que permite a un objeto **cambiar su comportamiento cuando cambia su estado interno**, como si el objeto cambiara de clase.

Explicación breve:

- Cada estado se representa mediante una clase concreta que implementa una interfaz común.
- El objeto principal delega la lógica del comportamiento a la instancia del estado actual.
- Esto permite que el objeto modifique su comportamiento en tiempo de ejecución sin necesidad de condicionales complejos.

206. Which two statements are true?

a. A class that is abstract cannot be instantiated or turned into an object (instance).

b. An interface cannot be extended by another interface.

c. If a class has abstract methods, then it must be specified as an abstract class.

d. An abstract class can be extended by an interface.

Las dos afirmaciones correctas son:

a. A class that is abstract cannot be instantiated or turned into an object (instance). ✅

- Esto es cierto: las clases abstractas **no pueden instanciarse directamente**.

c. If a class has abstract methods, then it must be specified as an abstract class. ✅

- Esto también es cierto: cualquier clase que tenga **métodos abstractos** debe declararse como `abstract`.

Explicación de las incorrectas:

- **b.** Falso: **una interfaz sí puede extender otra interfaz**.
- **d.** Falso: **una interfaz no puede extender una clase abstracta**, solo otras interfaces.

207. ¿Cuál de las siguientes herramientas es la más adecuada para el seguimiento de problemas y la gestión de proyectos en entornos de desarrollo de software?

a. Jenkins

b. Jira

c. Artifactory

d. SonarQube

e. Bitbucket

La respuesta correcta es:

b. Jira ✅

Justificación:
Jira es una herramienta de **gestión de proyectos y seguimiento de incidencias** ampliamente utilizada en entornos de desarrollo de software. Permite crear y asignar tareas, planificar sprints, llevar control de bugs, gestionar historias de usuario y generar reportes del progreso del proyecto.

Explicación de las demás opciones:

- **Jenkins:** herramienta de integración y entrega continua (CI/CD), no gestión de proyectos.
- **Artifactory:** gestor de artefactos y dependencias.
- **SonarQube:** análisis de calidad de código y detección de vulnerabilidades.
- **Bitbucket:** sistema de control de versiones basado en Git.

208. ¿Cuál es el objetivo de la clase abstracta de la transacción APX?

a. Recuperar información de solicitud HTTP.

b. Agregar parámetros del cuerpo de la respuesta.

c. Todas las opciones son correctas.

d. Unir las operaciones relacionadas con los parámetros de entrada y salida de la lógica de la transacción.

e. Separar las operaciones relacionadas con los parámetros de entrada y salida de la lógica de la transacción.

209. ¿Cuál es el propósito del plugin maven-surefire-plugin y en qué etapa del ciclo de vida de Maven se ejecuta principalmente?

a. Instalar el artefacto en el repositorio local durante la etapa 'install'.

b. Empaquetar el proyecto durante la etapa 'package'.

c. Ejecutar pruebas unitarias durante la etapa de 'test'.

d. Generar la documentación del proyecto durante la etapa de 'site'.

La respuesta correcta es:

c. Ejecutar pruebas unitarias durante la etapa de 'test'. ✅

Justificación:
El **plugin** `maven-surefire-plugin` es el encargado de ejecutar las **pruebas unitarias** definidas en un proyecto Maven. Su propósito principal es:

- Detectar y ejecutar automáticamente todos los métodos de prueba que sigan las convenciones de JUnit o TestNG.
- Integrar la ejecución de pruebas en el ciclo de vida de Maven, normalmente durante la fase `test`.

- Generar reportes de resultados de pruebas para revisión y seguimiento de calidad del código.

Otros plugins se encargan de empaquetar, instalar o generar documentación, pero **Surefire se**

210. ¿Cuál es el propósito principal de OSGi (Open Service Gateway Initiative) en el desarrollo de software moderno?

a. Gestionar dependencias y permitir el despliegue dinámico de módulos en aplicaciones Java.

b. Proporcionar un entorno de desarrollo integrado (IDE) para aplicaciones Java.

c. Servir como un sistema de control de versiones para proyectos de software.

d. Actuar como un servidor de aplicaciones para ejecutar aplicaciones empresariales Java.

La respuesta correcta es:

a. Gestionar dependencias y permitir el despliegue dinámico de módulos en aplicaciones Java. ✅

Justificación:
OSGi es un **framework modular para Java** que permite:

- Dividir una aplicación en **módulos independientes llamados bundles**.
- Gestionar **dependencias entre estos bundles** de manera controlada.
- Realizar **despliegue, actualización y resolución de módulos en tiempo de ejecución** sin necesidad de reiniciar la aplicación.

No es un IDE, ni un sistema de control de versiones, ni un servidor de aplicaciones completo, aunque puede integrarse dentro de servidores o entornos más grandes.

211. ¿Cuál será el resultado del siguiente fragmento de código?

```
class Animal {
    public void makeSound() {
        System.out.println("Animal sound");
    }
}
class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Bark");
    }
}
public class Test {
    public static void main(String[] args) {
        Animal myDog = new Dog();
        myDog.makeSound();
    }
}
```

a. Animal sound

b. Error de ejecución

c. Error de compilación

d. Bark

La respuesta correcta es:

d. Bark ✅

Justificación:

- La variable `myDog` es de tipo `Animal`, pero el objeto asignado es de tipo `Dog`.
- En Java, **la llamada a métodos se resuelve en tiempo de ejecución mediante *polimorfismo***.
- Como `Dog` sobrescribe el método `makeSound()`, se ejecutará la versión de `Dog`, imprimiendo `"Bark"`.

212. What are the modifiers that can precede a toplevel class declaration?

a. protected

b. abstract

c. strictfp

d. static

e. private

f. public

g. All the options are correct.

h. final

The correct answers are:

b. abstract
c. strictfp
f. public
h. final ✅

Justification:

- **Top-level classes** (i.e., classes not nested inside another class) **cannot** be `private`, `protected`, or `static`.
- Allowed modifiers for top-level classes in Java are:
    - `public` → makes the class accessible from any other class.
    - `abstract` → indicates the class cannot be instantiated.
    - `final` → prevents the class from being subclassed.
    - `strictfp` → enforces strict floating-point calculations.